



Modelos Analíticos para Criação de Gémeos Digitais

Tipo de Documento Deliverable

Número do Documento D2

Título do Documento Modelos Analíticos para Criação de Gémeos Digitais

Autor(es) Principal(is) Joaquim Ferreira | Instituto de Telecomunicações

Versão / Estado do Documento 2 | Final

Data 30/07/2026

Nível de Distribuição PU (public)



arte



INFORMAÇÕES GERAIS

DETALHES DO PROJETO

Acrónimo do Projeto	DT4MOB
Título do Projeto	Digital Twins for Mobility
Site do Projeto	https://dt4mob.av.it.pt/
Coordenador	Instituto de Telecomunicações
Referência do Projeto	2025.00153.DT4ST

CONTRIBUIDORES

Nome	Organização
aquim Ferreira	Instituto de Telecomunicações
Diogo Gomes	Instituto de Telecomunicações
Rui Aguiar	Instituto de Telecomunicações
João Almeida	Instituto de Telecomunicações
Mário Antunes	Instituto de Telecomunicações
Cecília Santos	Instituto de Telecomunicações
Paulo Macedo	Instituto de Telecomunicações
João Capucho	Instituto de Telecomunicações
Zakhar Kruptsala	Instituto de Telecomunicações
IGor Coelho	Instituto de Telecomunicações
Tomás Victal	Instituto de Telecomunicações
Bernardo Figueiredo	Instituto de Telecomunicações
Tiago Alves	Instituto de Telecomunicações
Vinicius Barbon	Brisa Gestão de Infraestruturas
Tiago Santos	Brisa Gestão de Infraestruturas
Paulo Barros	Brisa Gestão de Infraestruturas
Ivo Romeiro	Brisa Gestão de Infraestruturas
João Costa	Infraestruturas de Portugal
Vasco Gonçalves	Infraestruturas de Portugal
José Abreu	Infraestruturas de Portugal
Tiago Dias	A-to-Be
Luis Pedro Silva	A-to-Be
Ana V. Silva	A-to-Be

HISTÓRICO DE ALTERAÇÕES

Revisão	Data	Autor / Organização	Descrição da Revisão	Aprovado
0.1	13/02/2026	Joaquim Ferreira / Instituto de Telecomunicações	Draft	Yes
1.0	30/05/2026	Joaquim Ferreira / Instituto de Telecomunicações	Final	Yes
2.0	21/07/2026	Joaquim Ferreira / Instituto de Telecomunicações	Final revised and updated	Yes

Financiado pela União Europeia - NextGenerationEU. No entanto, os pontos de vista e opiniões expressos são exclusivamente do(s) autor(es) e não refletem necessariamente os pontos de vista e opiniões da União Europeia ou da Comissão Europeia. Nem a União Europeia nem a Comissão Europeia são responsáveis por eles.

1	DESCRIÇÃO DOS MODELOS ANALÍTICOS	10
1.1	CASO DE USO: OTIMIZAÇÃO DE PORTAGENS E SUPORTE À CONDUÇÃO AUTÓNOMA	10
1.1.1	Fontes de dados.....	11
1.1.2	Formatos de dados	12
1.1.3	Gantry Service.....	20
1.1.4	Serviços de conversão de dados de sensores.....	25
1.1.5	Simulação de Faixa Reservada A5	27
1.2	CASO DE USO: MONITORIZAÇÃO DE MURO DE SUPORTE E TALUDE	30
1.2.1	Fontes de dados.....	31
1.2.2	Formatos de dados	33
1.2.3	Data Bridge e modelos de dados.....	36
1.2.4	Serviço de carregamento LevelAMS	41
1.2.5	Resultados	45
1.3	CASO DE USO: GESTÃO DE AUTOESTRADAS RESILIENTES ÀS ALTERAÇÕES CLIMÁTICAS ATRAVÉS DA PREVISÃO DE RISCO DE INCÊNDIO 51	
1.3.1	Motor de simulação: ForeFire.....	51
1.3.2	Fluxo de trabalho proposto	52
1.3.3	Fontes de dados.....	54
1.3.4	Modelo de Rothermel.....	57
1.3.5	Arquitetura de software proposta.....	57
1.3.6	Resultado e interoperabilidade	59
1.4	CASO DE USO – GESTÃO DE CIRCULAÇÃO E OPERAÇÕES DE TRÁFEGO	59
1.4.1	Fontes de dados.....	60
1.4.2	Contadores de tráfego e LiDARs	60
1.4.3	Formatos de dados	61
1.4.4	Monitorização de contadores de tráfego	61
1.4.5	Modelos analíticos.....	62
1.5	CASO DE USO – GESTÃO DE ATIVOS RODOVIÁRIOS	68
1.5.1	Fontes de dados.....	68
1.5.2	Mapeamento móvel e LidaR de UAV	69
1.5.3	Formatos de dados	69
2	MODELOS DE DADOS.....	70
2.1	MODELOS DO DATA BRIDGE.....	70
2.1.1	Modelos de dados geográficos.....	70
2.1.2	Modelos de dados meteorológicos.....	70
2.1.3	Modelos de dados de tráfego.....	72
2.1.4	Modelos de dados de características rodoviárias.....	73
2.2	PESQUISAS GEOGRÁFICAS.....	74
	SIMULAÇÃO DE FAIXA RESERVADA A5	76
3	INSTRUÇÕES DE INSTALAÇÃO E EXECUÇÃO	77
3.1	SERVIÇOS DE CONVERSÃO DE DADOS DE SENSORES	77
3.2	GANTRY SERVICE.....	77
3.3	LEVELAMS LOADER.....	77
3.4	MÓDULO DATA BRIDGE	77
3.5	MÓDULO TEXTURE CREATION	78
3.6	MÓDULO METEO POPULATOR.....	78
3.7	NAMESPACE REMOVER	78
3.8	SIMULAÇÃO DE FAIXA RESERVADA A5	78
3.9	SIMULAÇÃO VCI	79
3.10	MODELOS DE PREVISÃO DE TRÁFEGO	79
3.11	MODELO DE AGENTES	79
4	CAMADA DE VISUALIZAÇÃO	80

4.1	VISUALIZAÇÃO	80
4.1.1	Arquitetura do sistema	80
4.1.2	Aquisição de dados	81
4.1.3	Conversão de coordenadas geográficas	82
4.1.4	Representação de entidades	82
4.1.5	Sistema de navegação e câmara	87
4.1.6	Gestores e lógica de aplicação	87
4.1.7	Interface do utilizador	88
4.1.8	Fluxo de interação do utilizador	89
5	FERRAMENTAS, LINGUAGENS E BIBLIOTECAS UTILIZADAS	90
5.1	PLATAFORMA DIGITAL TWINS	90
5.1.1	Segurança	90
5.1.2	Monitorizarização	91
5.1.3	Serviços	91
5.2	DATA BRIDGE	96
5.3	METEO POPULATOR	97
5.4	TEXTURE CREATION	97
5.5	NAMESPACE REMOVER	98
5.6	GANTRY SERVICE	98
5.7	SERVIÇO DE CONVERSÃO	99
5.8	SCRIPT DE CARREGAMENTO DA API LEVELAMS	100
5.9	SERVIÇO DE SIMULAÇÃO DE INCÊNDIOS	101
5.10	HISTORICAL DATA API	101
5.11	HISTORICAL DATA WRITER	102
5.12	BASE DE DADOS DE DADOS HISTÓRICOS	102
5.13	GARBAGE COLLECTOR	102
5.14	IMPLEMENTAÇÃO E ORQUESTRAÇÃO	102
5.15	SIMULAÇÃO DA FAIXA RESERVADA A5	103
5.16	SIMULAÇÃO VCI	103
5.17	MODELOS DE PREVISÃO DE TRÁFEGO	103
5.18	MODELO DE AGENTES	104
6	MÉTRICAS DE QUALIDADE	105
6.1	SIMULAÇÃO DE TRÁFEGO VCI	106
6.2	MODELOS DE ML DE TRÁFEGO	106
6.3	SIMULAÇÃO DA FAIXA RESERVADA A5	107
6.4	SIMULAÇÃO DE PROPAGAÇÃO DE INCÊNDIOS	108
7	LICENÇAS UTILIZADAS	110

LISTA DE FIGURAS

Figura 1 - Arquitetura de Portagem em Via Aberta.....	10
Figura 2 - Esquema JSON para o Formato ORT Tracking.....	14
Figura 3 - Coordenadas de referência para as mensagens JSON.....	14
Figura 4 - Exemplo da utilização do ponto de referência para um veículo	15
Figura 5 - Diagrama de classes das mensagens JSON publicadas no broker MQTT.....	16
Figura 6 - Exemplo JSON para o tópico vehicle/realtime-tracking	16
Figura 7 - Exemplo JSON para o tópico vehicle/history-tracking.....	18
Figura 8 - Formato de mensagens OutSight.....	18
Figura 9 - Exemplo JSON para o tópico OutSight/vehicle/realtime-tracking.....	18
Figura 10 - Exemplo JSON para o tópico OutSight/vehicle/history-tracking.....	19
Figura 11 - Exemplo JSON para o tópico tzc/vehicle/realtime-tracking	20
Figura 12 - Captura de ecrã do SUMO GUI do segmento da autoestrada A5 simulada	29
Figura 13 - Captura de ecrã do SUMO da interface da simulação de tráfego em execução. Veículos (triângulos) e rede viária colorida por velocidade em m/s (à esquerda a legenda de cor dos veículos e à direita a legenda de cor das faixas com base na velocidade média em m/s).....	29
Figura 14 - Localização do Muro e Sensores	30
• Figura 15 - Exemplo de um gráfico das células de carga.....	33
Figura 16 - Exemplo de gráfico de inclinómetros.....	34
Figura 17 - Resultados de deslocamento medidos em metros (m).	46
Figura 18 - Identificação de perfis no muro para uma análise mais detalhada: A) Imagem 3D; B) pontos de validação	47
Figura 19 - Painel de monitorização de células de carga com o instrumento A19 (em estado OK) selecionado.....	48
Figura 20 - Painel de monitorização de células de carga com o instrumento A35 (em estado ALERT) selecionado.....	49
Figura 21 - Painel de monitorizarização de inclinómetros com o instrumento I2C (em estado OK) selecionado.....	50
Figura 22 - Painel de monitorização de inclinómetros com o instrumento I4 (em estado ALARM) selecionado.....	50
Figura 23 - Cálculo do cone de fogo.....	53
Figura 24 - Fluxo proposto: desde a entrada dinâmica de incidentes e análise espacial imediata até ao pipeline de extração e simulação ForeFire	54

Figura 25 - Arquitetura proposta para simulação de incêndios	58
Figura 26 - Visualização resultante da simulação de incêndios	59
Figura 27 - Painel de monitorização de contadores de tráfego com IP_Sensor_722 selecionado (intervalos de 15 minutos).....	62
Figura 28 - Captura de ecrã do SUMO gui da rede VCI modelada	63
Figura 29 - Captura de ecrã do SUMO gui de detalhe da simulação de tráfego, veículos (amarelo) e detetores (caixas amarelas) estão visíveis.	64
Figura 31 - Modelo de dados do veículo.....	65
Figura 30 - Diagrama do agente coordenador.....	68
Figura 32 - Modelo das localizações geográficas.....	70
Figura 33 - Modelo de dados meteorológicos.....	71
Figura 34 - Modelo de dados de tráfego.....	73
Figura 35 - Modelo de dados de características rodoviárias	74
Figura 38 - Exemplo de Informação de Barreira	83
Figura 39 - Exemplo de Informação de Estação Meteorológica	84
Figura 40 - Exemplo de Informação de Talude.....	84
Figura 41 - Exemplo de Informação de Equivia Pavimentos	85
Figura 42 - Exemplo de Informação de Equivia Integração Paisagística	85
Figura 43 - Exemplo de Informação de Equivia Iluminação	86
Figura 44 - Exemplo de Informação de Equivia Taludes	86
Figura 45 - Definição da tabela da base de dados	92
Figura 46 - Exemplo de evento presente na base de dados	93
Figura 47 - Interação dos componentes com a infraestrutura	94
Figura 48 - Especificação OpenAPI da Historical Data API.....	96
Figura 49 - Diagrama de implementação e componentes de software adotados.....	110

LISTA DE TABELAS

Tabela 1 - Conversão para coordenadas locais	15
Tabela 2 - Mensagem de Histórico	21

Tabela 3 - Caminho do Item	21
Tabela 4 - localCoordinates	21
Tabela 5 - absoluteCoordinates	21
Tabela 6 - dimensions	22
Tabela 7 - Mensagem em Tempo Real	22
Tabela 8 - OutSightMessage	22
Tabela 9 – Dados	22
Tabela 10 - Alerta	23
Tabela 11 - BoundingSquare	23
Tabela 12 - Definição de um Toll Thing	24
Tabela 13 - Definição de um Thing de Objeto detetado	25
Tabela 14 - OutSightToAToBeConfig	26
Tabela 15 - ReferencePlane	26
Tabela 20 - API LevelAMS	40
Tabela 21 - Ativo Geotécnico	42
Tabela 22 - Instrumento	42
Tabela 23 - Parâmetro (definição do instrumento)	43
Tabela 24 - Limite	43
Tabela 25 - ValueNamePair	43
Tabela 26 - Parameter (reading de medição)	43
Tabela 27 - Definição de um Thing que Representa um Ativo Geotécnico	44
Tabela 28 - Avaliação do MonitorStateEnum	45
Tabela 29 - Comparação entre ForeFire e Cell2Fire	52
Tabela 30 - Diferentes definições de combustível no ForeFire	56
Tabela 31 - Dados Meteorológicos	72
Tabela 32 - Representação do gémeo digital de Tráfego	73
Tabela 33 - Endpoints da Historical Data API	95
Tabela 30 - Ferramentas utilizadas no desenvolvimento do Gantry Service	99

Tabela 31 - Ferramentas utilizadas no desenvolvimento do serviço de conversão de dados de sensores	100
Tabela 32 - Ferramentas utilizadas no desenvolvimento do LevelAMS Loader	101
Tabela 33 - Resultados da simulação selecionada	106
Tabela 34 - Resultados da simulação selecionada	107
Tabela 35 - Desempenho no test set.....	107
Tabela 36 - Robustez do ForeFire	109

1 DESCRIÇÃO DOS MODELOS ANALÍTICOS

1.1 Caso de uso: otimização de portagens e suporte à condução autónoma

A A-to-Be dispõe de várias soluções de portagem que se baseiam em conhecimento adquirido ao longo dos anos para diferentes tipos de sensores. Para o DT4Mob, foram escolhidas três soluções ou protótipos de portagem, conforme descrito no Entregável 1, secção 6.3. Estas soluções utilizam apenas sensores instalados acima do solo — nomeadamente câmaras, leitores de etiquetas DSRC, LiDARs 2D e LiDARs 3D — montados em pórticos sobre a autoestrada.

O local escolhido para o DT4Mob é o local de teste de Portagem em Via Aberta (ORT) da A-to-Be, na autoestrada A5, ao quilómetro 12, no sentido de tráfego Lisboa→Cascais (ver Entregável 1, Figura 3). Neste local, a via tem 3 faixas e um elevado volume de tráfego que aumenta significativamente durante as horas de ponta da manhã e da tarde. Isto garantirá a captação de uma ampla gama de categorias de veículos, velocidades e comportamentos de condução, como ultrapassagens ou mudanças de faixa.

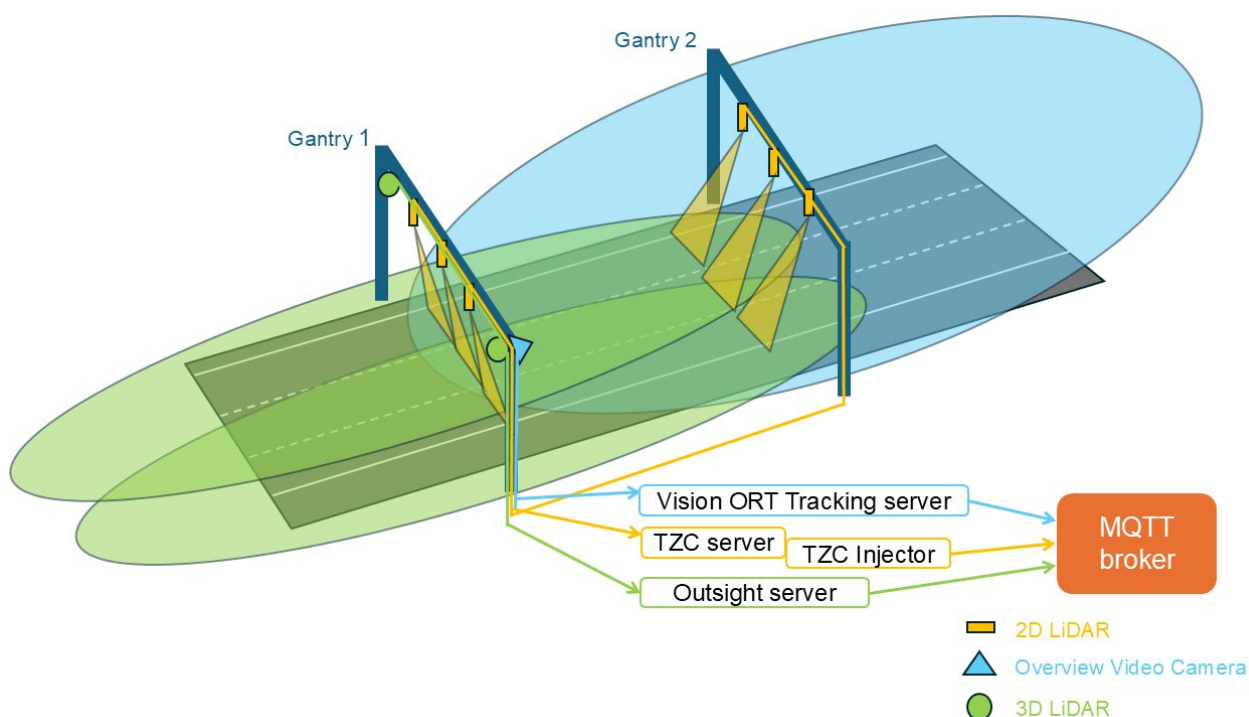


Figura 1 - Arquitetura de Portagem em Via Aberta.

Os sensores estão distribuídos por dois pórticos. O primeiro pórtico, de acordo com o sentido do tráfego, será designado por "Pórtico 1" e o segundo por "Pórtico 2". Os dois LiDARs 3D disponíveis estão ambos montados no Pórtico 1 e cobrem a área circundante desse pórtico. A Câmara de Vídeo de Visão Geral também está instalada no Pórtico 1, do lado direito da via, apontada para a frente, de modo que o seu campo de visão fique centrado no Pórtico 2. Por fim, os sensores LiDAR 2D estão montados em ambos os pórticos, mas com configurações diferentes. No Pórtico 1, existe um LiDAR 2D no centro de cada faixa, orientado perpendicularmente à via. No Pórtico 2, existem dois LiDARs 2D no centro de cada faixa, um orientado perpendicularmente à via e o outro transversalmente. Esta configuração emparelhada permite o cálculo preciso da velocidade e do comprimento de cada veículo.

As soluções de portagem da A-to-Be recolhem dados em bruto destes diferentes sensores e processam-nos para produzir dois tipos de informação. O primeiro é uma deteção em tempo real dos veículos dentro da área de cobertura do sensor. Esta informação é essencial para acionar as câmaras e captar imagens de matrículas em tempo real (as câmaras de reconhecimento de matrículas foram omitidas da figura para evitar confusão visual, uma vez que não são utilizadas no DT4Mob). O segundo é a obtenção de um histórico agregado para cada veículo. Isto garante uma forte garantia de cobrança correta da portagem, mesmo quando um ou mais sensores ou câmaras estão obstruídos ou a funcionar incorretamente.

Os dados brutos recolhidos não são relevantes para o Gémeo Digital (DT); são os dados processados provenientes destes três sistemas que alimentam o DT. Para o conseguir, cada sistema publica mensagens JSON num broker MQTT à medida que a informação se torna disponível — tanto as deteções em tempo real como os dados de histórico do veículo.

Os servidores que suportam cada solução são normalmente instalados em armários junto à via, de forma a minimizar a latência de rede, uma vez que o acionamento de câmaras em tempo real é um requisito. No entanto, como dois dos três sistemas envolvidos são protótipos e ainda não estão envolvidos na captação de imagens de matrículas, apenas alguns dos servidores permanecem junto à via, enquanto outros estão instalados nas instalações próximas da A-to-Be. Esta configuração melhora a acessibilidade para operações de manutenção, uma vez que o acesso aos armários junto à via envolve riscos de segurança adicionais. O broker MQTT também se encontra nas instalações da A-to-Be.

1.1.1 Fontes de dados

1.1.1.1 Vision ORT Tracking

O Vision ORT Tracking da A-to-Be é uma solução interna da A-to-Be, existente antes do projeto DT4Mob. É uma solução de aprendizagem automática baseada em visão que utiliza vídeo (nomeadamente streams de vídeo em direto) para identificar, segmentar, seguir e medir veículos em movimento. Fornece uma solução flexível que pode ser utilizada com câmaras de vídeo para recolher rapidamente dados sobre a posição em tempo real do veículo, as suas dimensões (3D — altura, comprimento e largura) e a velocidade. Esta solução apresenta algumas limitações, nomeadamente uma precisão dimensional inferior à das soluções baseadas em LiDAR, devido às limitações de resolução das câmaras de vídeo utilizadas.

Para o projeto DT4Mob, a solução Vision ORT Tracking foi configurada para uma câmara de visão geral no local de teste ORT do A5 Lagoas Park (ver Entregável 1, secção 6.3).

1.1.1.2 OutSight

A A-to-Be e a Outsight colaboraram na construção de um protótipo de Portagem em Via Aberta (ORT) de pórtico único, baseado em LiDARs 3D, que foi adotado como fonte de dados para o projeto DT4MOB. Os LiDARs 3D estão colocados em lados opostos da via e, em conjunto, cobrem uma área contínua alargada (ver Figura 4 do D1). Os dados de ambos os sensores são processados por um sistema central que funde os dados individuais de cada sensor numa visão unificada que permite o seguimento contínuo de cada veículo à medida que se aproxima, passa sob o pórtico e se afasta. Para cada veículo, o sistema regista a sua largura, altura, comprimento e velocidade, bem como as suas coordenadas em diferentes instantes temporais ao longo da sua passagem.

Para o projeto DT4Mob, o protótipo Outsight foi configurado com dois LiDARs 3D Ouster OS1 128° montados no local de teste ORT do A5 Lagoas Park (ver Entregável 1, secção 6.3).

1.1.1.3 Toll Zone Controller (TZC)

O Toll Zone Controller da A-to-Be faz parte do Sistema de Gestão de Portagens (TMS) da A-to-Be e é um agregador das entradas provenientes de vários dispositivos junto à via para ORT. O TZC é uma solução versátil que pode funcionar com diferentes tipos de sensores; para o projeto DT4Mob, optou-se por utilizar LiDARs 2D. Esta é a solução que fornece mais informação, nomeadamente a largura, altura, comprimento e velocidade do veículo, a sua posição dentro da via e o instante de deteção.

O sistema TZC foi configurado como uma solução ORT de dois pórticos, o que significa que existem dois pontos de deteção separados por aproximadamente 12 metros (ao longo do sentido do tráfego). Ambas as deteções são convertidas em mensagens JSON e enviadas para o broker MQTT.

No primeiro pórtico, está instalado um único sensor LiDAR 2D por faixa. Isto permite medir a largura, a altura, a posição na faixa e o instante de deteção de cada veículo. No segundo pórtico, estão instalados dois sensores LiDAR 2D por faixa. Esta configuração de sensor duplo fornece um perfil completo do veículo, adicionando o comprimento e a velocidade aos campos anteriormente indicados.

Para o projeto DT4Mob, o TZC foi configurado no primeiro e segundo pórticos do local de teste ORT do A5 Lagoas Park (ver Entregável 1, secção 6.3, Figura 4).

1.1.2 Formatos de dados

1.1.2.1 Formato ORT Tracking

O formato ORT Tracking é um formato de mensagem JSON definido pela A-to-Be, utilizado para transmitir informação em tempo real sobre veículos que circulam num troço de via, tal como percecionado pelos sensores da infraestrutura. No projeto DT4Mob, este formato é utilizado para os dados recolhidos pelas soluções Vision ORT Tracking e TZC descritas nas subsecções anteriores. Esta secção descreve o formato de dados, enquanto as 3 secções seguintes explicam como é feita a conversão a partir de cada fonte de dados.

Este formato pode ser utilizado para dois tipos de mensagens: uma deteção pontual única ou uma coleção de deteções. O primeiro tipo pode ser utilizado para o momento em que um veículo é detetado. O segundo tipo é uma coleção de todas as deteções de um veículo agregadas por essa solução ORT específica. A mensagem JSON inclui um array *path*: para deteções pontuais únicas, existe um único ponto neste array, ou esse ponto pode substituir todo o array; para coleções de deteções, cada entrada deste array conterá a informação de cada deteção individual.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",

  "definitions": {
    "timestamp": {
      "type": "string",
      "format": "date-time",
      "pattern": "^\\d{4}-\\d{2}  \\d{2}T\\d{2}:\\d{2}:\\d{2}\\.\\d{1,6}Z$"
    }
  },
}
```

```

"objectbase": {
  "type": "object",
  "properties": {
    "objectID": { "type": "integer" },
    "triggeringtimestamp":{"$ref": "#/definitions/timestamp"}
  },
  "obrigatório": ["objectID"]
},

"coordinatereference": {
  "type": "string",
  "enum":["right_rear_bottom_corner", "right_front_bottom_corner"],
  "default": "right_rear_bottom_corner"
},

"pathitem": {
  "type": "object",
  "properties": {
    "timeofmeasurement": {
      "$ref": "#/definitions/timestamp"
    },
    "speedkmh": {
      "type": "number"
    },
    "coordinatesreference": {
      "$ref": "#/definitions/coordinatereference"
    },
    "localcoordinates": {
      "type": "object",
      "properties": {
        "x": { "type": "number" },
        "y": { "type": "number" }
      },
      "obrigatório": ["x", "y"]
    },
    "absolutecoordinates": {
      "type": "object",
      "properties": {
        "latitude": { "type": "number" },
        "longitude": { "type": "number" }
      },
      "obrigatório": ["latitude", "longitude"]
    },
    "dimensions": {
      "type": "object",
      "properties": {
        "width": { "type": "number" },
        "length": { "type": "number" },
        "height": { "type": "number" }
      }
    }
  },
  "obrigatório": [
    "timeofmeasurement",
    "localcoordinates",
    "absolutecoordinates",
    "dimensions"
  ]
}
},

```

```

"oneof": [

  // Case 1: Multi-posição object (original structure)
  {
    "allof": [
      { "$ref": "#/definitions/objectbase" },
      {
        "type": "object",
        "properties": {
          "path": {
            "type": "array",
            "items": { "$ref": "#/definitions/pathitem" },
            "minitems": 1
          }
        },
        "obrigatório": ["path"]
      }
    ]
  },

  // Case 2: Single-posição flattened object
  {
    "allof": [
      { "$ref": "#/definitions/objectbase" },
      { "$ref": "#/definitions/pathitem" }
    ]
  }
]
}

```

Figura 2 - Esquema JSON para o Formato ORT Tracking

É importante definir a referência para as coordenadas na mensagem JSON. Na mensagem, utilizamos coordenadas locais e absolutas. Enquanto as coordenadas absolutas são apenas a latitude e longitude geográficas, as coordenadas locais requerem um eixo local definido da seguinte forma:

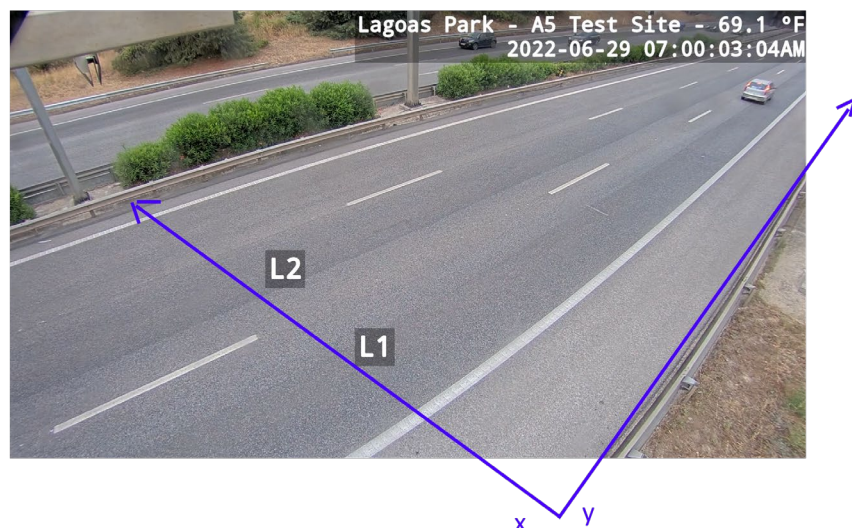


Figura 3 - Coordenadas de referência para as mensagens JSON

O eixo dos y está alinhado com o primeiro pórtico, e o eixo dos y é paralelo à via, de acordo com o sentido do tráfego. Qualquer veículo a aproximar-se do primeiro pórtico terá valores negativos de y . O segundo pórtico está localizado 12 metros após o primeiro, pelo que os veículos sob esse

pórtico terão $y=12$. No local existe uma faixa de berma, com 2 metros de largura, que não está coberta, e 3 faixas, cada uma com 3,5 metros de largura. Isto significa que, por exemplo, a faixa mais à direita começa em $x=2$ e vai até $x=5,5$.

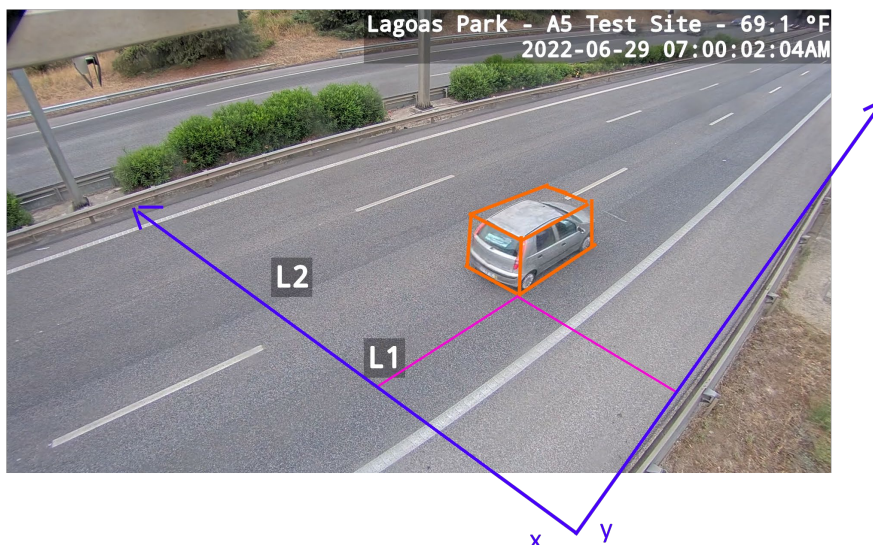


Figura 4 - Exemplo da utilização do ponto de referência para um veículo

O ponto de referência selecionado para o veículo é o ponto traseiro direito. Este valor por defeito está definido no esquema como "right_rear_bottom_corner", mas existe também a opção de utilizar o ponto frontal direito. Cada solução, ou até cada sensor, pode utilizar a referência mais adequada aos dados que estão a ser recolhidos.

Para converter as *localCoordinates* em coordenadas geográficas, foram utilizados os seguintes pontos:

x	y	latitude	longitude
0	0	38.711330	-9.309163
0	12	38.711274	-9.309273
12.5	0	38.7112361	-9.3090750
12.5	12	38.711179	-9.309191

Tabela 1 - Conversão para coordenadas locais

As mensagens JSON serão publicadas num broker MQTT em dois tópicos diferentes, dependendo do tipo de mensagem. O tópico *realtime-tracking* será utilizado para mensagens de deteção imediata ou deteções pontuais únicas, enquanto o tópico *history-tracking* será utilizado para as mensagens que reúnem toda a informação relativa à passagem de um veículo.

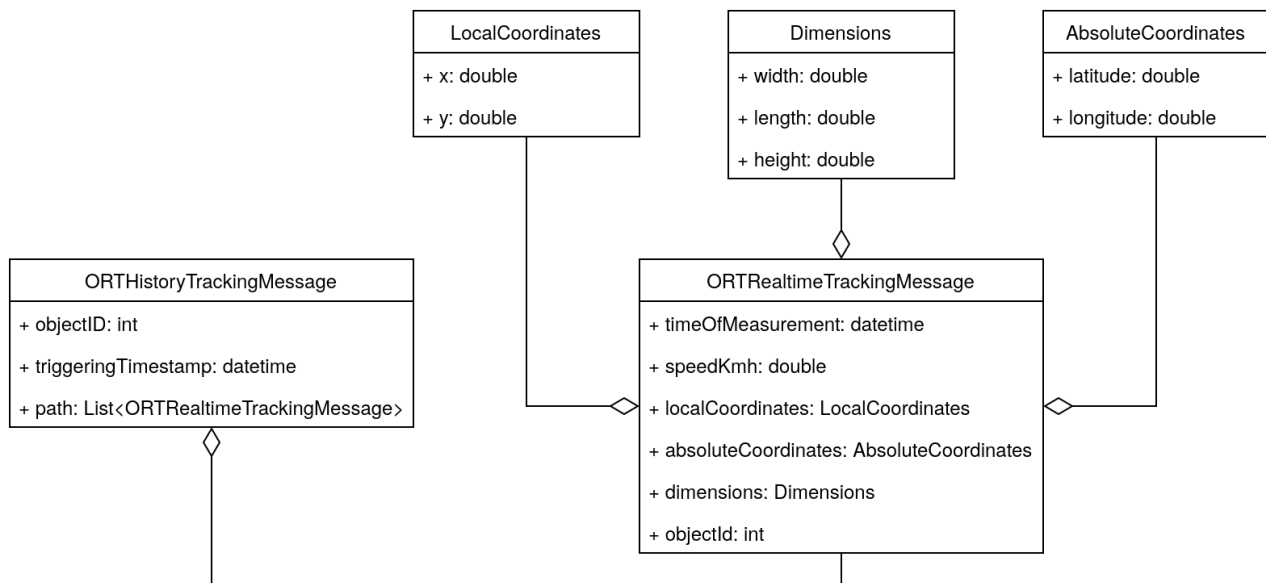


Figura 5 - Diagrama de classes das mensagens JSON publicadas no broker MQTT.

1.1.2.1.1 Mensagens Vision ORT Tracking

As mensagens JSON são publicadas no broker MQTT sob os tópicos: *vehicle/realtime-tracking* e *vehicle/history-tracking*.

```

{
  "absolutecoordinates": {
    "latitude": 38.7111741007343,
    "longitude": -9.30929791776754
  },
  "dimensions": {
    "height": 2.28,
    "length": 5.921,
    "width": 2.7
  },
  "localcoordinates": {
    "x": 5.62829808061388,
    "y": 19.5431703701857
  },
  "objectId": 745601,
  "speedkmh": 2,
  "timeofmeasurement": "2026-03-04T14:56:07.223Z"
}
  
```

Figura 6 - Exemplo JSON para o tópico *vehicle/realtime-tracking*

```

{
  "objectId": 745601,
  "path": [
    {
      "absolutecoordinates": {
        "latitude": 38.7111741007343,
        "longitude": -9.30929791776754
      },
      "dimensions": {
        "height": 2.28,
        "length": 5.921,
        "width": 2.7
      }
    }
  ]
}
  
```

```

        "localcoordinates": {
            "x": 5.62829808061388,
            "y": 19.5431703701857
        },
        "objectID": 745601,
        "speedkmh": 2,
        "timeofmeasurement": "2026-03-04T14:56:07.223Z"
    },
    {
        "absolutecoordinates": {
            "latitude": 38.711156446518,
            "longitude": -9.30933491666725
        },
        "dimensions": {
            "height": 2.28,
            "length": 6.015,
            "width": 2.728
        },
        "localcoordinates": {
            "x": 5.51864400541034,
            "y": 23.2712891769875
        },
        "objectID": 745601,
        "speedkmh": 2,
        "timeofmeasurement": "2026-03-04T14:56:07.310Z"
    },
    {
        "absolutecoordinates": {
            "latitude": 38.71114069703,
            "longitude": -9.30936935719724
        },
        "dimensions": {
            "height": 2.295,
            "length": 6.259,
            "width": 2.645
        },
        "localcoordinates": {
            "x": 5.35258148528284,
            "y": 26.6894272791604
        },
        "objectID": 745601,
        "speedkmh": 2,
        "timeofmeasurement": "2026-03-04T14:56:07.424Z"
    },
    {
        "absolutecoordinates": {
            "latitude": 38.71112475077,
            "longitude": -9.30940325948298
        },
        "dimensions": {
            "height": 2.295,
            "length": 6.259,
            "width": 2.645
        },
        "localcoordinates": {
            "x": 5.23055061163541,
            "y": 30.0879420983343
        },
        "objectID": 745601,
        "speedkmh": 2,
        "timeofmeasurement": "2026-03-04T14:56:07.535Z"
    }
}

```

```

    ],
    "triggeringtimestamp": "2000-01-20T06:38:08.254Z"
  }

```

Figura 7 - Exemplo JSON para o tópico vehicle/history-tracking

1.1.2.2 Mensagens OutSight

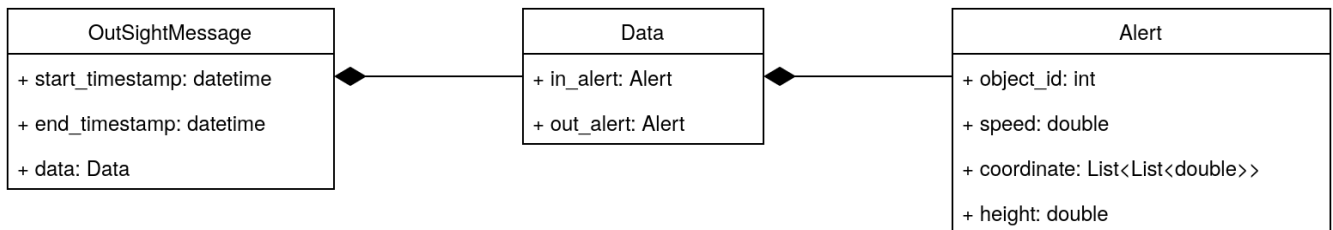


Figura 8 - Formato de mensagens OutSight

Este formato é convertido para o formato de Portagem em Via Aberta (ORT) Tracking através de um conversor de mensagens publicado como parte dos repositórios de código aberto deste projeto. O próprio formato inclui uma variedade de campos, mas quatro deles não são utilizados no serviço; apenas os apresentados no diagrama são relevantes para o processo de conversão. As mensagens JSON são então publicadas no broker MQTT sob os tópicos: *outsight/vehicle/realtime-tracking* e *outsight/vehicle/history-tracking*.

```

{
  "absolutecoordinates": {
    "latitude": 38.711326565736,
    "longitude": -9.30909723045238
  },
  "dimensions": {
    "height": 1.50254285335541,
    "length": 4.85778043517589,
    "width": 1.93075036153318
  },
  "localcoordinates": {
    "x": 0.913455769597306,
    "y": -5.19060603655761
  },
  "objectID": 9153234,
  "speedkmh": 27.05,
  "timeofmeasurement": "2026-03-04T12:38:22.033763Z"
}

```

Figura 9 - Exemplo JSON para o tópico OutSight/vehicle/realtime-tracking

```

{
  "objectID": 9153234,
  "path": [
    {
      "absolutecoordinates": {
        "latitude": 38.711326565736,
        "longitude": -9.30909723045238
      },
      "dimensions": {
        "height": 1.50254285335541,
        "length": 4.85778043517589,
        "width": 1.93075036153318
      },
      "localcoordinates": {
        "x": 0.913455769597306,

```

```

        "y": -5.19060603655761
      },
      "objectID": 9153234,
      "speedkmh": 27.05,
      "timeofmeasurement": "2026-03-
04T12:38:22.033763Z"
    },
    {
      "absolutecoordinates": {
        "latitude": 38.7111856262688,
        "longitude": -9.30938341851149
      },
      "dimensions": {
        "height": 1.54137992858887,
        "length": 4.88447282550038,
        "width": 1.96561240285323
      },
      "localcoordinates": {
        "x": 0.477515758450717,
        "y": 24.2067334698015
      },
      "objectID": 9153234,
      "speedkmh": 29.16,
      "timeofmeasurement": "2026-03-
04T12:38:23.084392Z"
    }
  ],
  "triggeringtimestamp": "2026-03-04T12:38:23.084392Z"
}

```

Figura 10 - Exemplo JSON para o tópico OutSight/vehicle/history-tracking

1.1.2.3 Mensagens do Toll Zone Controller

A solução TZC regista e processa mensagens de cada sensor, fundindo-as depois numa única transação de veículo. Para o DT4Mob, foi criado um módulo adicional para monitorizar os registos (logs) do TZC em busca de novas deteções ou transações. Este módulo converte deteções e transações em mensagens JSON e publica-as no broker MQTT nos respetivos tópicos: *tzc/vehicle/realtime-tracking* e *tzc/vehicle/history-tracking*.

```

{
  "absolutecoordinates": {
    "coordinatereference": "right_front_bottom_corner",
    "latitude": 38.7113021,
    "longitude": -9.3092177
  },
  "dimensions": {
    "height": 1.62,
    "width": 1.75
  },
  "localcoordinates": {
    "coordinatereference": "right_front_bottom_corner",
    "x": 6.22,
    "y": 0.0
  },
  "objectID": 1325426,
  "timeofmeasurement": "2026-03-04T13:17:05.819Z"
}

```

Figura 11 - Exemplo JSON para o tópico TZC/vehicle/realtime-tracking.

```
{
  "objectID": 1325426,
  "path": [
    {
      "absolutecoordinates": {
        "coordinatereference":
"right_front_bottom_corner",
        "latitude": 38.7113021,
        "longitude": -9.3092177
      },
      "dimensions": {
        "height": 1.62,
        "width": 1.75
      },
      "localcoordinates": {
        "coordinatereference":
"right_front_bottom_corner",
        "x": 6.22,
        "y": 0.0
      },
      "objectID": 1325426,
      "timeofmeasurement": "2026-03-04T13:17:05.819Z"
    },
    {
      "absolutecoordinates": {
        "coordinatereference":
"right_front_bottom_corner",
        "latitude": 38.7112068,
        "longitude": -9.3091345
      },
      "dimensions": {
        "height": 1.61,
        "length": 4.31,
        "width": 1.77
      },
      "localcoordinates": {
        "coordinatereference":
"right_front_bottom_corner",
        "x": 6.41,
        "y": 12
      },
      "objectID": 1325426,
      "speedkmh": 108.46,
      "timeofmeasurement": "2026-03-04T13:17:06.180Z"
    }
  ],
  "triggeringtimestamp": "2026-03-04T13:17:06.330Z"
}
```

Figura 11 - Exemplo JSON para o tópico tzc/vehicle/realtime-tracking

1.1.3 Gantry Service

O objetivo deste serviço é ligar os dados dos sensores de portagem à plataforma de Gémeo Digital da forma mais transparente possível, permitindo que os objetos detetados nas portagens tenham uma contraparte digital. Este serviço faz a ponte entre os dados estruturados dos sensores da via e a plataforma de Gémeo Digital Eclipse Ditto, através do Eclipse Hono, suportando ingestão via MQTT e via webhook.

1.1.3.1 Formato ORT Tracking (entrada — conversor A-to-Be)

History Message

Campo	Tipo
<i>objectID</i>	<i>integer</i>
<i>triggeringTimestamp</i>	<i>string</i>
<i>path</i>	<i>array</i>

Tabela 2 - Mensagem de Histórico

Item path

Campo	Tipo
<i>timeOfMeasurement</i>	<i>string</i>
<i>speedKmh</i>	<i>number</i>
<i>coordinatesReference</i>	<i>string</i>
<i>LocalCoordinates</i>	<i>object</i>
<i>absoluteCoordinates</i>	<i>object</i>
<i>dimensions</i>	<i>object</i>

Tabela 3 - Caminho do Item

localCoordinates

Campo	Tipo
<i>x</i>	<i>number</i>
<i>y</i>	<i>number</i>

Tabela 4 - localCoordinates

absoluteCoordinates

Campo	Tipo
<i>Latitude</i>	<i>number</i>
<i>Longitude</i>	<i>number</i>

Tabela 5 - absoluteCoordinates

Dimensions

Campo	Tipo
<i>width</i>	<i>number</i>
<i>length</i>	<i>number</i>
<i>height</i>	<i>number</i>

Tabela 6 - dimensions

Real-time Message

Campo	Tipo
<i>objectID</i>	<i>integer</i>
<i>triggeringTimestamp</i>	<i>string</i>
<i>timeOfMeasurement</i>	<i>string</i>
<i>speedKmh</i>	<i>number</i>
<i>coordinatesReference</i>	<i>string</i>

Tabela 7 - Mensagem em Tempo Real

1.1.3.2 Mensagem OutSight (entrada — conversor OutSight)

Quando o Gantry Service está configurado com um conversor de mensagens OutSight, aceita mensagens diretamente dos produtores de mensagens no formato OutSight.

OutSightMessage

Campo	Tipo
<i>id</i>	<i>string</i>
<i>type</i>	<i>string</i>
<i>start_timestamp</i>	<i>datetime</i>
<i>end_timestamp</i>	<i>datetime (opcional)</i>
<i>data</i>	<i>Data</i>

Tabela 8 - OutSightMessage

Data

Campo	Tipo
<i>in_alert</i>	<i>Alert</i>
<i>out_alert</i>	<i>Alert (opcional)</i>

Tabela 9 – Dados

Alert

Campo	Tipo
<i>object_id</i>	<i>integer</i>
<i>class_name</i>	<i>string</i>
<i>speed</i>	<i>number (≥ 0)</i>
<i>coordinate</i>	<i>BoundingBoxSquare (4 × (x, y) tuples)</i>
<i>height</i>	<i>number (≥ 0)</i>
<i>zone_name</i>	<i>string</i>
<i>speed_vector</i>	<i>(number, number, number)</i>

Tabela 10 - Alerta

BoundingBoxSquare: Quatro coordenadas dos cantos da bounding box de detecção:

Índice	Descrição
<i>[0]</i>	<i>Primeiro canto (x, y)</i>
<i>[1]</i>	<i>Segundo canto (x, y)</i>
<i>[2]</i>	<i>Terceiro canto (x, y)</i>
<i>[3]</i>	<i>Quarto canto (x, y)</i>

Tabela 11 - BoundingBoxSquare

Os valores de coordenadas encontram-se no referencial local do sensor (metros relativamente à origem do sensor). O canto [3] é utilizado como ponto de referência para a transformação de coordenadas.

1.1.3.3 Representação Ditto Thing: toll-feature (saída)

Uma Ditto Thing por portagem. Cada sensor é uma *feature*; os dados de veículo são armazenados como propriedades sob a feature desse sensor, indexados pelo ID do veículo.

Nome do campo			Tipo	Indexado?
Attributes	id		Integer	sim
	name		String	não
	location	latitude	Float	não
		longitude	Float	não
	geotile		Integer	sim
Features	<SourceName>	<Object ID> or Literal<"historic">	ORT Tracking Message	não

Tabela 12 - Definição de um Toll Thing

Example of a Toll Thing:

```
{
  "thingId": "tolls:test:toll-0",
  "policyid": "dt4mob:default",
  "features": {
    "camera-ORT": {
      "properties": {
        "42": {
          "objectId": 42,
          "timeofmeasurement": "2026-06-29T12:00:00Z",
          "speedkmh": 85.2,
          "coordinatesreference": "right_rear_bottom_corner",
          "localcoordinates": { "x": 12.3, "y": 4.5 },
          "absolutecoordinates": { "latitude": 38.7521, "longitude": -9.1575
        },
        "dimensions": { "width": 1.8, "length": 4.5, "height": 1.6 }
      },
      "43": { "...": "..." }
    }
  },
  "lidar-tzc": {
    "properties": {
      "42": { "...": "..." }
    }
  }
}
```

Ciclo de vida:

- Os dados de veículo são adicionados/atualizados sob a feature do sensor correspondente.
- É mantido um buffer circular de 50 IDs de veículo; os IDs expirados são removidos através de comandos `delete`.
- Uma limpeza periódica (a cada 150s, por defeito) limpa todas as propriedades sob cada feature de sensor.

1.1.3.4 Representação Ditto Thing: Ditto-Thing (saída)

Um Ditto Thing por veículo seguido. Os dados do veículo são armazenados numa feature `State`;

os metadados são armazenados nos atributos.

Nome do campo		Tipo	Indexado?
Attributes	id	Integer	yes
	expiry_ts	ISO 8601 String	no
	geotile	Integer	yes
Features	State	ORT Tracking Message	no

Tabela 13 - Definição de um Thing de Objeto detetado

O `expiry_ts` é definido como 30 segundos a partir da última atualização. Isto serve como indicação para o garbage collector da plataforma remover automaticamente o Thing quando este expirar.

Example of a detected OBJECT Thing:

```
{
  "thingId": "tolls/test:camera-ORT.42",
  "policyid": "dt4mob:default",
  "attributes": {
    "id": 42,
    "expirar_ts": "2026-06-29T12:00:30Z"
  },
  "features": {
    "State": {
      "properties": {
        "objectid": 42,
        "triggeringtimestamp": "2026-06-29T12:00:00Z",
        "timeofmeasurement": "2026-06-29T12:00:00Z",
        "speedkmh": 85.2,
        "coordinatesreference": "right_rear_bottom_corner",
        "localcoordinates": { "x": 12.3, "y": 4.5 },
        "absolutecoordinates": { "latitude": 38.7521, "longitude": -9.1575 },
        "dimensions": { "width": 1.8, "length": 4.5, "height": 1.6 }
      }
    }
  }
}
```

Ciclo de vida:

- A cada atualização, o `expiry_ts` é atualizado para `now + 30s`.
- Os "Future Things" são pré-criados com um deslocamento de ID (`vehicle_id + deferred_creation_offset`, por defeito 10) e um TTL mais longo (10 min 30s), permitindo uma transição transparente entre sensores.
- É mantido um buffer circular de 50 IDs futuros e os "future Things" expirados são eliminados em lotes.

1.1.4 Serviços de conversão de dados de sensores

Este serviço converte deteções de objetos no formato da OutSight para um formato unificado, ORT Tracking, reduzindo assim a complexidade antes dos dados chegarem ao Gantry Service.

- **Lógica algorítmica:**

1. Receber detecção com o TIP original do sensor.
2. Transformar de coordenadas ENU locais para coordenadas absolutas WGS84 utilizando `pymap3d` e `pyproj`.
3. Calcular as dimensões reais do veículo (comprimento, largura, altura) a partir dos parâmetros da caixa delimitadora.
4. Serializar a saída no formato ORT Tracking utilizando validação `pydantic`.

1.1.4.1 Mensagem de entrada OutSight

Este serviço lida apenas com mensagens OutSight definidas anteriormente na secção 1.1.3.2.

1.1.4.2 Formato de saída ORT Tracking

A saída convertida segue o formato de mensagem ORT Tracking documentado na secção 1.1.3.1. São produzidos dois tipos de mensagens por entrada:

- `ORTTrackingRealtimeMessage`: Utiliza `out_alert` (preferido) ou `in_alert`; produz um único item do caminho (`path`) com coordenadas e dimensões em tempo real.
- `ORTTrackingHistoryMessage`: Contém um array `path` com `in_alert` e `out_alert` como entradas (apenas produzido quando `out_alert` está presente na entrada).

1.1.4.3 Planos de referência de coordenadas

A transformação entre o referencial de coordenadas local da OutSight e o referencial A-to-Be (ORT Tracking) é configurada através de dois planos de referência:

OutSightToAToBeConfig

Campo	Tipo
<code>outsight_reference_plane</code>	<code>ReferencePlane</code>
<code>atobe_reference_plane</code>	<code>ReferencePlane</code>

Tabela 14 - OutSightToAToBeConfig

Referenceplane

Campo	Tipo
<code>origin</code>	<code>(latitude, longitude)</code>
<code>point_on_positive_horizontal_axis</code>	<code>(latitude, longitude)</code>

Tabela 15 - ReferencePlane

Os valores são definidos em `config.toml` e podem ser sobrepostos por variáveis de ambiente com o prefixo `SENSOR_DATA_CONVERSION_CONVERTER__..`

1.1.5 Simulação de Faixa Reservada A5

A Simulação de Faixa Reservada na A5 foi desenvolvida como um serviço de simulação de tráfego a pedido, destinado à avaliação da implementação de uma faixa reservada com acesso pago na autoestrada A5 em condições de congestionamento.

1.1.5.1 Interface

Através da interface disponibilizada, Figura 12, o utilizador pode configurar os principais parâmetros da simulação, nomeadamente a percentagem de veículos dispostos a pagar pela utilização da faixa reservada, o preço da portagem e o fator de escala da procura de tráfego. Após a submissão destes parâmetros, o sistema executa automaticamente um conjunto de cenários experimentais e apresenta uma comparação dos respetivos indicadores operacionais e económicos (Figura 13).

A5 Reserved Lane Simulation

Value Definition

Paying Vehicle Percentage: 20%

Fraction of eligible vehicles allowed to pay for the reserved lane.

Lane Price (€)

1.00

Scale Traffic

2.0

Start Simulation

Simulations

Auto-refreshing...

STATUS	PARAMETERS	CREATED	
running	20% · €1.00 · ×2	3:00:17 PM	
running	20% · €1.00 · ×2	2:58:32 PM	
done	20% · €1.00 · ×2	2:49:20 PM	View
done	20% · €1.00 · ×2	2:39:08 PM	View

Figure 12 - Interface da aplicação para execução das simulações.

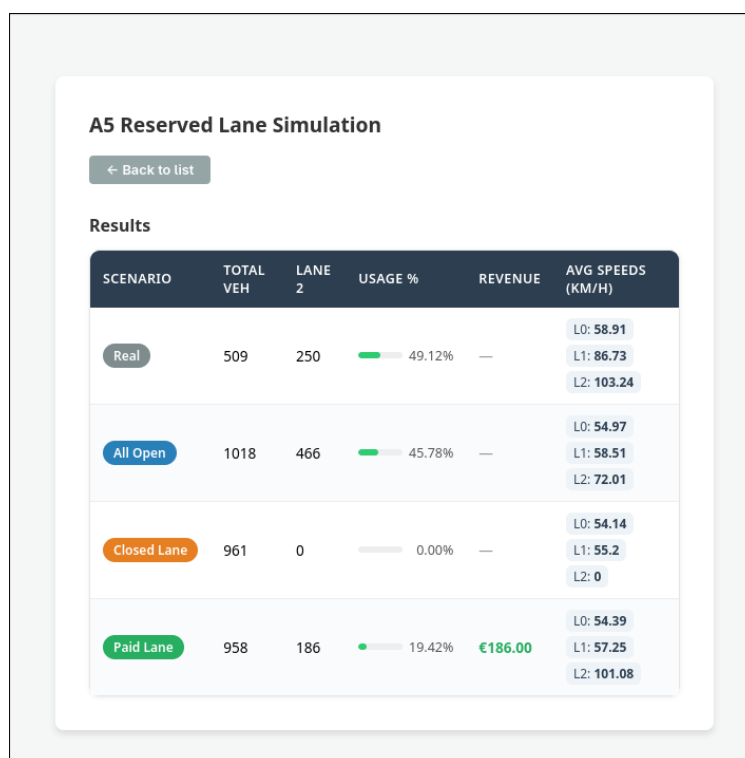


Figure 13 - Interface da aplicação com a tabela de resultados das simulações.

A aplicação foi desenvolvida como um serviço web assíncrono utilizando FastAPI. Cada cenário é executado numa instância independente do SUMO, recorrendo ao módulo multiprocessing de Python, uma vez que cada ligação TraCI controla uma simulação. Os quatro cenários são executados em paralelo, sendo posteriormente agregados numa única resposta. A arquitetura encontra-se organizada em componentes independentes responsáveis pela execução das simulações, recolha das estatísticas e coordenação dos cenários.

Em cada pedido são avaliados quatro cenários distintos: um cenário correspondente ao tráfego histórico observado, um cenário com aumento da procura mantendo todas as faixas disponíveis, um cenário em que a faixa reservada permanece encerrada ao tráfego geral e um cenário de faixa gerida com acesso condicionado ao pagamento da portagem. A execução simultânea destes cenários permite comparar diretamente os efeitos operacionais da implementação da faixa reservada sob as mesmas condições.

1.1.5.2 Simulador SUMO

A simulação baseia-se no Eclipse SUMO, recorrendo a um modelo microscópico em que cada veículo é representado individualmente e o seu estado é atualizado continuamente ao longo da execução. Os movimentos dos veículos seguem os modelos de seguimento e de mudança de faixa disponibilizados pelo SUMO, enquanto a lógica associada à gestão da faixa reservada é implementada através da Traffic Control Interface (TraCI). Durante a simulação, os veículos elegíveis podem obter dinamicamente autorização para utilizar a faixa reservada de acordo com a probabilidade de pagamento definida pelo utilizador, permitindo avaliar diferentes estratégias de tarifação e diferentes níveis de adesão ao sistema.

A rede viária utilizada corresponde a um troço da autoestrada A5 (Figura 12), gerado a partir de dados do OpenStreetMap e convertido para um formato compatível com o SUMO. A procura de tráfego é reconstruída a partir de registos históricos provenientes da infraestrutura de portagem, permitindo gerar tráfego aproximado das condições reais. Desta forma, a simulação (Figura 13)

combina uma representação realista da infraestrutura rodoviária com padrões de tráfego observados, proporcionando uma base para a análise dos diferentes cenários.

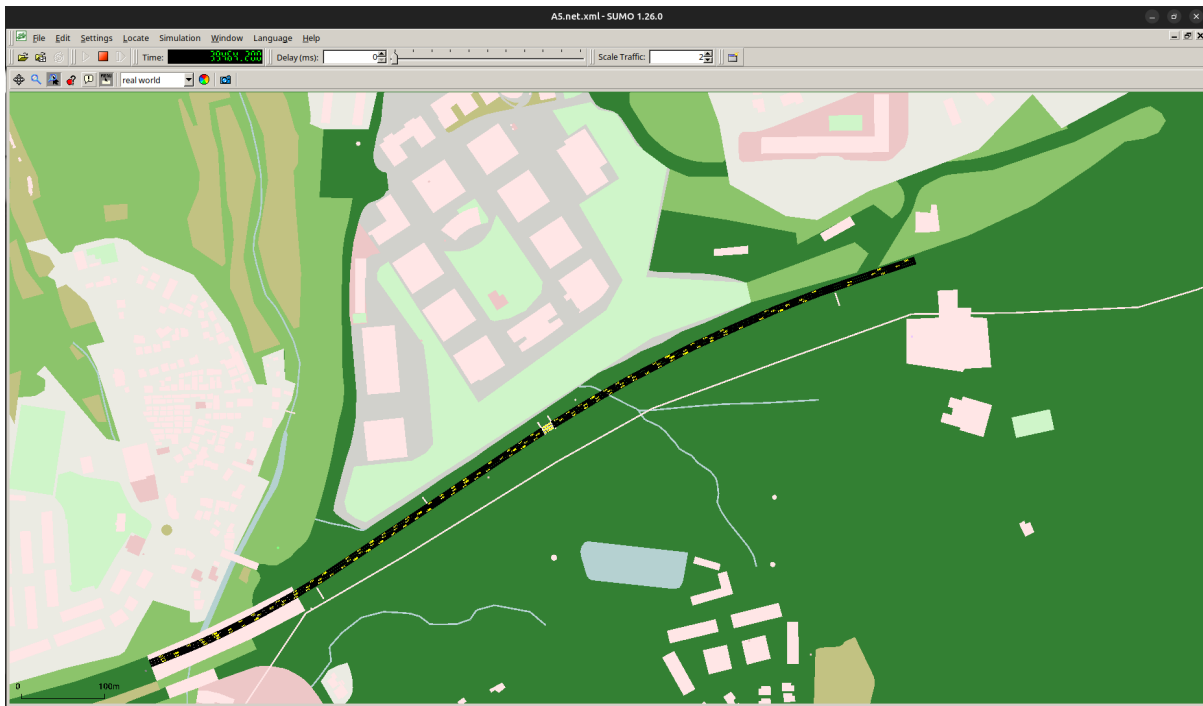


Figura 12 - Captura de tela do SUMO GUI do segmento da autoestrada A5 simulada

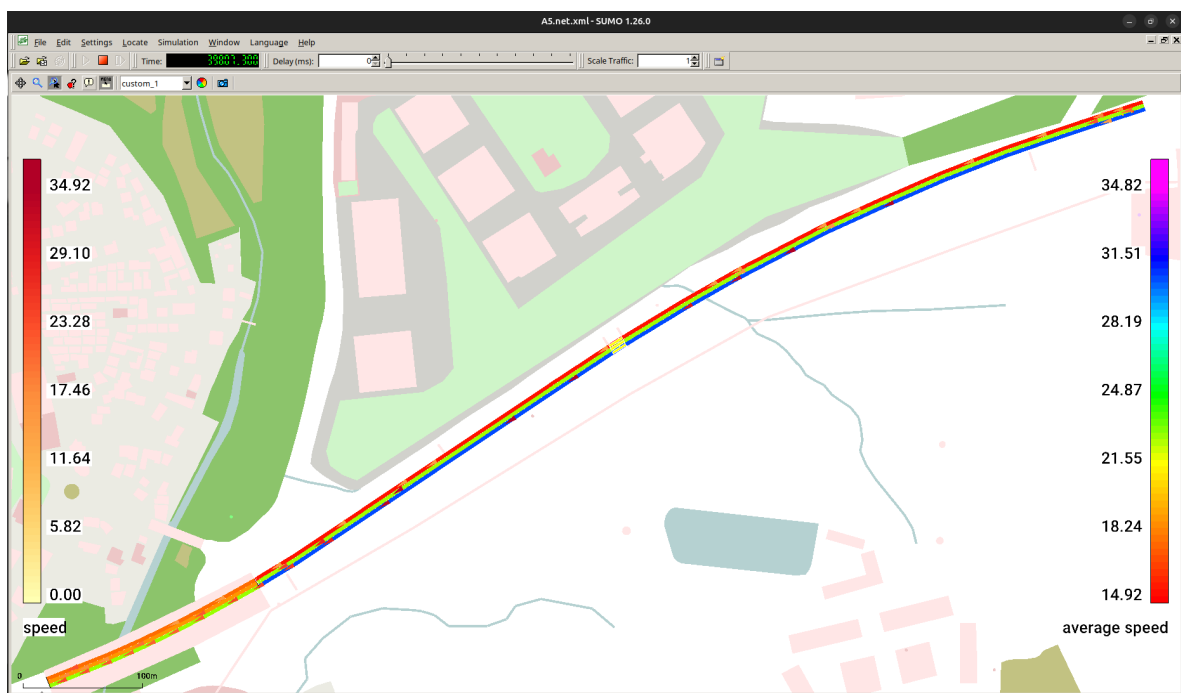


Figura 13 - Captura de tela do SUMO da interface da simulação de tráfego em execução. Veículos (triângulos) e rede viária colorida por velocidade em m/s (à esquerda a legenda de cor dos veículos e à direita a legenda de cor das faixas com base na velocidade média em m/s)

1.1.5.3 Resultados

Para cada cenário são calculados um conjunto de indicadores que caracterizam o desempenho do sistema. Entre estes incluem-se o número total de veículos simulados, o número e a percentagem de veículos que utilizaram a faixa reservada, a velocidade média nas faixas monitorizadas e a receita estimada da portagem. A comparação destes indicadores permite avaliar o impacto da implementação da faixa gerida em termos de desempenho do tráfego, utilização da infraestrutura e retorno económico, constituindo um instrumento de apoio à decisão para diferentes estratégias de gestão.

1.2 Caso de uso: monitorização de muro de suporte e talude

Este caso de uso foca-se na monitorização de um muro de suporte e do talude adjacente ao longo do corredor da autoestrada A9, na área de Lisboa, dentro da rede de concessão da Brisa. O quadro de monitorização combina instrumentação geotécnica *in situ* com campanhas periódicas de deteção remota, para apoiar o desenvolvimento de um Gémeo Digital da estrutura. A instrumentação instalada no muro inclui células de carga (A19, A25.1, A25.2, A35) e inclinómetros (I3C, I3MEEC, I4, I2C, I1C), conforme mostrado na Figura 16, que medem continuamente ou periodicamente a tensão estrutural e o deslocamento, fornecendo informação direta sobre o comportamento interno do sistema.



Figura 14 - Localização do Muro e Sensores

Estas medições são complementadas por campanhas periódicas de aquisição de LiDAR e imagens, utilizando sistemas de mapeamento móvel e plataformas UAV, permitindo uma monitorização geométrica de alta resolução das superfícies do muro e do talude. A comparação entre épocas sucessivas de nuvens de pontos permite a deteção e quantificação de pequenas alterações geométricas, que podem depois ser correlacionadas com as leituras dos sensores instalados. Através da integração de dados contínuos de sensores e levantamentos espaciais episódicos, o Gémeo Digital pretende fornecer uma representação abrangente e baseada em dados da condição estrutural e evolução dos ativos monitorizados.

O quadro de monitorização implementado neste caso de uso combina medições contínuas provenientes de instrumentação *in situ* com informação espacialmente distribuída proveniente de levantamentos de deteção remota. A integração destes conjuntos de dados heterogéneos permite o desenvolvimento de uma representação em Gémeo Digital do muro de suporte e do talude adjacente.

O fluxo de trabalho analítico implementado para este quadro de monitorização inclui as seguintes etapas:

1. Aquisição de dados a partir de sensores e levantamentos de deteção remota
2. Ingestão e armazenamento de dados na plataforma LevelAMS
3. Processamento e harmonização dos conjuntos de dados de monitorização
4. Análise geométrica de nuvens de pontos
5. Geração de indicadores de monitorização
6. Cálculo de índices de condição da infraestrutura

Este fluxo de trabalho permite a avaliação contínua do comportamento estrutural da infraestrutura monitorizada e apoia os processos de decisão relacionados com a manutenção e a gestão de risco.

1.2.1 Fontes de dados

A monitorização eficaz dos muros de contenção e dos taludes requer a integração de diferentes tipos de informação, cada um captando aspetos específicos do comportamento estrutural e ambiental do ativo.

1.2.1.1 Instrumentação *in situ*

A monitorização eficaz de muros de suporte e taludes requer a integração de diferentes tipos de informação, cada um captando aspetos específicos do comportamento estrutural e ambiental do ativo.

A instrumentação típica inclui:

- inclinómetros, utilizados para medir deslocamentos horizontais dentro da estrutura ou do solo circundante;
- células de carga, instaladas para medir forças estruturais;
- sensores de deslocamento, utilizados para monitorizar movimentos de superfície;
- sensores de temperatura e humidade, que captam condições ambientais que afetam o comportamento dos materiais.

Estes sensores permitem a monitorização contínua do comportamento estrutural e fornecem dados de série temporal de alta frequência que podem revelar padrões de deformação progressiva.

A colocação e a configuração dos sensores dependem da geometria do muro de suporte, das características geotécnicas do terreno circundante e dos mecanismos de deformação esperados. A configuração utilizada pode ser consultada na Figura 16.

1.2.1.2 Técnicas de deteção remota

Os levantamentos de deteção remota incluem varrimento LiDAR e aquisição fotogramétrica através de plataformas UAV. Estas técnicas produzem representações tridimensionais de alta resolução do

muro de suporte e do terreno circundante, na forma de nuvens de pontos densas, ortofotos e modelos digitais de elevação.

Estes conjuntos de dados permitem a identificação de alterações geométricas ocorridas entre campanhas de aquisição e fornecem informação espacial que complementa as medições estruturais obtidas pela instrumentação instalada.

1.2.1.3 Informação de inspeção e ativos

Além dos dados de monitorização obtidos a partir de sensores e técnicas de deteção remota, o quadro de monitorização do muro integra informação da infraestrutura e dados de inspeção através da plataforma de gestão de ativos LevelAMS.

A plataforma LevelAMS, já implementada no fluxo de trabalho do departamento de geotecnia da Brisa, atua como o sistema de informação central responsável pelo armazenamento, gestão e análise dos dados de monitorização da infraestrutura. No âmbito do quadro de monitorização do muro, a plataforma integra diferentes tipos de informação, incluindo as características da infraestrutura, os resultados de inspeções, as medições dos sensores e os resultados analíticos derivados das análises de deteção remota.

A plataforma permite o registo e a gestão de ativos geotécnicos, possibilitando aos utilizadores armazenar informação relativa a muros de suporte, taludes e outros elementos críticos da infraestrutura. Cada ativo está associado a informação espacial, conjuntos de dados de monitorização e registos de inspeção que descrevem a sua condição estrutural e histórico operacional.

Dentro do LevelAMS, várias funcionalidades apoiam a monitorização e a análise da infraestrutura, incluindo:

- registo e gestão de ativos;
- armazenamento e visualização de dados de monitorização;
- integração de medições de sensores;
- visualização de modelos de nuvens de pontos e imagens de satélite;
- análise gráfica de leituras de sensores;
- gestão de relatórios de inspeção;
- geração de alertas e indicadores de monitorização.

A plataforma também permite o cálculo de indicadores de avaliação da infraestrutura, como o Índice de Condição, que representa uma avaliação composta da condição estrutural do ativo monitorizado. Estes indicadores combinam informação de diferentes fontes, incluindo inspeções, medições de monitorização e fatores ambientais.

Através desta integração, a plataforma fornece um ambiente unificado para analisar a condição da infraestrutura e apoiar as decisões de manutenção e gestão de risco.

A arquitetura da plataforma baseia-se numa estrutura de aplicação web, com um backend implementado em .NET Core e uma base de dados relacional utilizada para armazenar os dados de monitorização e a informação dos ativos. Os dados armazenados no LevelAMS são disponibilizados através de uma API REST, permitindo o acesso estruturado e programático a parâmetros-chave como cargas estruturais, deslocamentos, gradientes de deformação e índices ambientais, incluindo medições de precipitação e registos de eventos extremos. Esta ligação baseada em API permite que os componentes analíticos e de visualização do sistema sejam atualizados em tempo real ou quase real, garantindo que o Gémeo Digital reflete consistentemente o estado estrutural e ambiental atual dos ativos monitorizados.

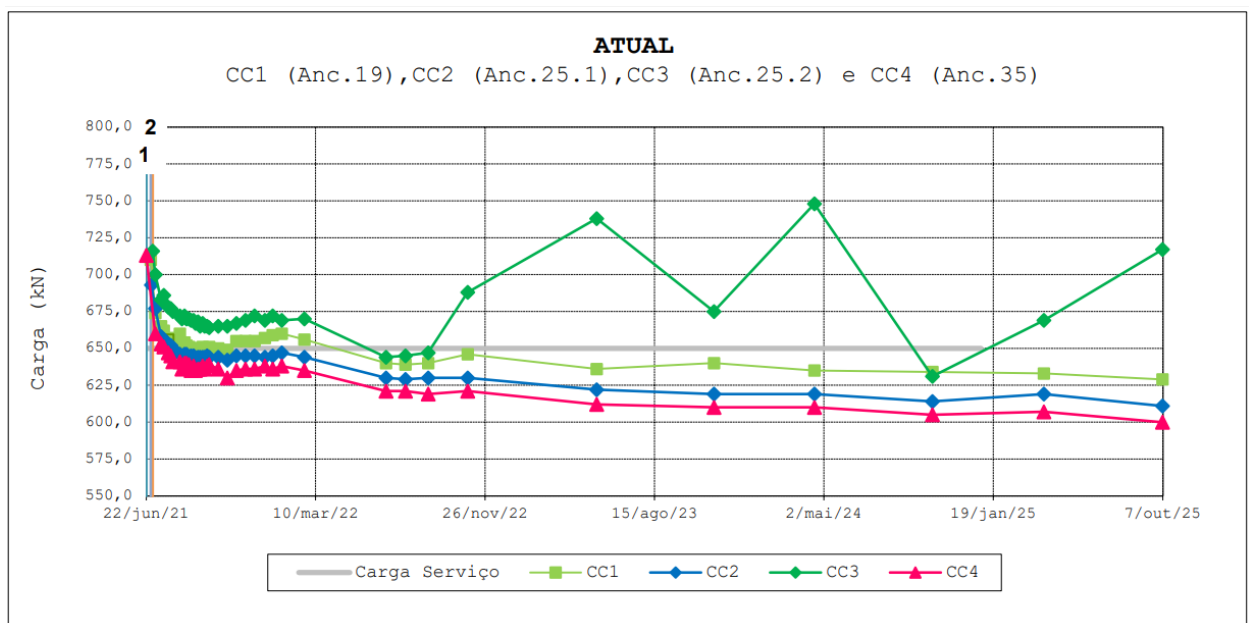
1.2.2 Formatos de dados

1.2.2.1 Formatos da API LevelAMS

Os dados de monitorização provêm de diferentes sistemas de aquisição e, por isso, requerem normalização antes de serem integrados na plataforma de monitorização.

As medições dos sensores são normalmente transmitidas em formatos estruturados, como ficheiros JSON ou CSV, e incluem:

- identificador do sensor;
- instante temporal (timestamp);
- valor da medição;
- informação de localização.



• Figura 15 - Exemplo de um gráfico das células de carga

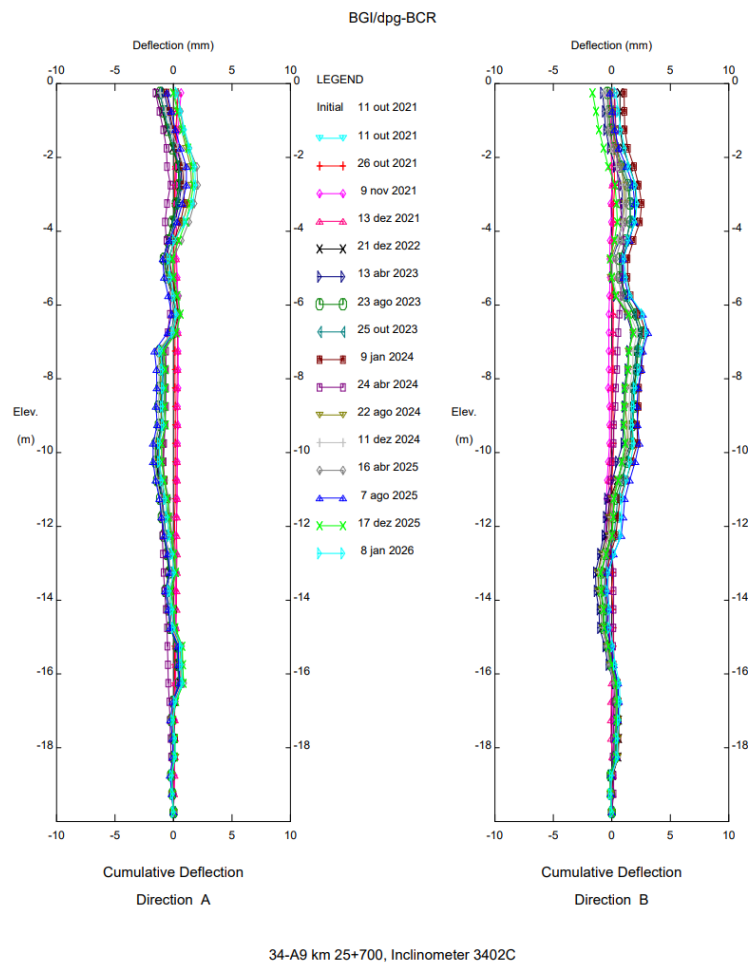


Figura 16 - Exemplo de gráfico de inclinómetros

Os conjuntos de dados de detecção remota, como os levantamentos de varrimento laser, são normalmente armazenados em formatos de nuvem de pontos, incluindo LAS, LAZ ou PLY. Estes ficheiros contêm coordenadas espaciais e atributos adicionais que descrevem cada ponto da nuvem.

Após o processamento, os resultados de monitorização são exportados para conjuntos de dados tabulares estruturados que facilitam a integração com ferramentas analíticas e sistemas de gestão de dados.

Para apoiar esta integração, o LevelAMS expõe um conjunto de pontos finais (endpoints) de interoperabilidade que permitem o acesso seguro e estruturado aos dados geotécnicos. O acesso autenticado é estabelecido através de chaves de API via o endpoint `/accounts/login`. Uma vez autenticado, estão disponíveis as seguintes funcionalidades:

Obtenção de informação sobre ativos geotécnicos, incluindo geometria, atributos e classificação:

- GET `/API/v1/ativos-geotecnicos`
- GET `/API/v1/ativos-geotecnicos/{id}`

Acesso a conjuntos de dados de índices necessários para a atualização contínua do modelo dentro do Gémeo Digital:

- GET /API/v1/ativos-geotecnicos/{id}/kpi

Integração de metadados que descrevem os sensores in situ associados a cada estrutura monitorizada:

- GET /API/v1/ativos-geotecnicos/{id}/sensor-list

Descarregamento de modelos geotécnicos disponíveis, quando aplicável, para visualização ou análise comparativa dentro do Gémeo Digital:

- GET /API/v1/ativos-geotecnicos/{id}/modelo/{modelotipo}/download

Estes endpoints garantem, em conjunto, uma interoperabilidade consistente entre o LevelAMS e o Gémeo Digital, apoiando a ingestão automática de dados, a sincronização de índices e o alinhamento entre as medições dos sensores de campo e as camadas analíticas.

O acesso a todos os serviços da API é protegido com chaves de API, garantindo um acesso a dados autenticado e controlado.

1.2.2.2 Formatos de dados de mapeamento móvel

Os levantamentos de mapeamento móvel foram realizados utilizando o Leica Pegasus TRK 700, um Sistema de Mapeamento Móvel (MMS) que integra um scanner LiDAR multi-retorno com uma unidade GNSS/IMU para georreferenciação direta. O sistema adquire dados de nuvem de pontos densos e contínuos ao longo do corredor de levantamento a alta velocidade, tornando-o particularmente adequado para infraestruturas lineares de grande escala, como aterros de estrada, muros de suporte e taludes de corte.

Os dados brutos produzidos pelo Pegasus TRK 700 são processados através do software Leica Pegasus Office, onde é realizado o cálculo de trajetória, a geração da nuvem de pontos e a georreferenciação. A nuvem de pontos georreferenciada resultante é exportada no formato padrão da indústria LAS, que codifica, para cada ponto, as coordenadas XYZ, a intensidade, o número de retorno, o número de retornos, o instante temporal GPS e os campos de classificação.

Para cada época de aquisição, a nuvem de pontos processada é o principal produto de dados. Estas nuvens de pontos são posteriormente utilizadas como entrada no pipeline de análise de deslocamento, onde são realizadas comparações entre épocas utilizando o método M3C2 (Multiscale Model to Model Cloud Comparison). A saída do M3C2, que representa distâncias com sinal ao longo da normal da superfície entre épocas, é depois exportada como ficheiros CSV de texto simples, contendo magnitudes de deslocamento por ponto, indicadores de qualidade e identificadores de região de interesse, para ingestão no Gémeo Digital.

As representações em malha 3D utilizadas para fins de visualização são derivadas das mesmas nuvens de pontos através de reconstrução de superfície e exportadas no formato OBJ, adequado para motores de renderização 3D baseados na web.

1.2.2.3 Formatos de dados de mapeamento com drone

Os levantamentos aéreos foram realizados utilizando o veículo aéreo não tripulado (UAV) DJI Matrice 350 RTK, equipado com duas cargas úteis DJI Zenmuse operadas em modos de aquisição complementares: o Zenmuse P1, uma câmara fotogramétrica de fotograma completo, e o Zenmuse L2, um sensor integrado de mapeamento LiDAR e RGB.

O Zenmuse P1 capta imagens sobrepostas de alta resolução, que são processadas através de um processo fotogramétrico no DJI Terra, produzindo nuvens de pontos densas e modelos de malha

3D georreferenciados. Os dados de nuvem de pontos são entregues no formato LAS, enquanto a malha 3D, utilizada para a representação visual das estruturas monitorizadas, é exportada no formato OBJ para apoiar a integração com ambientes de visualização de Gémeo Digital baseados na web.

O Zenmuse L2 fornece aquisição LiDAR direta com uma IMU integrada, produzindo nuvens de pontos nativamente georreferenciadas. Os dados brutos são processados através do DJI Terra, sendo os resultados finais entregues no formato LAS. As nuvens de pontos do L2 complementam os dados fotogramétricos, fornecendo geometria estrutural precisa em áreas com textura de imagem reduzida, como as superfícies de muros de betão.

Em ambos os casos, as nuvens de pontos de épocas de levantamento sucessivas são registadas num referencial comum e comparadas através do método M3C2, com os resultados exportados como ficheiros CSV contendo magnitudes de deslocamento por ponto, indicadores de qualidade e identificadores de região de interesse, para ingestão na plataforma.

1.2.3 Data Bridge e modelos de dados

O sistema de monitorização baseia-se em modelos de dados estruturados que organizam os conjuntos de dados de monitorização e apoiam a integração de informação de múltiplas fontes na plataforma de monitorização.

Cada ativo de infraestrutura é representado no sistema através de um modelo de dados que associa informação espacial, medições de monitorização, resultados de inspeção e indicadores analíticos.

O modelo de dados permite a integração de diferentes tipos de dados de monitorização, incluindo:

- resultados de comparação de nuvens de pontos, derivados de levantamentos de deteção remota;
- medições em série temporal, obtidas a partir de instrumentação com sensores;
- observações de inspeção que descrevem condições estruturais visíveis;
- dados ambientais que descrevem as condições climáticas que afetam a infraestrutura.

Nesta estrutura de dados, os conjuntos de dados de monitorização estão associados aos respetivos ativos de infraestrutura e incluem atributos espaciais e temporais que descrevem cada medição.

Os modelos de dados também apoiam o cálculo de indicadores de avaliação da infraestrutura, como o Índice de Condição, que fornece uma representação composta da condição estrutural global do ativo monitorizado.

Os modelos de dados implementados no quadro de monitorização permitem a integração de conjuntos de dados heterogéneos que descrevem diferentes aspetos do comportamento da infraestrutura.

Cada ativo geotécnico registado na plataforma contém vários conjuntos de dados associados, incluindo:

- metadados da infraestrutura
- informação de instrumentação de sensores
- registos de inspeção
- conjuntos de dados de deteção remota
- indicadores de monitorização

Esta estrutura permite que os modelos analíticos acessem tanto a informação histórica como a informação de monitorização em tempo real, e apoia o cálculo de índices de avaliação da infraestrutura.

1.2.3.1 Índice de Condição

O Índice de Condição (IC) é um indicador composto que representa a condição global de um ativo geotécnico monitorizado na plataforma. O objetivo deste índice é combinar informação de múltiplas fontes de dados e fornecer uma representação simplificada da condição da infraestrutura que possa apoiar a priorização da manutenção e a avaliação de risco.

O Índice de Condição é calculado como uma combinação ponderada de vários fatores contribuintes que descrevem diferentes aspetos do desempenho da infraestrutura. Estes fatores podem incluir indicadores de comportamento estrutural, observações de inspeção, condições ambientais e resultados de monitorização.

O cálculo do Índice de Condição pode ser expresso conceptualmente como:

$$IC = \sum_{i=1}^n w_i \cdot I_i$$

em que:

- w_i representa o fator de ponderação associado a cada indicador;
- I_i representa os indicadores de condição individuais.

Os fatores de ponderação refletem a importância relativa de cada indicador na avaliação global da condição da infraestrutura. Estes pesos podem ser ajustados de acordo com o tipo de infraestrutura, os objetivos de monitorização e as prioridades operacionais.

O Índice de Condição é atualizado sempre que nova informação de monitorização ou dados de inspeção fiquem disponíveis no sistema. Este mecanismo de atualização dinâmica garante que a avaliação da condição reflete a informação mais recente relativa à infraestrutura. O índice de condição varia de 1 a 5, sendo 1 um ativo em muito boa condição e 5 um ativo em condição muito má.

1.2.3.2 Indicadores de monitorização

O quadro de monitorização implementado no LevelAMS gera vários grupos de indicadores que descrevem diferentes aspetos do comportamento da infraestrutura e das condições ambientais.

Estes indicadores estão agrupados em três categorias principais:

- indicadores climáticos
- indicadores estruturais
- indicadores de monitorização

Cada grupo fornece informação complementar utilizada para avaliar a condição da infraestrutura e apoiar os processos de tomada de decisão.

Indicadores climáticos

Os indicadores climáticos descrevem condições ambientais que podem influenciar a estabilidade e o comportamento da infraestrutura geotécnica.

Estes indicadores são derivados de conjuntos de dados meteorológicos e incluem informação como:

- precipitação histórica;
- previsões de precipitação;
- alertas meteorológicos relacionados com eventos extremos.

A informação meteorológica é periodicamente obtida a partir de bases de dados externas e atualizada diariamente com base na localização geográfica da infraestrutura monitorizada.

Estes indicadores permitem a identificação de condições ambientais que possam aumentar a probabilidade de instabilidade geotécnica.

Indicadores estruturais

Os indicadores estruturais descrevem a condição física e o comportamento estrutural da infraestrutura monitorizada.

Estes indicadores podem incluir:

- índice de condição da infraestrutura;
- índice de suscetibilidade;
- avaliações de condição baseadas em inspeção.

O índice de suscetibilidade representa a exposição da infraestrutura a fatores de risco, como a localização, a relevância económica e o contexto ambiental.

Os indicadores estruturais são atualizados sempre que novos dados de monitorização, observações de inspeção ou resultados analíticos são integrados na plataforma.

Indicadores de monitorização

Os indicadores de monitorização descrevem o estado operacional do sistema de monitorização e o comportamento detetado pela instrumentação.

Os indicadores de monitorização típicos incluem:

- número de sensores em condição de alerta;
- número de sensores em condição de alarme;
- desvio das leituras dos sensores relativamente aos limites definidos;
- disponibilidade e fiabilidade dos dispositivos de monitorização.

Estes indicadores permitem que o sistema de monitorização detete comportamentos anómalos nas medições dos sensores e acione alertas quando os limites predefinidos são excedidos.

Por exemplo, o indicador de monitorização associado a alertas de sensores pode ser calculado da seguinte forma:

$$IM = \frac{N_{alert}}{N_{total}}$$

em que:

- n_{alerta} representa o número de sensores atualmente em condição de alerta;
- n_{total} representa o número total de sensores instalados.

Este rácio fornece uma medida quantitativa da proporção de sensores que estão a detetar condições anómalas na infraestrutura monitorizada.

1.2.3.3 Modelo de dados da API LevelAMS

A plataforma LevelAMS expõe uma API baseada em REST que permite o acesso estruturado e seguro aos dados de monitorização associados a ativos geotécnicos. Esta API permite que módulos analíticos externos e sistemas de Gémeo Digital obtenham medições de monitorização, metadados da infraestrutura e indicadores derivados.

Os seguintes endpoints definem as principais interações com o modelo de dados do LevelAMS.

Etiqueta / Domínio	Método	Endpoint (caminho)	Descrição	Notas / Glossário
Conta	POST	<code>/ {tenant} /API/ accounts/login</code>	Endpoint de autenticação de utilizadores. Autentica utilizadores do sistema (operadores, gestores de ativos, administradores) e devolve um token de acesso necessário para aceder aos restantes recursos da API.	Entrada: credenciais do utilizador (email/nome de utilizador e password).
Ativo Geotécnico	GET	<code>/ {tenant} /API/v1/ activos-geotecnicos</code>	Obtém a lista de ativos geotécnicos registados, com filtragem e paginação opcionais.	A resposta inclui atributos resumidos do ativo, como assetId, nome, tipo, localização, estado de conservação, estado de manutenção, índice de condição e quilometragem (PK).
Ativo Geotécnico	GET	<code>/ {tenant} /API/v1/ activos-geotecnicos/ {id}</code>	Obtém informação detalhada sobre um ativo geotécnico específico.	Inclui dados de identificação, georreferenciação, atributos geotécnicos e parâmetros de configuração dos

				sensores (limites e thresholds).
Ativo Geotécnico	GET	<code>/ {tenant} /API/v1/ativos-geotecnicos/{id}/kpi</code>	Obtém os indicadores-chave de desempenho (KPIs) associados ao ativo selecionado.	Devolve uma lista de KPIs, incluindo valor, unidade e instante temporal da última atualização.
Ativo Geotécnico	GET	<code>/ {tenant} /API/v1/ativos-geotecnicos/{id}/sensor-list</code>	Obtém a lista de sensores de monitorização instalados no ativo.	Inclui identificador do sensor, tipo de sensor, unidade de medida, localização espacial, estado de alerta e parâmetros monitorizados.
Ativo Geotécnico	GET	<code>/ {tenant} /API/v1/ativos-geotecnicos/{id}/modelo/{modelotipo}/download</code>	Descarrega modelos analíticos ou conjuntos de dados processados associados ao ativo.	Fluxo de ficheiro com metadados como versão do modelo, data de criação e checksum. Atualmente exportado em formato CSV.
Ativo Geotécnico	GET	<code>/ {tenant} /API/v1/ativos-geotecnicos/{id}/instrument/{instrumentid}/parametro/{parametrochave}/data</code>	Obtém a série temporal de um parâmetro monitorizado a partir de um sensor específico.	Estrutura padrão contendo uma lista de registos de medição: {timestamp, value, unit, qualityFlag}.
Monitorização Contínua	POST	<code>/ {tenant} /API/v1/Continuousmonitorizarizaçã/get-data</code>	Obtém fluxos de dados de monitorização contínua gerados pela instrumentação.	Devolve eventos de medição, incluindo instantes temporais e metadados que descrevem os parâmetros monitorizados.
Medições	POST	<code>/ {tenant} /API/v1/Measures/receive-values</code>	Endpoint utilizado para inserir novas medições de sensores no sistema.	A estrutura de entrada inclui normalmente {sensorId, timestamp, parameter, value}.

Tabela 16 - API LevelAMS

1.2.3.4 Disponibilidade de dados abertos do modelo analítico

Os resultados do modelo analítico produzidos neste caso de uso derivam de dados de monitorização operacional de infraestrutura viária ativa sob a concessão da Brisa. Estes dados

refletem as condições estruturais em tempo real de ativos críticos e constituem informação proprietária pertencente à Brisa. Como tal, a divulgação pública irrestrita não é aplicável, pois entraria em conflito com as políticas de governação de dados da Brisa e levantaria preocupações legítimas relativamente à segurança e integridade da infraestrutura monitorizada.

O acesso aos resultados analíticos é disponibilizado através da API REST do LevelAMS, que expõe os resultados de deslocamento do M3C2 e os modelos de malha 3D através de endpoints autenticados. O acesso é concedido através de um token Bearer JWT, emitido após a validação das credenciais, garantindo que a obtenção de dados é controlada, auditável e restrita a partes autorizadas. Investigadores qualificados e parceiros institucionais podem solicitar acesso às credenciais através do departamento de geotecnia da Brisa.

Esta abordagem está em conformidade com os princípios de dados FAIR (Findable, Accessible, Interoperable e Reusable — Localizável, Acessível, Interoperável e Reutilizável), respeitando simultaneamente as restrições de governação de dados sensíveis de infraestrutura. Os dados são localizáveis através da API documentada, acessíveis sob condições definidas, interoperáveis através de formatos padrão (CSV e OBJ) e reutilizáveis dentro do âmbito de acordos de utilização autorizados.

1.2.4 Serviço de carregamento LevelAMS

Este serviço monitoriza ativos geotécnicos (inclinómetros, células de carga, etc.) e sincroniza o seu estado no mundo real com a plataforma de Gémeo Digital, mantendo a representação digital alinhada com a infraestrutura física.

- **Entrada:** Leituras de sensores geotécnicos obtidas a partir dos endpoints de gestão de ativos.
- **Saída:** Atualizações sincronizadas de Gémeos Digitais para cada ativo monitorizado, com instantes temporais (timestamps) e indicadores de qualidade.
- **Variáveis Principais:**
 - Identificador e tipo do ativo (inclinómetro, célula de carga, etc.)
 - Valor da leitura e unidade de medida
 - Intervalo de sincronização (agendamento via CronJob)
 - Valores-limite para alertas
- **Lógica algorítmica:**
 1. No agendamento (Kubernetes CronJob), executar um ciclo de sincronização.
 2. Consultar a API do LevelAMS para o ativo geotécnico e a sua lista de instrumentos.
 3. Para cada instrumento, obter as leituras de parâmetros mais recentes (de forma incremental, com base no instante temporal `modified` do Ditto).
 4. Avaliar o `MeasurementState` (OK / ALERT / ALARM) em relação aos limites configurados.
 5. Criar ou atualizar as Ditto Things para o ativo e para cada instrumento.
 6. Opcionalmente, agrupar e enviar eventos históricos para a API Histórica através de um POST.

1.2.4.1 Formatos de resposta da API LevelAMS

GeotechnicalAsset

Campo	Tipo
<i>id</i>	<i>string</i>
<i>matricula</i>	<i>string</i>
<i>localizacao</i>	<i>string</i>
<i>tipoActivo</i>	<i>ValueNamePair</i>
<i>latitude</i>	<i>number (-90..90)</i>
<i>longitude</i>	<i>number (-180..180)</i>
<i>indiceCondicaoAtual</i>	<i>number</i>
<i>indiceFiabilidade</i>	<i>number</i>
<i>altura</i>	<i>number</i>
<i>extensao</i>	<i>number</i>
<i>inclinacaoGraus</i>	<i>integer</i>
<i>pkExploracaoInicial</i>	<i>integer</i>
<i>pkExploracaoFinal</i>	<i>integer</i>
<i>pkProjectInicial</i>	<i>integer</i>
<i>pkProjectFinal</i>	<i>integer</i>

Tabela 17 - Ativo Geotécnico

Instrument

Campo	Tipo
<i>instrumentoId</i>	<i>string</i>
<i>tipoInstrumento</i>	<i>ValueNamePair</i>
<i>matricula</i>	<i>string</i>
<i>pkInicial</i>	<i>integer</i>
<i>parametros</i>	<i>Parameter[]</i>

Tabela 18 - Instrumento

Parameter (instrument definition)

Campo	Tipo
<i>parametroNome</i>	<i>string</i>
<i>parametroChave</i>	<i>string</i>
<i>unidades</i>	<i>string</i>
<i>limiteAlerta</i>	<i>Limit (opcional)</i>
<i>limiteAlarme</i>	<i>Limit (opcional)</i>

Tabela 19 - Parâmetro (definição do instrumento)

Limit

Campo	Tipo
<i>critério</i>	<i>ValueNamePair</i>
<i>valor</i>	<i>number</i>
<i>variacaoAbsoluta</i>	<i>number</i>

Tabela 20 - Limite

ValueNamePair

Campo	Tipo
<i>value</i>	<i>string</i>
<i>name</i>	<i>string</i>

Tabela 21 - ValueNamePair

Parameter (reading de medição)

Campo	Tipo
<i>timestamp</i>	<i>datetime</i>
<i>valor</i>	<i>number</i>

Tabela 22 - Parameter (reading de medição)

1.2.4.2 Representações Ditto Thing

Existem duas Things definidas para o carregamento dos dados disponíveis pela API da LevelAMS, sendo uma a representação de um ativo geotécnico e a outra representação dos instrumentos de cada ativo.

Nome do campo		Tipo	Indexado?
Attributes	id	UUID	sim
	matricula	String	sim
	localizacao	String	sim
	tipoActivo	ValueNamePair<AssetTypeEnum>	não
	latitude	Float	não
	longitude	Float	não
	geotile	Integer	sim
	indiceCondicaoAtual	Float	não
	indiceFiabilidade	Integer	não
	altura	Integer	não
	extensao	Integer	não
	inclinacaoGraus	Integer	não
	pkExploracaoInicial	Integer	não
	pkExploracaoFinal	Integer	não
	pkProjectInicial	Integer	não
	pkProjectFinal	Integer	não
	instrumentList	List<InsrumentThingId>	não

Tabela 23 - Definição de um Thing que Representa um Ativo Geotécnico

O InstrumentThingId segue o seguinte padrão: {id do geotechnical asset}.instrument.{matrícula do instrument}.

Valores disponíveis para o AssetTypeEnum:

- ActivoGeotecnicoTipoActivo_Muro
- ActivoGeotecnicoTipoActivo_Talude

Tabela 27 - Definição do Thing que Representa um Instrumento

Nome do campo		Tipo	Indexado?
Attributes	instrumentId	UUID	yes
	tipoInstrumento	NameValuePair<InstrumentTypeEnum>	no
	matricula	String	yes
	pkInicial	Integer	no
	coordinates	latitude	Float
		longitude	Float
		altitude	Float
	geotile	Integer	yes
	dashboardURL	String	no

Features	<Paramater Key>	parametroNome	String	no
		parametroChave	String	no
		unidades	String	no
		limiteAlerta	Limit (opcional)	no
		limiteAlarme	Limit (opcional)	no
		latestValue	timestamp	String
			valor	Float
		state	MonitorStateEnum	no

Avaliação do MonitorStateEnum

Estado	Condição
OK	Valor dentro dos limites normais (ou sem limites definidos)
ALERT	Valor excede o limite de alerta do instrumento ou do tipo de instrumento
ALARM	Valor excede o limite de alarme do instrumento ou do tipo de instrumento

Tabela 24 - Avaliação do MonitorStateEnum

Os limites por instrumento têm precedência sobre os limites por tipo de instrumento.

1.2.5 Resultados

Foram obtidos levantamentos tridimensionais de alta resolução do muro de suporte através de campanhas de varrimento laser móvel, realizadas em diferentes intervalos temporais. Estes levantamentos geraram nuvens de pontos densas, representando a geometria do muro e do terreno circundante.

Antes da comparação, foram aplicadas várias etapas de pré-processamento para garantir a consistência dos conjuntos de dados. Estas incluíram a segmentação das superfícies relevantes, a remoção de áreas não comparáveis, o recorte espacial para garantir cobertura idêntica entre levantamentos, e o registro das nuvens de pontos.

A comparação geométrica entre épocas de aquisição foi realizada utilizando o método open-source Multiscale Model to Model Cloud Comparison (M3C2). Este método calcula as distâncias entre duas nuvens de pontos ao longo da normal local da superfície, permitindo a detecção precisa de variações geométricas entre campanhas de monitorização.

O campo escalar resultante representa as distâncias medidas entre as duas superfícies e permite a identificação das áreas onde ocorreram alterações geométricas.

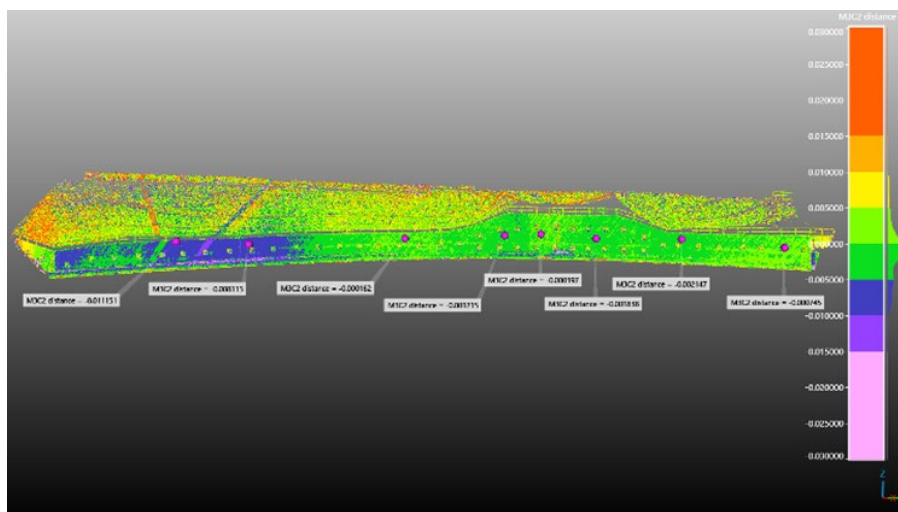


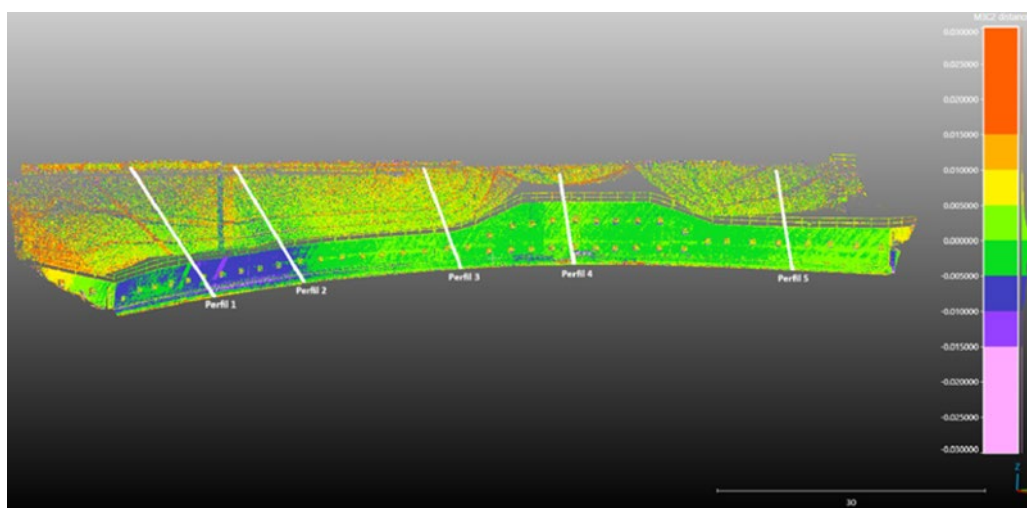
Figura 17 - Resultados de deslocamento medidos em metros (m).

Para o troço de muro de suporte analisado, os resultados mostram magnitudes de deslocamento muito limitadas, indicando um comportamento estrutural globalmente estável. Após a remoção de outliers, as distâncias medidas variam entre aproximadamente $-0,011$ m e $0,012$ m, com uma Raiz do Erro Quadrático Médio (RMS) de aproximadamente $0,008$ m. O valor médio das distâncias medidas é próximo de zero, indicando a ausência de deslocamento sistemático em toda a estrutura analisada.

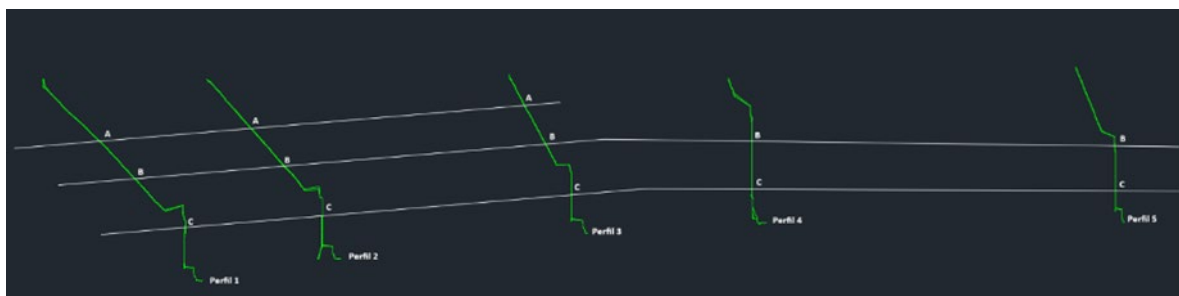
A maior parte da superfície do muro apresenta valores de deslocamento dentro do intervalo $-0,005$ m a $0,005$ m, confirmando a magnitude limitada das alterações geométricas detetadas entre as duas campanhas de monitorização.

Uma zona localizada, próxima da extremidade sul do muro, apresenta deslocamentos negativos ligeiramente superiores, atingindo aproximadamente $-1,1$ cm ao longo de uma extensão de cerca de 20 m. Estes valores sugerem um pequeno movimento do muro para dentro, relativamente ao aterro, embora a magnitude permaneça reduzida e não indique instabilidade estrutural significativa.

Os resultados obtidos a partir da comparação das nuvens de pontos foram ainda validados através de perfis transversais e comparados com medições obtidas pela instrumentação geotécnica instalada no muro de suporte, incluindo as leituras dos inclinómetros. A comparação confirmou a consistência dos padrões de deslocamento detetados e reforçou a fiabilidade da análise de deteção remota.



A)



B)

Figura 18 - Identificação de perfis no muro para uma análise mais detalhada: A) Imagem 3D; B) pontos de validação

1.2.5.1 Texture Creation do muro de suporte

Para apoiar a camada de visualização, é importante converter a nuvem de pontos adquirida e os respetivos dados de deslocamento num modelo com uma textura incorporada (neste caso, um ficheiro .gltf). Para conseguir isto automaticamente, foi desenvolvido um pipeline de execução periódica. Este pipeline obtém o polígono PLY em branco e a nuvem de pontos a partir da API REST fornecida pelo parceiro. A nuvem de pontos, obtida como ficheiro CSV, contém 5 campos importantes: as coordenadas X, Y e Z, juntamente com os campos `Quality_flag` e `dist_3d_m`. Os primeiros campos são importantes para localizar geograficamente (e, portanto, geometricamente) esse ponto em relação ao polígono obtido. O último contém a flag de Alert/Alarm que classifica se um determinado ponto é motivo de preocupação, juntamente com o deslocamento exato do ponto em comparação com nuvens de pontos anteriores. Os níveis de alerta/alarme estão diretamente correlacionados com uma cor correspondente: verde se o ponto não for motivo de preocupação, amarelo se o deslocamento do ponto for motivo de alerta, e vermelho se o deslocamento do ponto for motivo de alarme. Os valores de deslocamento são, em vez disso, convertidos num gradiente de cor fornecido pelo parceiro, que varia de magenta a vermelho dependendo do deslocamento do ponto. Os valores negativos são coloridos de magenta a verde, enquanto os valores positivos são coloridos de verde a vermelho. Depois de atribuir a cada ponto a sua cor em cada textura, estes pontos são projetados sobre o polígono previamente obtido. Isto é feito encontrando o ponto mais próximo no polígono relativamente a cada ponto da nuvem de pontos. A partir daqui, o valor de cor do ponto é "pintado" sobre o polígono. Este processo é repetido para todos os pontos da nuvem de pontos, sendo o resultado exportado como um ficheiro .gltf. Este tipo de ficheiro foi escolhido devido ao seu tamanho compacto, à facilidade de interoperabilidade com software de visualização e ao facto de incluir a geometria e a textura do polígono num único ficheiro. O ficheiro resultante é depois carregado no armazenamento compatível com S3 da infraestrutura, tornando-se disponível para o software de visualização.

Para garantir que os recursos computacionais não são desperdiçados, antes de solicitar o polígono e a nuvem de pontos à API REST fornecida, é realizado um pedido HEAD para obter a data de última modificação dos ficheiros. Se os ficheiros disponibilizados pela API do parceiro forem mais antigos do que as texturas já existentes no armazenamento compatível com S3, então as texturas já estão atualizadas e a execução pode ser interrompida. Isto foi feito para poupar não só recursos computacionais na infraestrutura, mas também largura de banda, devido ao tamanho dos ficheiros transferidos.

1.2.5.2 Monitorização de instrumentos do muro de suporte

Para monitorizar cada instrumento do muro de suporte, estão disponíveis dois painéis Grafana — um para células de carga e outro para inclinómetros. O painel das células de carga consolida dados em tempo real e históricos de todos os instrumentos de células de carga numa única interface. Este suporta análise por instrumento, em que o utilizador pode escolher a célula de carga a partir de uma lista suspensa, com as visualizações individuais a serem atualizadas de acordo com o instrumento selecionado, fornecendo simultaneamente uma visão geral de alto nível. Este painel inclui cinco visualizações:

- **Fotografia do instrumento** — fotografia do muro de suporte com a célula de carga atualmente selecionada.
- **CC - Carga** — evolução em série temporal da carga medida pela célula de carga selecionada, em quilonewtons (kN). Duas linhas horizontais tracejadas representam os limiares de alerta e alarme de cada instrumento, permitindo avaliar visualmente se as leituras acionaram, ou estão próximas de acionar, uma condição de alerta ou alarme.
- **CC - Temperatura** — evolução em série temporal da temperatura registada pela célula de carga selecionada, em graus Celsius (°C).
- **Leituras de Ancoragem (todas as CCs)** — evolução em série temporal da carga de todos os instrumentos de células de carga simultaneamente, permitindo a comparação entre instrumentos.
- **Alertas e Alarmes (todas as CCs)** — tabela que lista todos os instrumentos de células de carga juntamente com o seu estado de alerta atual e o valor medido mais recente. Cada instrumento é codificado por cor de acordo com o seu estado — verde (OK), amarelo (ALERT), vermelho (ALARM) e cinzento (INDISPONÍVEL).



Figura 19 - Painel de monitorização de células de carga com o instrumento A19 (em estado OK) selecionado

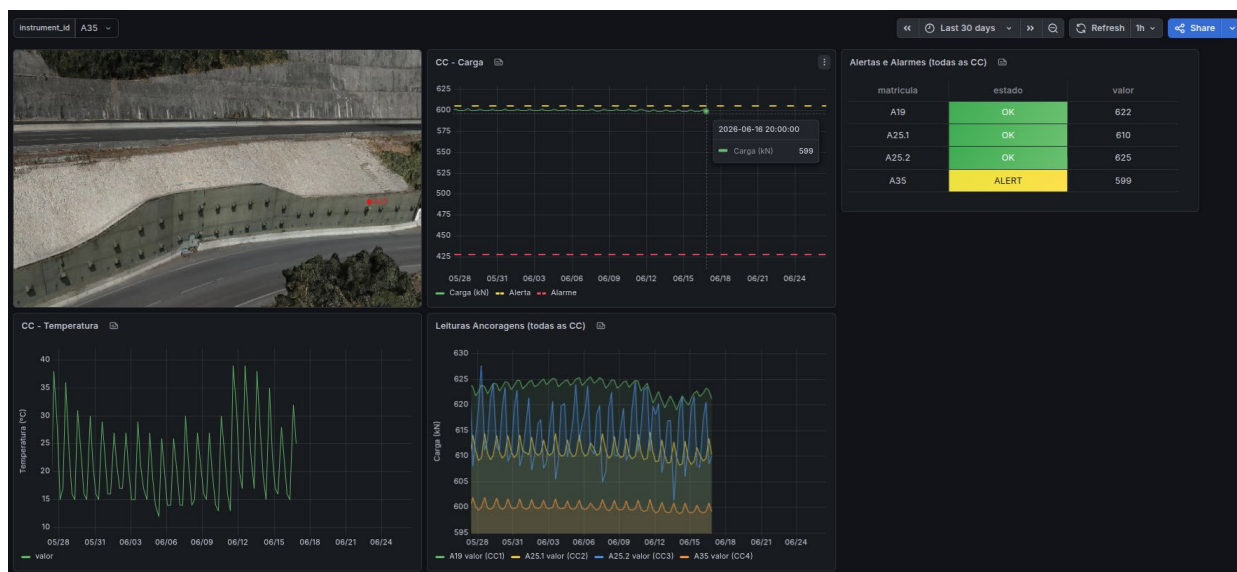


Figura 20 - Painel de monitorização de células de carga com o instrumento A35 (em estado ALERT) selecionado

O painel dos inclinómetros segue uma estrutura semelhante, mas é adaptado às características específicas dos dados de inclinómetros. Ao contrário das células de carga, que produzem uma única medição por leitura, os inclinómetros medem o deslocamento do solo em múltiplas profundidades ao longo do seu comprimento, exigindo visualizações dedicadas para transmitir o perfil de profundidade completo de cada instrumento. Este painel inclui sete visualizações:

- **Fotografia do instrumento** — fotografia do muro de suporte com o inclinómetro atualmente selecionado.
- **Alertas e Alarmes - Direção A** — tabela que lista todos os instrumentos de inclinómetro juntamente com o seu estado de alerta atual na direção X (Direção A), incluindo as profundidades específicas nas quais foram acionadas condições de alerta ou alarme.
- **Alertas e Alarmes - Direção B** — tabela equivalente para a direção Y (Direção B).
- **Deslocamento Acumulado - Direção A** — gráfico de perfil de profundidade que mostra a deflexão acumulada na Direção A em cada profundidade de medição. São apresentadas múltiplas séries, cada uma correspondendo a uma data em que foram realizadas medições in situ, permitindo a comparação da deflexão entre diferentes campanhas de medição ao longo do tempo.
- **Desvio - Direção A** — gráfico de perfil de profundidade que mostra, para cada campanha de medição, o desvio na Direção A relativamente à campanha mais antiga (utilizada como leitura de referência), com linhas de alerta e alarme sobrepostas em cada profundidade.
- **Deslocamento Acumulado - Direção B** — perfil equivalente de deflexão acumulada para a Direção B.
- **Desvio - Direção B** — perfil de desvio equivalente para a Direção B relativamente à campanha mais antiga, com linhas de limite de alerta e alarme sobrepostas.

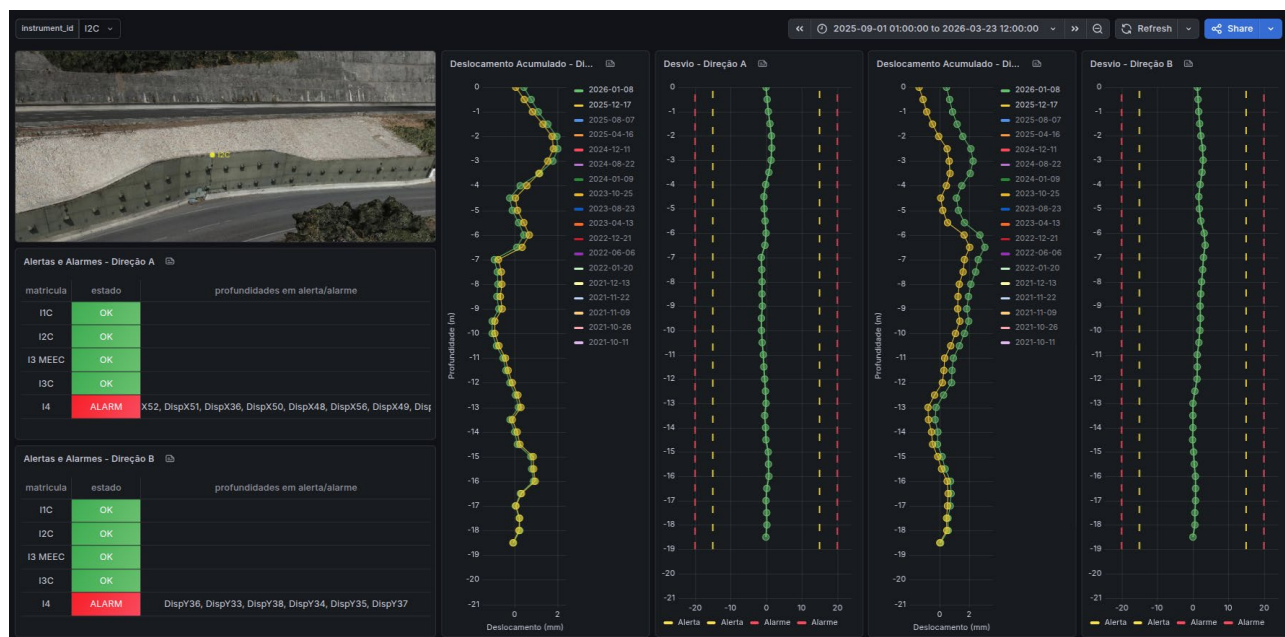


Figura 21 - Painel de monitorização de inclinómetros com o instrumento I2C (em estado OK) selecionado

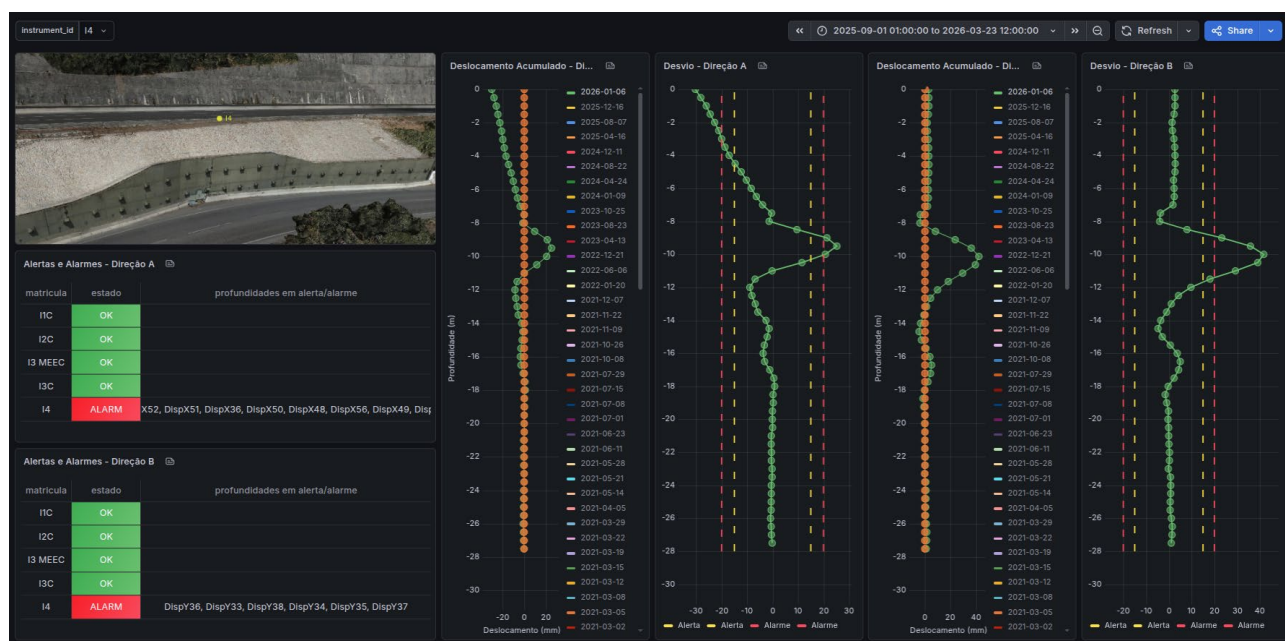


Figura 22 - Painel de monitorização de inclinómetros com o instrumento I4 (em estado ALARM) selecionado

Os estados de alerta e alarme apresentados em ambos os painéis são calculados por um agente de monitorização dedicado, implementado como um Kubernetes CronJob que é executado numa base horária. Em cada execução, o agente obtém as leituras mais recentes de cada instrumento a partir da API histórica e avalia-as em relação aos limiares configurados, de forma a derivar um estado, que é depois escrito no Ditto e armazenado como parte do Gémeo Digital de cada instrumento.

Para as células de carga, o estado é determinado comparando a medição de força mais recente com dois limiares: se o valor ficar abaixo do limiar de alarme, o instrumento é definido como ALARM; se ficar abaixo apenas do limiar de alerta, é definido como ALERT; caso contrário, permanece OK.

Para os inclinómetros, o desvio em cada profundidade é calculado como a diferença entre as campanhas de medição mais recente e mais antiga, e o valor resultante é depois avaliado em

relação aos limiares configurados, de forma independente para cada profundidade — se a diferença exceder o limiar de alarme, o estado do instrumento nessa profundidade específica é definido como ALARM; se exceder apenas o limiar de alerta, é definido como ALERT; caso contrário, permanece OK. O estado geral do inclinómetro apresentado no painel reflete a condição mais desfavorável observada em todas as profundidades, tendo o ALARM precedência sobre o ALERT.

1.2.5.3 Métricas de qualidade

As métricas de qualidade são utilizadas para avaliar a fiabilidade e a robustez dos resultados de monitorização obtidos a partir das análises de deteção remota e das medições dos sensores.

Para a comparação geométrica de nuvens de pontos, são utilizados vários indicadores estatísticos para avaliar a qualidade dos deslocamentos detetados.

Estas métricas incluem:

- deslocamento médio;
- desvio-padrão das distâncias medidas;
- erro da raiz quadrática média (RMS);
- percentis de deslocamento.

Para o conjunto de dados do muro de suporte analisado, o valor da Raiz do Erro Quadrático Médio (RMS) obtido a partir da comparação das nuvens de pontos foi de aproximadamente **0,008 m**, indicando um elevado nível de consistência entre os conjuntos de dados analisados e confirmando a fiabilidade das variações geométricas detetadas.

Estas métricas permitem a identificação de potenciais outliers e fornecem medidas quantitativas de incerteza nos resultados de monitorização.

1.3 Caso de uso: gestão de autoestradas resilientes às alterações climáticas através da previsão de risco de incêndio

As alterações climáticas estão a provocar incêndios mais frequentes e severos, capazes de perturbar e ameaçar infraestruturas de mobilidade críticas, como as autoestradas. Para melhorar a prevenção e permitir respostas mais rápidas, verifica-se uma transição da abordagem reativa atual para uma gestão preditiva destas situações, garantindo uma atuação mais célere por parte das entidades responsáveis.

Este caso de uso tira partido da plataforma de Gémeo Digital para realizar simulações de alta resolução que preveem a progressão e o comportamento de incêndios florestais, prevendo o impacto na infraestrutura, principalmente nos corredores de autoestrada.

Esta solução é construída sobre a infraestrutura existente do Eclipse Ditto e os seus componentes envolventes, para armazenar as atualizações provenientes das simulações. O núcleo do sistema de simulação é o motor de simulação ForeFire. Toda a integração é gerida através de um serviço Python que interage com a API do Ditto e executa as simulações do ForeFire. Em conjunto, estas ferramentas tornam toda a solução totalmente baseada em software open-source.

1.3.1 Motor de simulação: ForeFire

Entre as ferramentas open-source disponíveis para simulação de incêndios, destacam-se o ForeFire e o Cell2Fire. Embora ambos suportem a modelação da propagação de fogo, diferem

significativamente na sua abordagem conceptual, nos pressupostos físicos assumidos e na facilidade de integração operacional.

Critério	Cell2Fire	ForeFire
Modelo base	Estocástico baseado em células (grealha regular)	Semi-empírico (equações de Rothermel)
Principal vantagem	Avaliação de risco florestal, múltiplos cenários	Robustez física, integração com dados observados
Integração meteorológica em tempo real	Menos direta	Nativa (via script .ff ou NetCDF)
Ficheiros de entrada	Formato específico da ferramenta	Modular e padronizado (fuels.csv, landscape.nc, .ff)
Adequação para este caso de uso	Limitada para integração operacional	Recomendado

Tabela 25 - Comparação entre ForeFire e Cell2Fire

O ForeFire implementa um modelo de propagação semi-empírico baseado nas equações de Rothermel, amplamente utilizado na modelação de incêndios de superfície. O comportamento do fogo é descrito através da Taxa de Propagação (ROS), intensidade, e da influência combinada do vento, declive e propriedades do combustível. Esta base física proporciona maior robustez ao pretender aproximar condições reais e integrar dados meteorológicos observados.

De uma perspetiva tecnológica, o ForeFire apresenta uma estrutura modular baseada em ficheiros de entrada padronizados — fuels.csv, landscape.nc e o script de simulação (.ff) — o que facilita a automação e a integração com serviços externos. Esta característica é essencial para um sistema operacional que necessita de gerar cenários com base em entradas variáveis.

1.3.2 Fluxo de trabalho proposto

O fluxo de trabalho operacional inclui duas etapas sequenciais, acionadas após a deteção de um incidente de incêndio, dependendo do risco identificado na primeira etapa.

Etapa 1:

Após a inserção de um incidente no Gémeo Digital, é criado um cone de propagação simplificado para vários horizontes temporais (30, 60, 90 e 120 minutos). O cone é calculado com base na direção e velocidade do vento. O cone tem 15° de abertura e o fogo propaga-se a 18% da velocidade do vento; com esta condição, obtemos o seguinte modelo que descreve o comportamento do incêndio e a taxa de propagação (ROS):

Fire direction= wind + 180°; Cone = ±15°;

Rate = 0.18 × wind speed (km/h)

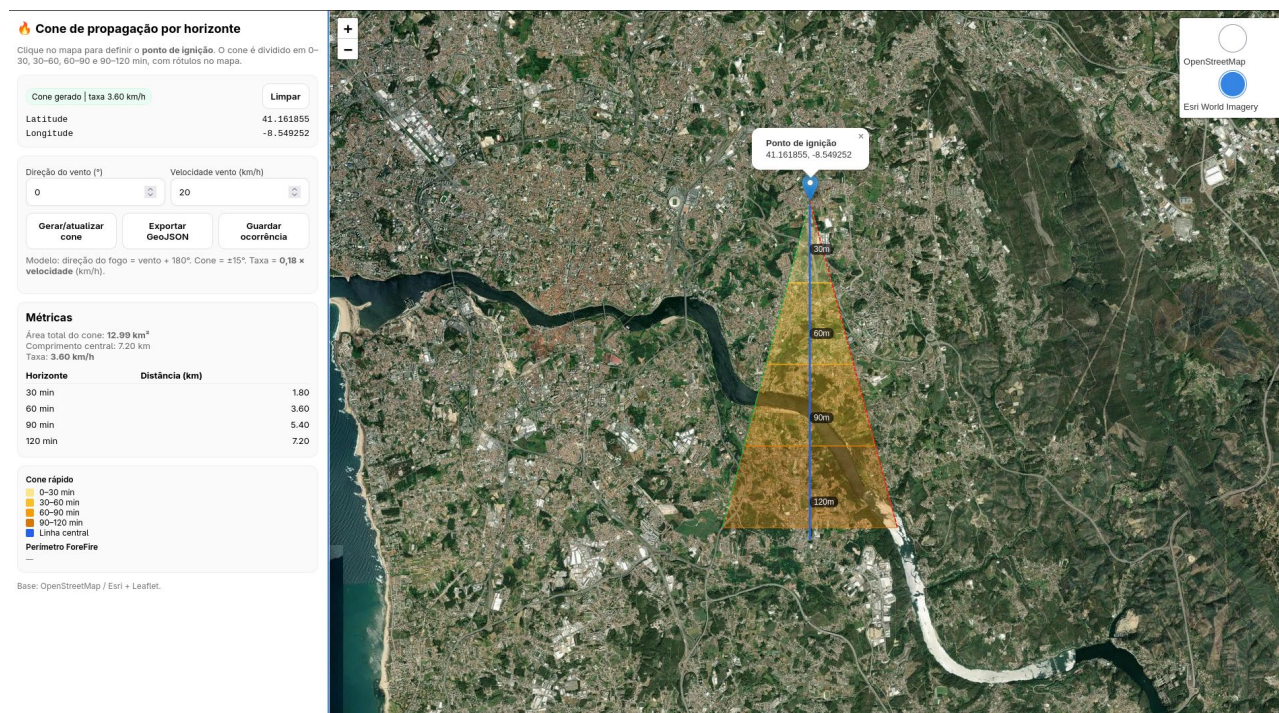


Figura 23 - Cálculo do cone de fogo

O ponto de ignição e o cone de propagação gerado são armazenados no Gémeo Digital. Durante este período, é também realizada uma interseção geométrica entre o cone e a zona de risco potencial da infraestrutura viária. Se existir interseção entre as duas geometrias, é executada a segunda etapa do pipeline.

Etapa 2:

Quando ocorre a interseção, o pipeline de simulação ForeFire é executado automaticamente. Este pipeline abrange todo o processo, desde a preparação dos ficheiros para execução, até à execução da simulação através do motor ForeFire e ao armazenamento dos resultados no Gémeo Digital.

O fluxo de trabalho segue a seguinte sequência:

1. **Preparação da entrada:** Obtenção das estações meteorológicas mais próximas, recorte raster em torno do ponto de ignição e obtenção das classificações do solo da área.
2. **Exportação da entrada:** Geração dos ficheiros necessários para o motor ForeFire. O ForeFire necessita do ficheiro NetCDF, que contém toda a informação de elevação, vento e classes de combustível, de um ficheiro CSV de combustíveis que mapeia as classes de combustível para o modelo Rothermel, e do script de simulação (.ff).
3. **Execução do motor:** O motor produz múltiplos ficheiros GeoJSON contendo os perímetros do incêndio para vários intervalos temporais (15, 30, 60, 90, 120 minutos); estes ficheiros são armazenados num object storage, juntamente com um modelo 3D gerado a partir dos perímetros. Os URLs destes recursos são atualizados no gémeo digital.
4. **Guardar resultados:** No final da execução, o serviço guarda todos os resultados no gémeo digital e nas soluções de armazenamento utilizadas. Estes resultados incluem o cone e os perímetros de simulação, bem como os respetivos modelos 3D.

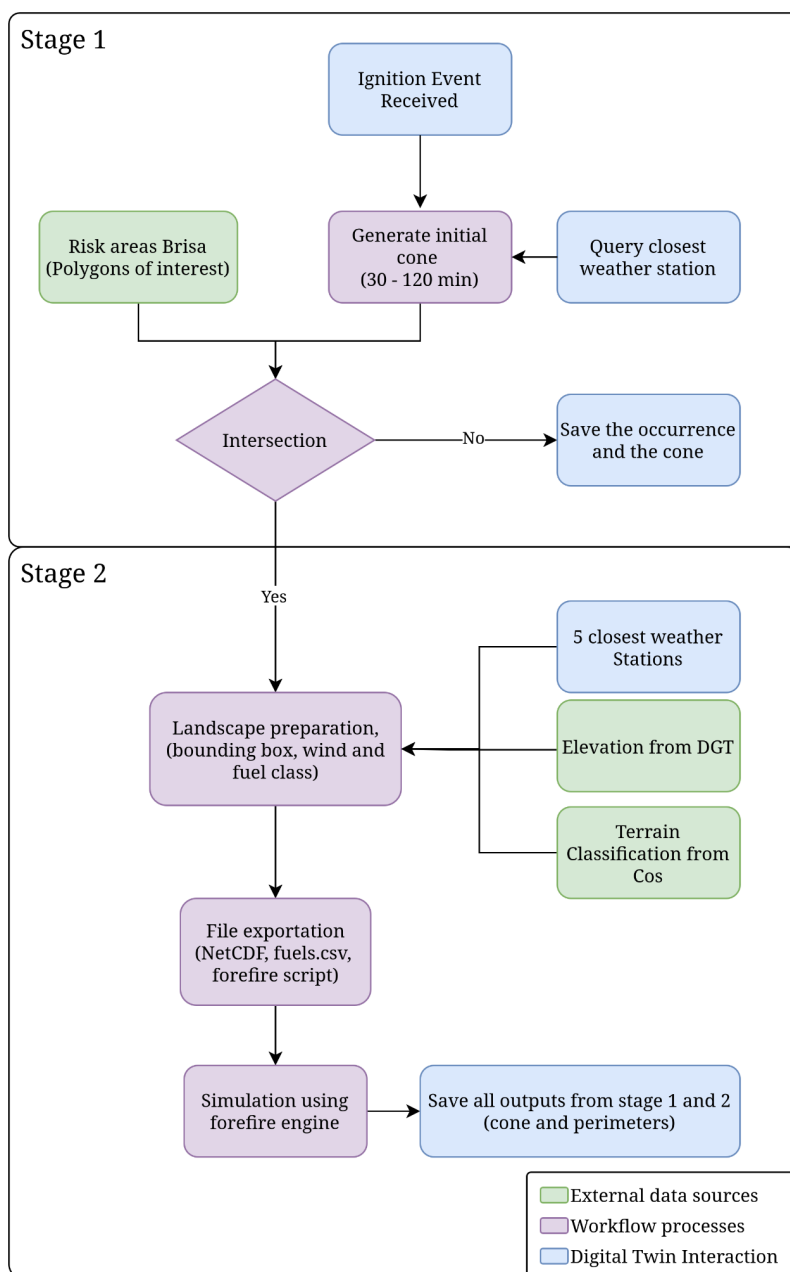


Figura 24 - Fluxo proposto: desde a entrada dinâmica de incidentes e análise espacial imediata até ao pipeline de extração e simulação ForeFire

1.3.3 Fontes de dados

Para alcançar simulações precisas de propagação de incêndios, é essencial integrar dados geológicos e meteorológicos no fluxo de modelação, para que o motor de simulação, ForeFire, possa produzir os resultados mais fiáveis possíveis. As condições do terreno, o comportamento do vento e as classes de combustível ou vegetação na área têm um impacto significativo na propagação do fogo, tornando crítico manter estes dados continuamente atualizados de acordo com as condições reais, especialmente a componente meteorológica, que se altera frequentemente.

Neste fluxo de trabalho, os principais conjuntos de dados utilizados são o DEM (Modelo Digital de Elevação) obtido a partir do LiDAR da DGT e a COS (Carta de Ocupação do Solo) de Portugal. Para a informação meteorológica, os dados das estações meteorológicas são obtidos através da plataforma de Gémeo Digital, que consome informação em tempo real fornecida pelo Instituto Português do Mar e da Atmosfera (IPMA).

Obtenção e pré-processamento do DEM e COS

Para obter os dados de terreno — nomeadamente o Modelo Digital de Elevação (DEM) e a Carta de Ocupação e Uso do Solo (COS) — é executado um job do Kubernetes durante a implantação inicial. Este job executa um script Python que descarrega e pré-processa os conjuntos de dados em formatos mais adequados para utilização subsequente no sistema.

O DEM é derivado do conjunto de dados LiDAR da DGT, com uma resolução espacial de 10 metros, e é disponibilizado pelo CIIMAR através do endereço fc.up.pt/pessoas/jagoncal/dtm-lidar-dgt/.

Após a descarga, o DEM é reprojetoado para o sistema de referência de coordenadas EPSG:4326, de forma a garantir a compatibilidade com o fluxo de trabalho do ForeFire. Este conjunto de dados fornece a base topográfica para o landscape do ForeFire. Utilizando o DEM, o ForeFire calcula o declive e a orientação do terreno, que são parâmetros críticos para o modelo de propagação de fogo, uma vez que influenciam significativamente o comportamento e os padrões de propagação do incêndio.

O modelo de propagação de fogo utilizado neste fluxo de trabalho baseia-se nos modelos de combustível de Rothermel. Consequentemente, é necessária uma classificação de ocupação do solo compatível com estes modelos de combustível. Para tal, o conjunto de dados da Carta de Ocupação do Solo (COS) é descarregado a partir do SNIG (snig.dgterritorio.gov.pt) e pré-processado. Este pré-processamento consiste no mapeamento das classes originais de ocupação do solo da COS para um novo conjunto de categorias que podem ser associadas aos modelos de combustível de Rothermel.

Esta classificação permite caracterizar cada classe de ocupação do solo de acordo com os parâmetros exigidos pelas equações de Rothermel, incluindo a carga de combustível, a razão superfície-volume, o conteúdo calorífico, as categorias de humidade do combustível morto e vivo, e a profundidade do leito de combustível. É então aplicado um cruzamento entre as classes da COS e os modelos de combustível de Rothermel, produzindo uma camada de modelo de combustível que pode ser diretamente integrada no fluxo de simulação do ForeFire.

A tabela abaixo representa o ficheiro `fuels.csv`, utilizado pelo modelo ForeFire, garantindo que a simulação sabe como o fogo reage em cada classe.

	modelo	carga de combustível (t/ha)	surface-to-volume (1/m)	extinção de humidade	Descrição Técnica
Pasto_Seco	Rothermel	1.5	3500	0.12	Pastagens secas e combustíveis finos. Propagação muito rápida.
Mato_Denso	Rothermel	13.0	2000	0.20	Arbustos densos como giestas e urzes. Elevada libertação de calor.
Pinhal_Bravo	Rothermel	7.0	1500	0.25	Floresta de pinheiro-bravo com manta morta de agulhas. Fogo de superfície moderado.

Eucaliptal_Limp o	Rothermel	12.0	4500	0.25	Povoamentos de eucalipto geridos com subcoberto limpo.
Eucaliptal_Aban donado	Rothermel	25.0	5500	0.20	Crítico: elevada carga de biomassa, casca solta e camada de arbustos acumulada.
Sobreiral_Alcor	Rothermel	5.0	1800	0.30	Povoamentos de sobreiro/azinheira. Propagação mais lenta e controlável.

Tabela 26 - Diferentes definições de combustível no ForeFire

Estações de vento

Os dados de vento são obtidos a partir de estações meteorológicas integradas no Gémeo Digital. Estas estações estão disponíveis no namespace *internal:meteo* e são inseridas através de um serviço que obtém dados fornecidos pelo IPMA. As estações meteorológicas são consultadas com base na sua proximidade espacial, utilizando um índice de geotile. Isto é conseguido aplicando um filtro RSQL na consulta à API do Eclipse Ditto, especificando um intervalo de valores de geotile através de limites inferior e superior:

```
ge(attributes/geotile,{lower_qk}),le(attributes/geotile,{upper_qk})
```

A consulta utiliza inicialmente geotiles a um nível de zoom predefinido. Se o número de estações devolvido for insuficiente para a simulação, o intervalo de pesquisa é progressivamente ampliado, aumentando a cobertura de geotile até se obter o número necessário de estações meteorológicas. Esta abordagem proporciona uma pesquisa espacial eficiente, garantindo simultaneamente que existem observações meteorológicas suficientes para suportar a simulação.

Preparação do landscape.nc

O ficheiro utilizado para a simulação do ForeFire é denominado landscape.nc. Contém toda a informação relevante sobre o terreno, as classes de combustível e as condições de vento. Este ficheiro é gerado na segunda etapa do pipeline, após a criação e validação da interseção do cone.

As camadas de combustível e terreno são derivadas dos conjuntos de dados DEM e COS pré-processados. É extraído um subconjunto espacial da região em torno do cone de incêndio inicial, de forma a garantir um domínio localizado, reduzindo o custo computacional durante a simulação.

A camada de vento é construída utilizando as estações meteorológicas mencionadas anteriormente. No entanto, como estas observações são fornecidas como medições pontuais discretas e o motor ForeFire requer um campo de vento contínuo em todo o domínio de simulação, é aplicado um método de interpolação. Neste fluxo de trabalho, é selecionada a Ponderação pelo Inverso da Distância (IDW), devido à sua simplicidade, eficiência computacional e adequação para gerar campos de vento espaciais contínuos a partir de observações dispersas.

Por fim, as camadas de terreno, combustível e vento são combinadas no ficheiro `landscape.nc`. Este processo garante que todas as camadas estão espacialmente alinhadas, partilham a mesma resolução e cumprem os requisitos de estrutura de entrada do motor de simulação.

Execução do motor

O motor de simulação utiliza o ficheiro `landscape.nc` como entrada, que contém os dados espaciais necessários para criar a simulação. Adicionalmente, utiliza o ficheiro `fuels.csv`, que define os parâmetros do modelo de combustível utilizados nas equações de Rothermel. O motor é configurado através de uma linguagem de scripting, onde o ficheiro `script.ff` é responsável por carregar todos os dados de entrada e parâmetros de simulação necessários antes da execução. Após a conclusão da simulação, o motor gera resultados de perímetro de incêndio a intervalos de 15 minutos, representando a propagação prevista do fogo ao longo do tempo.

1.3.4 Modelo de Rothermel

Como mencionado anteriormente, o modelo de propagação de incêndios florestais utilizado pelo motor `ForeFire` neste serviço baseia-se nas equações de Rothermel, um modelo semi-empírico amplamente adotado para simulação de incêndios florestais. O modelo é popular porque é computacionalmente eficiente, ao mesmo tempo que suporta uma vasta gama de tipos de combustível, proporcionando uma vantagem significativa em relação a modelos puramente empíricos mais simples, que assumem uma taxa de propagação constante ou dependem exclusivamente de observações de incêndios passados. O modelo de Rothermel utiliza uma descrição de combustível altamente parametrizada, com os respetivos modelos de combustível definidos no ficheiro `fuels.csv` incluído neste serviço.

Além das características do combustível, o modelo incorpora informação topográfica e meteorológica para estimar o comportamento do fogo. Os dados de elevação são utilizados para calcular o declive do terreno, que influencia diretamente a taxa de propagação do fogo. A informação meteorológica, especificamente a velocidade e direção do vento, é também incorporada para calcular tanto a taxa de propagação como a direção principal de propagação do fogo. Ao combinar as propriedades do combustível, o terreno e as condições meteorológicas, o modelo proporciona uma representação mais realista da propagação de incêndios florestais, mantendo um baixo custo computacional.

1.3.5 Arquitetura de software proposta

Uma vez que este caso de uso está integrado num sistema de Gémeo Digital mais amplo, a stack tecnológica está alinhada com o ecossistema existente. Isto garante uma integração transparente com as funcionalidades, fontes de dados e componentes de visualização já presentes na plataforma. O diagrama abaixo contém todos os componentes da arquitetura, com as respetivas responsabilidades e comunicações entre eles.

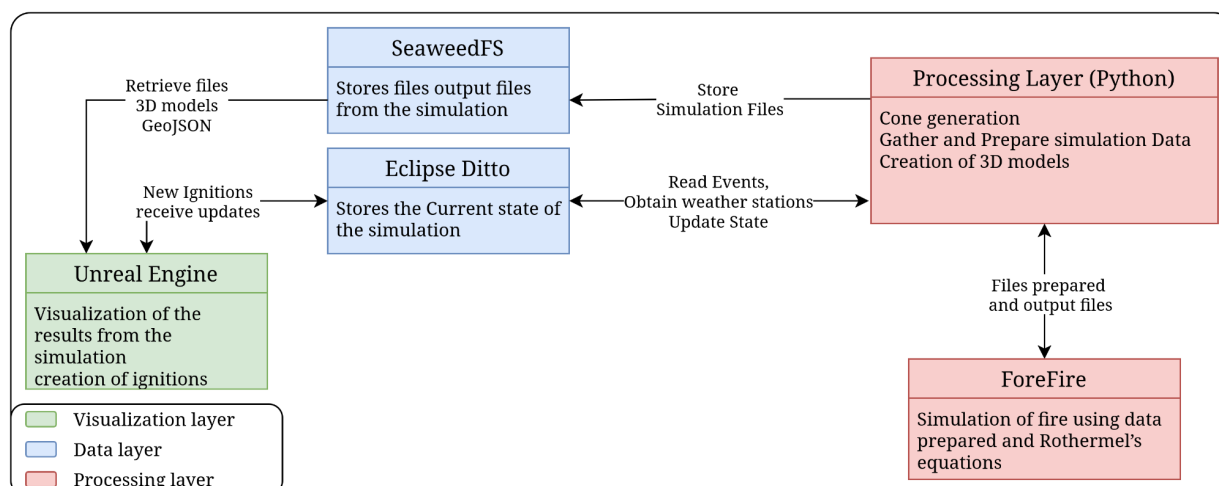


Figura 25 - Arquitetura proposta para simulação de incêndios

- Eclipse Ditto:** armazena o estado atual do incidente de incêndio e serve como a principal fonte de informação para novas ignições e dados das estações meteorológicas. O estado do incidente é continuamente atualizado no Ditto, incluindo referências aos ficheiros de resultados gerados e ao estado atual da simulação. Os estados de incidente suportados incluem:
 - NEW_IGNITION — Foi detetada uma nova ignição e aguarda processamento.
 - NO_RISK — O cone inicial não intersecta quaisquer áreas relevantes para realizar uma simulação.
 - SIMULATING — Uma simulação está atualmente em execução.
 - SIMULATED — A simulação foi concluída com sucesso, e os ficheiros resultantes estão disponíveis no SeaweedFS através dos URLs presentes no Ditto.
 - FAILED — A simulação não pôde ser concluída devido a um erro.
- SeaweedFS:** armazena os ficheiros de resultados da simulação, incluindo modelos 3D e conjuntos de dados GeoJSON que representam o cone inicial e os perímetros de simulação resultantes. Isto reforça a interoperabilidade em todo o sistema, permitindo que estes ficheiros sejam acedidos, visualizados e processados fora da plataforma principal, utilizando ferramentas externas como o QGIS.
- Camada de Processamento (Python):** é responsável por coordenar todo o fluxo de trabalho da simulação e preparar todos os dados necessários para a sua execução. Gere o pipeline de ponta a ponta, desde a receção de eventos de incidentes do Ditto através de uma ligação WebSocket, até à recolha, validação e preparação de todos os dados de entrada necessários para a simulação. Após a conclusão do processo de simulação, o orquestrador atualiza o Ditto com a informação mais recente do incidente, incluindo o estado da simulação e as referências aos ficheiros de resultados gerados. Adicionalmente, gera os modelos 3D necessários para a camada de visualização e armazena todos os ficheiros gerados pela simulação, como modelos 3D e ficheiros GeoJSON que representam o cone inicial e os perímetros de incêndio resultantes, num bucket do SeaweedFS. Isto garante que os resultados da simulação estão centralmente disponíveis para visualização, análise e integração com ferramentas e serviços externos.
- Motor de simulação (ForeFire):** motor que executa a simulação de propagação de fogo com base nas equações de Rothermel. Recebe três ficheiros de entrada — landscape.nc, fuels.csv e script.ff — e produz resultados de perímetros associados aos respetivos instantes temporais.

- **Interface de visualização:** A camada de visualização faz parte do sistema mais amplo de Gémeo Digital e é implementada através de uma aplicação 3D. A interface permite a inserção de novas ignições e suporta a visualização de múltiplos estados de incidente. Permite também observar a evolução do perímetro do incêndio ao longo do tempo. Como os resultados da simulação e os dados de suporte são armazenados num bucket do SeaweedFS e fornecidos em formato GeoJSON, podem ser integrados em qualquer interface de visualização ou UI compatível.

1.3.6 Resultado e interoperabilidade

Os resultados da simulação são armazenados em object storage, permitindo que qualquer serviço ou interface de utilizador os aceda. Isto torna os dados da simulação disponíveis não só no visualizador de infraestrutura, mas também em qualquer outro sistema que necessite deles.

O pipeline gera modelos 3D que são utilizados no visualizador de infraestrutura. Isto acrescenta a capacidade de modificar o modelo em tempo real de acordo com necessidades específicas, como mostrar mais ou menos parâmetros ou ajustar a opacidade.



Figura 26 - Visualização resultante da simulação de incêndios

1.4 Caso de uso – gestão de circulação e operações de tráfego

Este caso de uso centra-se no desenvolvimento de um Gémeo Digital operacional para o corredor A20/VCI, no Porto. O objetivo é apoiar a monitorização contínua das condições de tráfego, a deteção de incidentes, a previsão de congestionamento e a tomada de decisão operacional num dos troços mais complexos e utilizados da rede viária nacional. Este caso de uso está definido no âmbito do projeto DT4MOB como parte da responsabilidade da IP na gestão da circulação e das operações de tráfego.

Este caso de uso integra sensores LiDAR 3D instalados entre o km 11+800 e o km 12+600, juntamente com os contadores de tráfego já existentes da IP, estando prevista a incorporação de fontes de dados externas adicionais — como informação de tráfego colaborativa (Waze) e conjuntos

de dados meteorológicos (IPMA) — de forma a reforçar ainda mais as capacidades de análise de tráfego em tempo real e preditiva.

Para a Infraestruturas de Portugal, este caso de uso centra-se na melhoria da operação de tráfego e da tomada de decisão no corredor A20/VCI. Ao tirar partido do Gémeo Digital, a IP pretende reforçar a consciência situacional, antecipar congestionamentos e incidentes, e avaliar cenários operacionais antes de qualquer intervenção. Isto contribui para uma gestão de tráfego mais proativa, tempos de resposta reduzidos e maior eficiência na alocação de recursos operacionais.

1.4.1 Fontes de dados

As fontes de dados utilizadas neste caso de uso de Gestão de Mobilidade incluem:

- Sensores LiDAR 3D, instalados na VCI para realizar a deteção contínua em tempo real dos movimentos, velocidades, trajetórias e padrões comportamentais dos veículos, sob diversas condições ambientais.
- Os contadores de tráfego já existentes da IP fornecem volumes de tráfego históricos e em tempo real, por faixa, essenciais para a calibração, validação e reconstrução de fluxos nos modelos analíticos e de simulação.
- Estão previstas outras fontes de dados externas a integrar futuramente, em conformidade com a arquitetura do modelo de dados do DT4MOB descrita no D2:
 - Alertas de tráfego colaborativos (Waze) — incidentes, obstáculos, relatos de congestionamento e velocidades anómalas.
 - Dados meteorológicos (IPMA) — condições ambientais e avisos meteorológicos para contextualizar padrões de mobilidade.
- Os dados operacionais internos da IP, como restrições de faixa, obras de manutenção e registos de incidentes registados pelo Centro de Controlo de Tráfego (CCT), fornecem contexto operacional para os padrões de mobilidade.

Em conjunto, estas fontes suportam uma representação de Gémeo Digital progressivamente enriquecida e contextualizada.

1.4.2 Contadores de tráfego e LiDARs

Este caso de uso emprega uma arquitetura de sensorização complementar:

Contadores de tráfego

Fornecem dados históricos de fluxo com vários anos, para calibração e validação da simulação microscópica de tráfego (SUMO), e apoiam a reconstrução de perfis horários de fluxo utilizados em modelos preditivos e análise de cenários.

Sensores LiDAR 3D

Fornecem deteções de veículos de alta resolução em tempo real, permitem o rastreio de trajetórias, a estimativa de velocidade e aceleração, e a deteção de anomalias comportamentais, operando eficazmente mesmo em condições de visibilidade adversas.

Os dados dos LiDAR 3D são processados com software de perceção especializado (por exemplo, Flasheye), que gera deteções estruturadas compatíveis com os modelos de dados do DT4MOB.

1.4.3 Formatos de dados

Dados de percepção de tráfego LiDAR

Os dados de percepção de tráfego / LiDAR são fornecidos como texto estruturado em formato JSON, contendo, para cada objeto detetado, um identificador, o instante temporal da deteção, a classe ou tipo do objeto correspondente, a posição do objeto expressa em coordenadas locais ou em coordenadas geográficas WGS84, a velocidade e direção de movimento estimadas e, quando aplicável, uma trajetória com múltiplos pontos que descreve o percurso do objeto ao longo do tempo.

Dados de contadores de tráfego

Os contadores de tráfego instalados ao longo da A20/VCI fornecem registos de deteção por veículo, incluindo o instante temporal e a faixa de passagem, permitindo a reconstrução de perfis de tráfego detalhados. Estes registos individuais são utilizados para construir métricas agregadas (como fluxo horário ou por faixa) e para calibrar tanto os modelos de simulação microscópica como os modelos preditivos de tráfego. Estes dados são fornecidos diretamente pela infraestrutura de deteção já existente da IP.

1.4.4 Monitorização de contadores de tráfego

Para monitorizar a informação dos sensores da A20/VCI, está disponível um painel Grafana, apresentado na Figura 27. Este painel consolida os dados de tráfego em tempo real e históricos de todos os sensores da A20/VCI numa única interface, suportando análise por sensor através de um seletor suspenso, com todas as visualizações a serem atualizadas de acordo com o sensor e o intervalo temporal selecionados. Este painel inclui quatro visualizações:

- "Contagem de veículos" — evolução em série temporal do número de veículos detetados pelo sensor selecionado, agregada em intervalos temporais derivados do período selecionado;
- "Velocidade dos veículos" — evolução em série temporal das velocidades máxima, média e mínima dos veículos registadas pelo sensor selecionado, em quilómetros por hora (km/h), também agregada em intervalos temporais derivados do período selecionado;
- "Velocidade mínima atual / Velocidade máxima atual / Velocidade média atual" — três painéis de valor único que apresentam, respetivamente, a velocidade mínima, máxima e média mais recentes registadas pelo sensor selecionado.

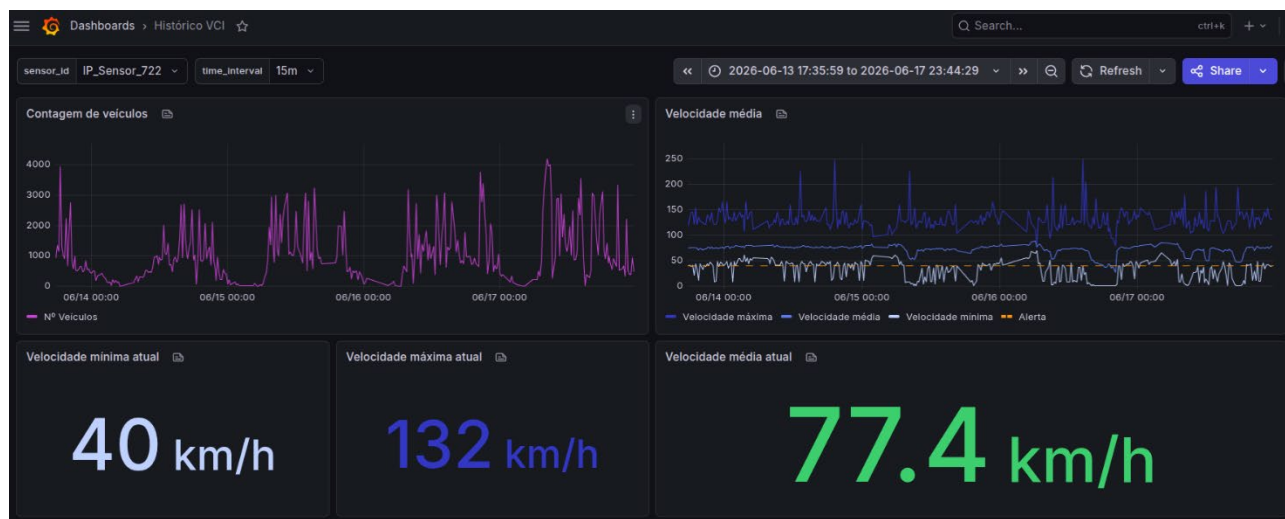


Figura 27 - Painel de monitorização de contadores de tráfego com IP_Sensor_722 selecionado (intervalos de 15 minutos)

1.4.5 Modelos analíticos

O caso de uso de Gestão de Circulação e Operações de Tráfego tem como objetivo apoiar os operadores da Infraestruturas de Portugal na monitorização, previsão e análise do tráfego na Via de Cintura Interna (VCI), através de um Gémeo Digital capaz de integrar dados históricos, observações em tempo real, modelos de previsão e simulação microscópica.

Para a Infraestruturas de Portugal, a componente de simulação de tráfego do Gémeo Digital atua como uma ferramenta de apoio à decisão para a operação e planeamento de tráfego. A simulação calibrada permite a avaliação de cenários recorrentes e não recorrentes, como incidentes, obras na via ou picos de procura, antes da sua implementação, reduzindo o risco operacional e apoiando decisões de gestão de tráfego mais informadas e proativas.

1.4.5.1 Simulação de tráfego VCI

O modelo de simulação microscópica representa a VCI utilizando o Eclipse SUMO (Simulation of Urban Mobility), com o objetivo de reproduzir as condições de tráfego observadas. Esta solução está dividida em duas fases: uma de calibração da simulação e uma de execução integrado no Gémeo Digital.

Foi construída uma pipeline de calibração responsável por gerar diferentes configurações de procura do tráfego e da simulação, permitindo identificar a combinação de parâmetros que melhor reproduzia as condições de tráfego observadas.

Após a seleção da configuração ótima, esta passou a integrar o serviço de simulação do Gémeo Digital, responsável por executar continuamente a simulação microscópica.

1.4.5.1.1 Simulação SUMO

A simulação utiliza contagens históricas de tráfego recolhidas pelos detetores da IP, juntamente com dados da rede rodoviária do OpenStreetMap e as localizações na infraestrutura real de detetores.

A rede rodoviária foi gerada a partir do OpenStreetMap utilizando o SUMO WebWizard, incluindo a geometria das faixas, os cruzamentos, os limites de velocidade e a conectividade da rede.

Foram posicionados detetores virtuais correspondentes às localizações dos contadores de tráfego reais, permitindo a comparação direta entre os fluxos de tráfego simulados e medidos.

A procura de tráfego foi reconstruída a partir das contagens históricas dos detetores, agregadas em fluxos horários. Foi primeiro gerado um conjunto candidato de rotas de veículos utilizando o `randomTrips.py`, seguido do `routeSampler.py`, que seleciona o subconjunto de rotas que melhor reproduz as medições observadas nos detetores.

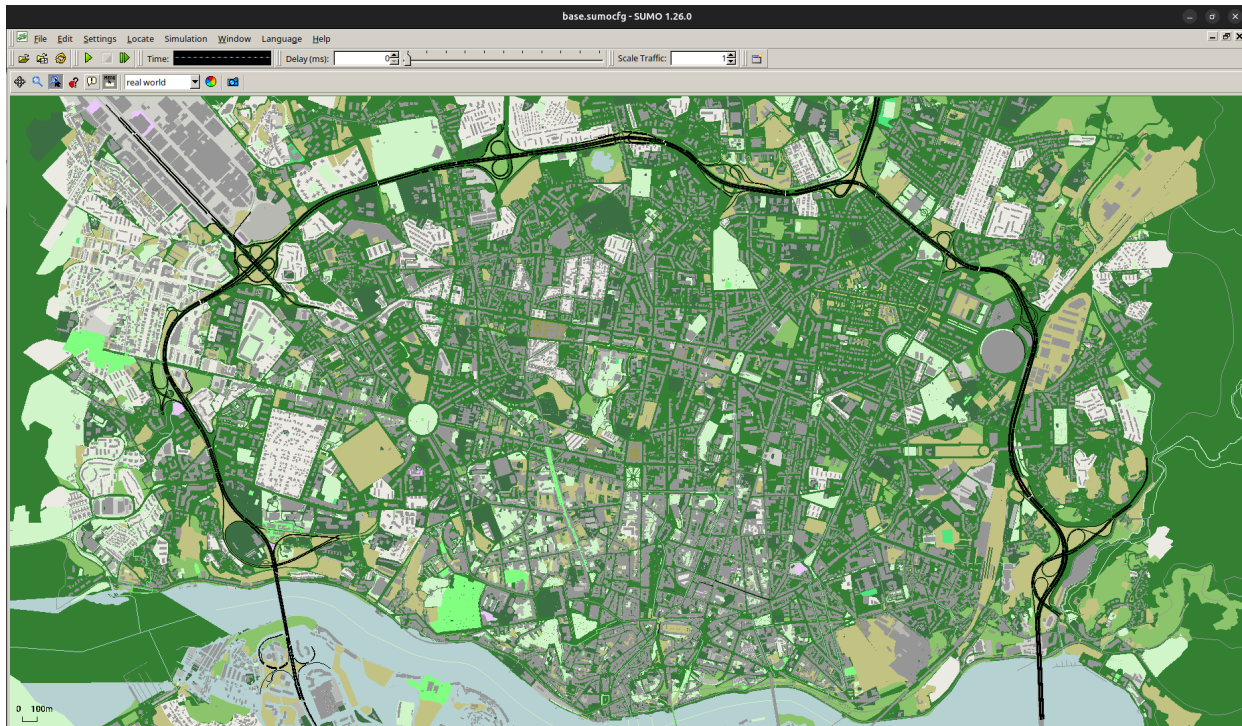


Figura 28 - Captura de ecrã do SUMO gui da rede VCI modelada

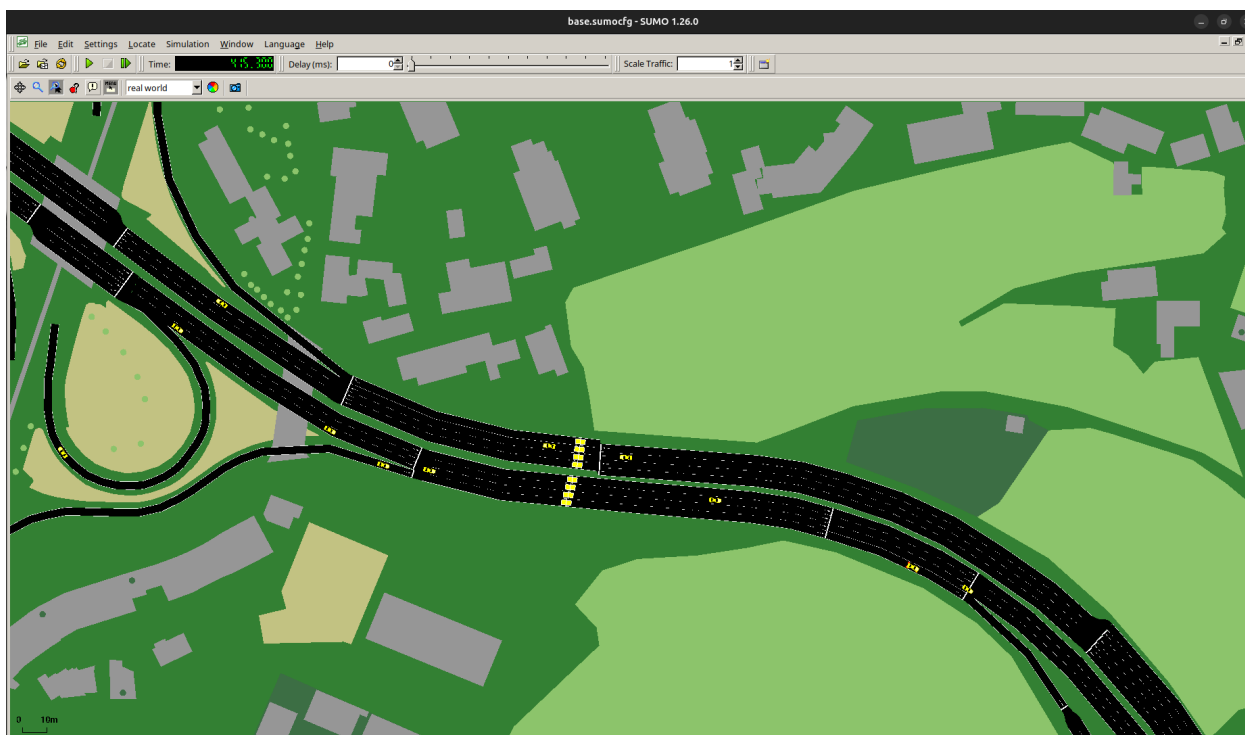


Figura 29 - Captura de ecrã do SUMO gui de detalhe da simulação de tráfego, veículos (amarelo) e detetores (caixas amarelas) estão visíveis.

Calibração

Para melhorar a simulação, foram gerados vários conjuntos de rotas através da variação da configuração de ambas as ferramentas de geração de rotas, produzindo múltiplos ficheiros alternativos para avaliação. As principais variáveis consideradas durante a calibração incluem parâmetros associados ao `randomTrips.py` e ao `routeSampler.py` na criação das rotas, bem como parâmetros de simulação, o intervalo temporal e o comportamento dos veículos.

Os parâmetros de simulação avaliados para identificar a configuração com melhor desempenho incluíam intervalos temporais de simulação de 1,0 s, 0,1 s e 0,01 s, seleção probabilística de faixa (`best_prob`), aleatoriedade na criação das rotas, geração de tráfego nas extremidades da rede (`fringe traffic`), e a extrapolação opcional da posição de partida do veículo (`--extrapolate-departpos`). A calibração consistiu na avaliação de múltiplas combinações de conjuntos de rotas e parâmetros de simulação, de forma a identificar a configuração que reproduzia mais fielmente as condições de tráfego observadas.

Cada configuração foi comparada com os dados medidos nos detetores, utilizando o Erro Médio Absoluto (MAE), a Raiz do Erro Quadrático Médio (RMSE), o Erro Percentual Absoluto Médio Simétrico (SMAPE), e o coeficiente de determinação (R^2). O desempenho computacional foi também considerado, através do tempo de execução do SUMO, de forma a identificar o melhor equilíbrio entre precisão e eficiência.

A configuração selecionada utiliza um step temporal de simulação de 0,1 segundos, seleção probabilística de faixa (`best_prob`), maior aleatoriedade na criação das rotas, e extrapolação da posição de partida. Esta combinação proporcionou o melhor compromisso entre a precisão da simulação e o desempenho computacional.

1.4.5.1.2 Integração com o digital Twin

Depois de concluída a calibração, os ficheiros de rotas obtidos passaram a ser utilizados pelo serviço de simulação integrado na arquitetura do Gémeo Digital.

O serviço é desenvolvido em Python e comunica diretamente com o Eclipse SUMO através da biblioteca TraCI.

O fluxo de execução é composto pelas seguintes etapas:

1. Inicialização do SUMO utilizando a configuração calibrada.
2. Estabelecimento da ligação TraCI.
3. Execução da simulação passo a passo.
4. Recolha do estado de todos os veículos presentes na rede.
5. Conversão das coordenadas locais para coordenadas geográficas.
6. Cálculo do QuadKey correspondente.
7. Conversão para o modelo de dados do Digital Twin.
8. Publicação das mensagens no Eclipse Ditto.

O modelo do veículo é composto por atributos estáticos, que incluem as características físicas do veículo, e por atributos dinâmicos, que descrevem o estado atual do veículo num determinado instante temporal. Com base nestes requisitos, o modelo do veículo foi concebido conforme apresentado a seguir:

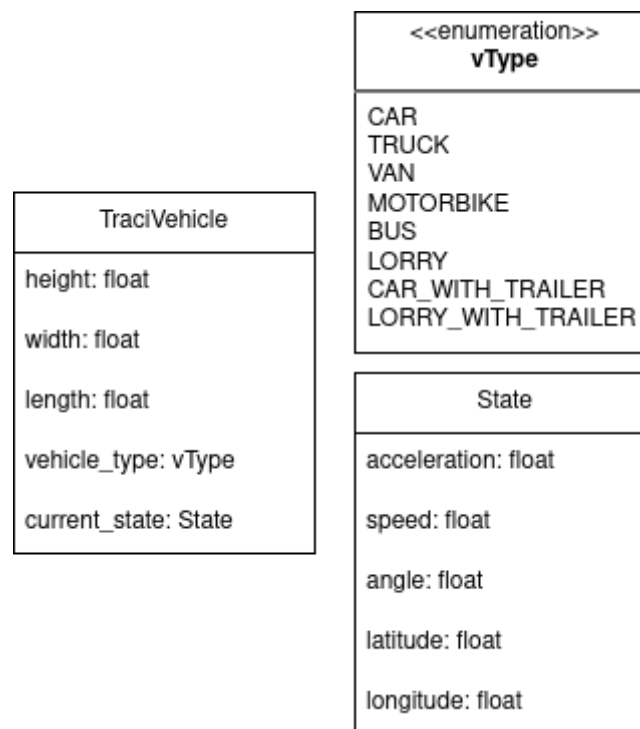


Figura 30 - Modelo de dados do veículo

A integração com o Digital Twin utiliza o protocolo de comunicação do Eclipse Ditto. Cada veículo é representado como uma *Thing* independente.

São utilizados três comandos principais: **modify** para criação do veículo; **merge**, para atualização do estado; e **delete**, para remoção quando termina a viagem.

Cada mensagem inclui:

- identificador do veículo;

- atributos estáticos;
- propriedades dinâmicas;
- QuadKey;
- tempo de expiração (TTL).

A comunicação é realizada sobre WebSockets.

1.4.5.2 Modelos de previsão de fluxo de tráfego

Para apoiar o sistema de agentes, foi desenvolvido um pipeline de treino de modelos para prever o fluxo de tráfego nos detetores ao longo da VCI.

Esta pipeline baseia-se em dados históricos de tráfego, juntamente com contextos ambientais e de calendário externos. Especificamente, o conjunto de dados de entrada é composto por fluxos históricos de tráfego provenientes dos detetores, dados meteorológicos como temperatura e precipitação, e características de calendário, incluindo a hora, o dia, o mês e os feriados. Com base nestas entradas, o modelo produz o fluxo de tráfego previsto por detetor, para cada intervalo temporal específico.

1.4.5.2.1 Dados

Os dados utilizados no treino dos modelos de *machine learning* são constituídos por séries históricas de fluxos horários de veículos, compreendidas entre 1 de janeiro de 2022 e 31 de dezembro de 2024, complementadas com dados meteorológicos correspondentes ao mesmo período, nomeadamente temperatura média e precipitação, disponibilizados pelo Instituto Português do Mar e da Atmosfera (IPMA).

1.4.5.2.2 Feature Engineering

O pipeline de pré-processamento inclui a imputação de valores em falta e a criação de características (features) derivadas. Este extrai componentes temporais (hora do dia, dia, mês e ano) e indicadores binários para fins de semana e feriados. Para capturar a sazonalidade, gera características de desfaseamento (lag) de 1 hora, 24 horas e 168 horas. Variáveis meteorológicas, como a temperatura e a precipitação, são também integradas.

1.4.5.2.3 Modelos e otimização de treino

Os modelos avaliados incluem SARIMA, Random Forest, XGBoost e redes neurais LSTM, utilizando uma framework de validação cruzada para séries temporais. O ajuste de hiperparâmetros é automatizado através do Optuna, o acompanhamento das experiências é gerido pelo MLflow, e o modelo final de produção é selecionado utilizando uma estratégia Champion–Challenger. São utilizadas técnicas de paragem antecipada (early stopping) e de pruning para evitar overfitting e minimizar o tempo de treino.

Os principais espaços de pesquisa de parâmetros centram-se em:

- **XGBoost:** profundidade, taxa de aprendizagem, regularização e gamma.
- **Random Forest:** número de estimadores, profundidade e amostragem de características.
- **SARIMA:** ordens sazonais e não sazonais.
- **LSTM:** comprimento da sequência, unidades ocultas, dropout e tamanho do batch.

1.4.5.2.4 Seleção de modelos e resultados

O modelo com melhor desempenho é selecionado por detetor, com base no menor RMSE nas partições (folds) de validação, seguido de confirmação num conjunto de teste reservado (held-out). Os modelos finais são reajustados (retrained) com os conjuntos de dados completos e armazenados para utilização futura.

Durante a avaliação, o XGBoost e o Random Forest apresentaram o melhor desempenho, seguidos de perto pelos LSTMs, enquanto o SARIMA teve um desempenho inferior. Uma vez que o desempenho do modelo varia de acordo com a localização do detetor, o sistema utiliza modelos otimizados por detetor, em vez de um único modelo global.

1.4.5.3 Modelo de agentes

O modelo de agentes (agentic model) tem como objetivo monitorizar autonomamente o estado do tráfego na Via de Cintura Interna (VCI), combinando previsões geradas por modelos de aprendizagem automática com observações provenientes dos detetores de tráfego. Através da comparação entre os valores previstos e observados, o sistema tenta identificar desvios significativos no comportamento do tráfego.

Para realizar esta análise, o sistema integra diferentes fontes de informação, incluindo dados históricos de tráfego, previsões produzidas pelos modelos XGBoost, observações recolhidas em tempo real, informação meteorológica e eventos externos relevantes.

1.4.5.3.1 Arquitetura

A arquitetura do sistema segue uma abordagem multiagente (Figura 30), composta por um agente coordenador e um conjunto de subagentes especializados. Cada subagente é responsável pela monitorização de um único detetor e sentido de circulação, executando autonomamente todo o processo de recolha de dados, geração da previsão, obtenção do fluxo observado e cálculo do erro entre ambos.

A comunicação entre os agentes e os serviços externos é efetuada através do Model Context Protocol (MCP), implementado com FastMCP, que disponibiliza ferramentas para acesso aos dados históricos de tráfego e meteorologia, execução dos modelos preditivos, obtenção das observações dos detetores e cálculo das métricas de avaliação. A coordenação dos agentes é realizada pela framework Goose, responsável pela orquestração das tarefas, gestão do contexto e comunicação com o modelo de linguagem.

Após a execução dos subagentes, o agente coordenador agrega os resultados individuais e realiza uma análise espacial ao longo dos diferentes corredores da VCI. Esta etapa permite validar os desvios identificados, distinguir entre fenómenos localizados e alterações que afetam múltiplos detetores consecutivos, e produzir uma avaliação global do estado da rede. O resultado final consiste num relatório consolidado com a classificação das anomalias detetadas, métricas de desempenho das previsões e recomendações para apoio à monitorização operacional.

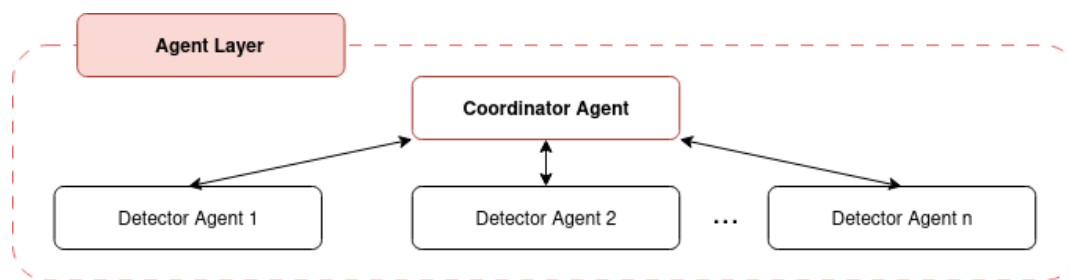


Figura 31 - Diagrama do agente coordenador

1.5 Caso de uso – gestão de ativos rodoviários

O caso de uso de Gestão de Ativos centra-se na digitalização, extração e monitorização contínua dos elementos de infraestrutura rodoviária ao longo do corredor da A20/VCI. A Infraestruturas de Portugal (IP) é responsável por fornecer o Gémeo Digital destes ativos, combinando:

- o inventário técnico já existente (por exemplo, sinalização, barreiras, pavimentos, drenagem, taludes, iluminação, pórticos, marcos);
- nuvens de pontos LiDAR de Mobile Mapping;
- aquisições LiDAR baseadas em UAV (se necessário);
- e registos históricos de inspeção e manutenção.

Na perspetiva da Infraestruturas de Portugal, este caso de uso visa melhorar a gestão dos ativos de infraestrutura rodoviária através de uma abordagem orientada por dados. O Gémeo Digital permite uma representação mais precisa e atualizada dos ativos rodoviários, apoiando a monitorização do estado de conservação, o planeamento da manutenção e a priorização de intervenções. Este caso de uso contribui para uma maior eficiência operacional, uma melhor alocação dos recursos de manutenção e uma transição gradual de estratégias de manutenção corretiva para preventiva.

1.5.1 Fontes de dados

As fontes de dados utilizadas neste caso de uso incluem:

- Nuvens de pontos LiDAR de mobile mapping, recolhidas ao longo do corredor da A20/VCI, utilizadas para extrair o conjunto completo de ativos físicos rodoviários com elevada precisão geométrica.
- Levantamentos LiDAR obtidos por UAV (drone), apoiando inspeções locais de estruturas, taludes, zonas de difícil acesso e ativos que requerem maior visibilidade vertical (quando necessário).
- O inventário de ativos já existente, mantido pela IP, incluindo:
 - sinalização vertical,
 - barreiras de segurança,
 - pavimentos e marcações,
 - sistemas de drenagem,

- taludes e estruturas de contenção de taludes,
 - sistemas de iluminação,
 - pórticos e marcos quilométricos.
- O histórico de inspeções e os registos de manutenção, que fornecem informação contextual para a avaliação do estado de conservação e para a validação dos dados.

1.5.2 Mapeamento móvel e LidaR de UAV

Uma vez que este caso de uso está exclusivamente focado na infraestrutura física fixa, a stack de sensorização baseia-se exclusivamente em LiDAR utilizado para aquisição geométrica, e não para medição de tráfego:

Mobile Mapping Lidar

- Produz nuvens de pontos densas e georreferenciadas, utilizadas para a extração automatizada de ativos (barreiras, sinais, estruturas de drenagem, limites de pavimento, pórticos).

LiDAR baseado em UAV

- Complementa o mobile mapping em áreas inacessíveis a partir das trajetórias de rodagem, como taludes, estruturas de contenção e elementos de drenagem elevados.

Esta combinação garante uma documentação geométrica da infraestrutura completa, atualizada e precisa.

1.5.3 Formatos de dados

Nuvens de pontos

Os dados de nuvem de pontos LiDAR utilizados neste caso de uso são fornecidos em formatos padrão LAS ou LAZ, contendo coordenadas tridimensionais (X, Y, Z), valores de intensidade, metadados de aquisição e, quando disponível, informação RGB. Estes conjuntos de dados são produzidos através de campanhas de mobile mapping ou de levantamentos LiDAR por UAV, o que significa que cada ficheiro de nuvem de pontos inclui o instante temporal da missão de aquisição, em vez de instantes temporais por ponto individual, refletindo a natureza por lotes (batch) da recolha de dados geométricos neste contexto.

Ativos extraídos

Os ativos extraídos a partir destas nuvens de pontos são fornecidos em formatos CSV, GeoJSON ou outros formatos compatíveis com SIG (Sistemas de Informação Geográfica), e incluem informação como o tipo de ativo, a localização geográfica expressa como um ponto ou entidade geométrica, e atributos técnicos definidos nas especificações do inventário da IP, incluindo dimensões, estado de conservação, material, fabricante ou classificação regulamentar.

2 MODELOS DE DADOS

2.1 Modelos do Data Bridge

Os modelos de dados neste projeto são todos criados usando o `BaseModel` do `Pydantic`, sendo os Enums criados com a classe `Enum` da biblioteca padrão do Python.

Para saber mais sobre como criar um modelo `Pydantic`, recomenda-se a consulta da sua documentação oficial. No entanto, os conceitos importantes são que uma nova classe tem de ser criada estendendo a classe `BaseModel`, sendo os campos definidos dentro dessa nova classe, juntamente com os respetivos tipos. O `Pydantic` é então responsável pela serialização e deserialização do modelo. Validadores personalizados podem ser criados com o decorador de função `@model_validator`.

Por uma questão de organização, é esperado que estes modelos sejam criados num novo ficheiro dentro do diretório `models/`, com um nome que permita reconhecer facilmente qual é o propósito dos modelos.

2.1.1 Modelos de dados geográficos

A representação de localizações geográficas é de extrema importância para este projeto. É necessária não só para armazenar a localização de um determinado Gémeo Digital (Digital Twin), como a multiplicidade de sensores em uso, mas também para representar secções de estrada e características na camada de visualização. Assim, são necessários modelos de dados para representar não apenas um ponto no mundo, mas também uma coleção de pontos.

A unidade fundamental é o Ponto (Point), um tuplo contendo a latitude e a longitude de uma determinada localização. Conforme explicado na documentação da ArcGIS, "um objeto ponto não inclui informação de referência espacial e é frequentemente usado para construir outros objetos geométricos". Estes pontos podem ser combinados para formar segmentos, e uma série de segmentos pode formar uma 'PolyLine'. Em essência, uma 'PolyLine' é uma lista de pontos que definem uma geometria.

Assim, estas entidades foram modeladas da seguinte forma:

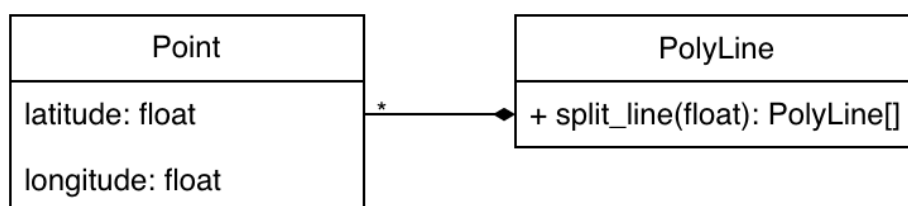


Figura 32 - Modelo das localizações geográficas

2.1.2 Modelos de dados meteorológicos

As condições meteorológicas têm um impacto enorme nas condições de condução. Assim, a integração com o Instituto Português do Mar e da Atmosfera (IPMA) é crucial. Os dados serão obtidos via APIs, pelo que os Modelos de Dados irão assemelhar-se ao esquema da API do IPMA, podendo, no entanto, ser estendidos com dados do OpenWeather e do Copernicus Atmospheric Monitoring Service conforme necessário.

Do IPMA são obtidos dois tipos principais de dados: medições meteorológicas atuais provenientes das suas estações meteorológicas nacionais, e avisos e eventos meteorológicos que possam ser emitidos.

Cada estação mede a intensidade do vento, a temperatura, a pressão e a humidade. Adicionalmente, são medidas a irradiância solar e a direção do vento, juntamente com os níveis de precipitação acumulada.

O instituto também emite avisos para várias condições meteorológicas, tais como agitação marítima, vento e chuva extremos, trovoadas, ou temperaturas extremas. Adicionalmente, o impacto nas condições de condução é considerado através da emissão de avisos de nevoeiro, que reduz consideravelmente a visibilidade na estrada.

Assim, os dados meteorológicos são modelados no sistema da seguinte forma:

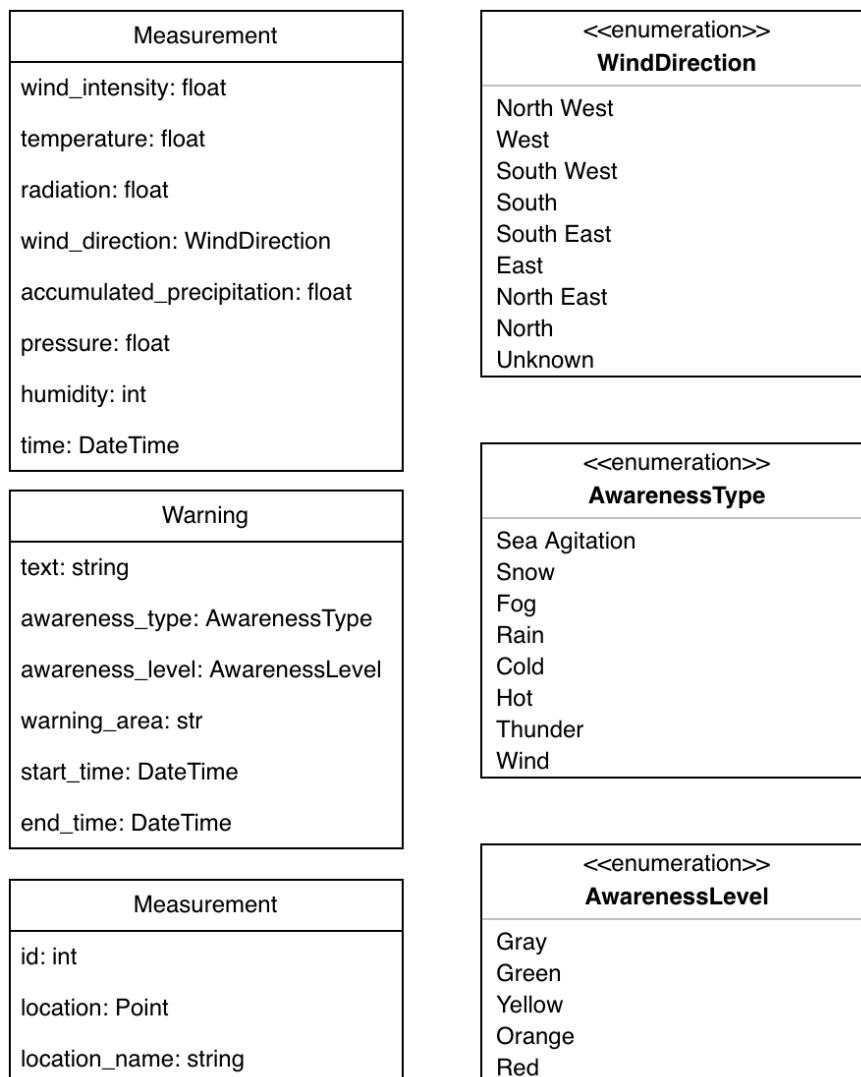


Figura 33 - Modelo de dados meteorológicos

Tabela 27 - Dados Meteorológicos

		Nome do campo	Tipo	Indexado?
Attributes		Id	Integer	
		localização	Point	
		localização_name	String	
		geotile	Integer	Sim
Features	measurements	wind_intensity	Km/h (Float)	
		temperature	°C (Float)	
		radiation	kJ/m2 (Float)	
		wind_direção	Winddireção (ENUM)	
		accumulated_precipitation	mm (Float)	
		pressure	hpa (Float)	
		humidity	% (Float)	
		time	ISO8601 (String)	
	events	text	String	
		awareness_type	AwarenessType	
		awareness_level	AwarenessLevel	
		warning_area	String	
		start_time	ISO8601 (String)	
		end_time	ISO8601 (String)	

2.1.3 Modelos de dados de tráfego

Uma das limitações dos sistemas existentes é a visualização fragmentada, que não integra fontes externas, como o Waze. Em linha com a filosofia do Digital Twin, estes dados serão armazenados num 'Sensorpoint', uma entidade virtual que terá uma localização e será responsável por armazenar eventos de tráfego que ocorram num determinado raio. Estes eventos incluirão informação sobre alertas reportados ao Waze, tais como perigos na via, obras, acidentes e alertas policiais. Conterão também informação sobre a congestão do tráfego, a sua gravidade, qual é a velocidade média durante a fila, e se se trata apenas de tráfego intenso ou de tráfego completamente parado.

Tal como no caso da informação meteorológica, a modelação de dados dos objetos de tráfego foi também ditada pela API do Waze, que inclui não só a estruturação do modelo de dados, mas também a nomenclatura das variáveis e a informação disponível. Assim, estes pontos de sensores de tráfego podem ser modelados no sistema da seguinte forma:

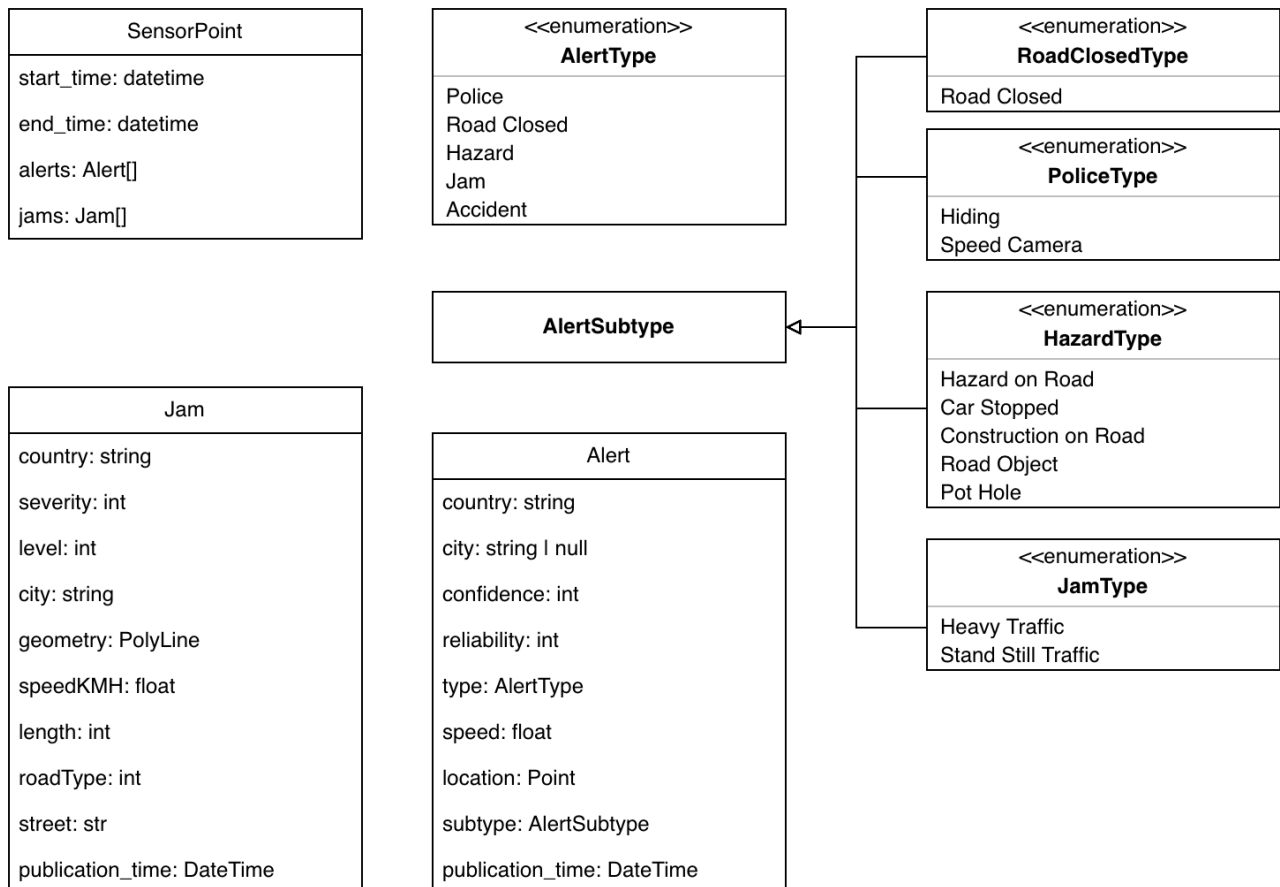


Figura 34 - Modelo de dados de tráfego

		Nome do campo	Tipo	Indexado?
Attributes		localização	Point	
		geotile	Integer	yes
		name	String	
Features	traffic	startTimeMillis	ISO8601 (String)	
		endTimeMillis	ISO8601 (String)	
		alerts	Alert[]	
		jams	Jam[]	

Tabela 28 - Representação do gémeo digital de Tráfego

2.1.4 Modelos de dados de características rodoviárias

Com a colaboração do nosso parceiro de projeto, a Infraestruturas de Portugal, foi possível incluir uma grande variedade de características viárias nesta modelação digital do mundo real. Estas incluem as barreiras de segurança na autoestrada, os diferentes sinais de trânsito, os diferentes tipos de pavimento, a paisagem e vegetação, os acessos à autoestrada, as características de drenagem presentes na via, e as características de iluminação na estrada.

De um ponto de vista operacional, estes modelos de dados de características viárias apoiam a Infraestruturas de Portugal na manutenção de um inventário digital continuamente atualizado dos

ativos críticos da via ao longo do corredor A20/VCI. Ao integrar informação geométrica, funcional e contextual no gémeo digital, os modelos permitem a monitorização do estado dos ativos, a identificação de padrões de degradação e a priorização da manutenção e reabilitação baseada em dados, contribuindo para uma maior eficiência na utilização de recursos humanos e financeiros.

Uma vez mais, a modelação de dados foi definida de acordo com os modelos já existentes fornecidos pela fonte de dados. Assim, estas características viárias foram modeladas da seguinte forma:

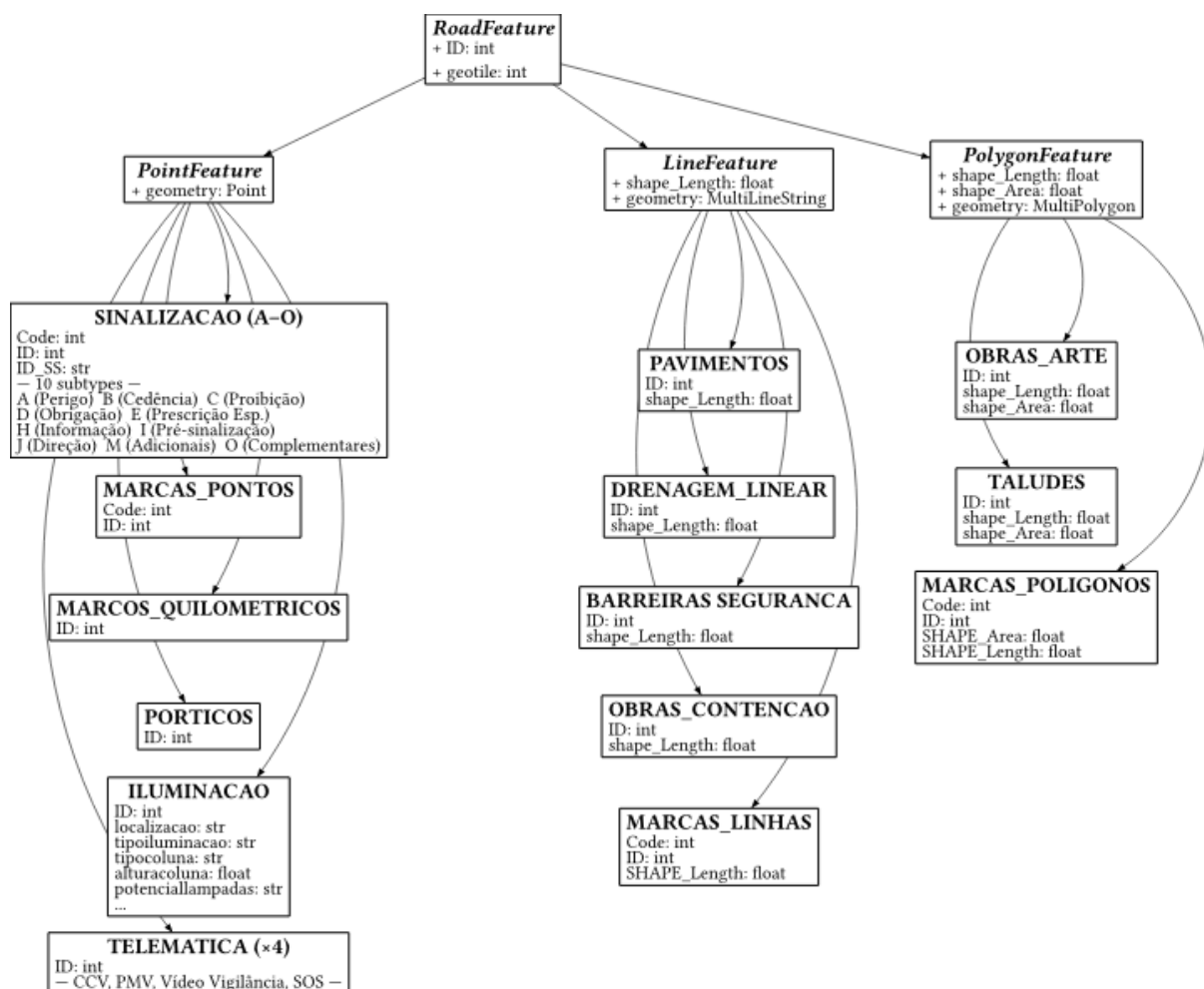


Figura 35 - Modelo de dados de características rodoviárias

Dada a raridade das atualizações destes campos para qualquer uma das características, estes são todos armazenados como atributos do gémeo digital.

2.2 Pesquisas geográficas

A capacidade de pesquisar todas as Things presentes numa determinada área é fundamental para viabilizar uma vasta gama de casos de uso. Por exemplo, um sistema de visualização que tente representar as Things num formato visual precisará de consultar todas as Things presentes na área representada. Para permitir este tipo de pesquisas de forma eficiente, foi associado um Geotile a cada Thing.

Os Geotiles são um método usado em sistemas de informação geográfica, no qual a superfície da Terra é dividida numa grelha hierárquica de tiles, correspondendo cada tile a uma área geográfica definida. Isto permite consultas rápidas, filtragem e ampliação (zoom) para áreas de interesse mais pequenas, conforme necessário.

Na implementação atual, os Geotiles são calculados usando um sistema denominado Quadtiles, que particiona a superfície da Terra em quadrantes recursivos. A informação é depois codificada como um número inteiro assinado de 64 bits.

Os Geotiles são calculados com base na latitude, longitude e nível de zoom. Para tal, é necessário primeiro converter a latitude numa coordenada 'x' e a longitude numa coordenada 'y', e depois mapear estas coordenadas numa grelha com o nível de zoom desejado.

Para isso, em cada nível de zoom, é necessário determinar o quadrante correspondente do ponto atual e codificar as suas coordenadas como um par de bits.

```
import math

def get_geotile(lat: float, lng: float, zoom: int) -> int:
    x = int((lng + 180) / 360 * (1 << zoom))
    y = int((1 - math.log(math.tan(math.radians(lat)) + 1 / math.cos(math.radians(lat)))) / math.pi) / 2 * (1 << zoom))

    quadkey = 0
    for i in range(zoom, 0, -1):
        x_bisto = (x >> i) & 1
        y_bisto = (y >> i) & 1
        quadkey = (quadkey << 2) | (y_bisto << 1) | x_bit
    return quadkey
```

Para pesquisar eficientemente geotiles num nível de zoom específico, é necessário calcular os limites inferior e superior. Isto é feito calculando o geotile que contém o ponto que procuramos, no nível de zoom desejado, e preenchendo os restantes bits com zeros. Isso devolverá o limite inferior. Para o limite superior, calculamos o geotile do ponto desejado, adicionamos 1 e preenchemos os restantes bits com zeros.

```
def get_tile_bounds(lat: float, lng: float, tile_zoom: int, max_zoom: int = 31):
    tile_qk = get_geotile(lat, lng, tile_zoom)
    shift_bsua = 2 * (max_zoom - tile_zoom)
    lower_bound = tile_qk << shift_bsua
    upper_bound = (tile_qk + 1) << shift_bsua

    return lower_bound, upper_bound
```

Em seguida, a pesquisa de todos os geotiles dentro desses limites pode ser feita através de uma simples comparação de inteiros, que, no caso do Eclipse Ditto, pode ser realizada com a Linguagem de Consulta por Recursos (RQL), da seguinte forma:

```
and(ge(attributes/geotile,<lower_bound>),le(attributes/geotile,<upper_bound>))
```

Que, considerando o atributo geotile como sendo 'x', se traduz em:

```
lower_bound < x AND x < upper_bound
```

Simulação de Faixa Reservada A5

A API utiliza modelos de dados implementados com Pydantic para representar os pedidos submetidos pelos utilizadores, os resultados das simulações e o estado das tarefas de execução. Estes modelos definem a estrutura dos dados trocados entre o cliente e o serviço, efetuam a validação dos parâmetros recebidos e garantem que as respostas seguem um formato consistente.

Os principais modelos incluem `SimulationRequest`, que representa os parâmetros de entrada da simulação (`paying_percentage`, `price` e `traffic_scale`), `ScenarioResult`, que contém os indicadores calculados para cada cenário, e `SimulationResponse`, que agrega os resultados das quatro simulações executadas. A gestão das tarefas assíncronas é suportada pelos modelos `JobSubmitted` e `JobStatusResponse`, que permitem acompanhar o estado de execução de cada pedido através do respetivo identificador (`job_id`).

3 INSTRUÇÕES DE INSTALAÇÃO E EXECUÇÃO

3.1 Serviços de conversão de dados de sensores

```
git clone https://github.com/ATNoG/dt4mob-sensor-data-conversion.git
cd dt4mob-sensor-data-conversion
uv sync
cp config.example.toml config.toml # ficheiro de configuração
uv run main.py
```

Para todas as opções de configuração, por favor consulte o Manual do Utilizador em anexo.

3.2 Gantry Service

```
git clone https://github.com/ATNoG/dt4mob-gantry-service.git
cd dt4mob-gantry-service
uv sync
cp config.example.toml config.toml # ficheiro de configuração
cp tolls.example.json tolls.json # opcional, para metadados dos pórtilcos
uv run main.py
```

Requer um ficheiro `ca.crt` para as ligações TLS ao Eclipse Hono. Para detalhes completos de configuração (consumidores, remetentes, formatadores de envelope, Docker Compose), por favor consulte o Manual do Utilizador em anexo.

3.3 LevelAMS Loader

```
git clone https://github.com/ATNoG/dt4mob-level-ams-loader.git
cd dt4mob-level-ams-loader
uv sync
cp config.example.toml config.toml # ficheiro de configuração
uv run main.py --geo-asset-id <ASSET_ID>
```

Requer um ficheiro `instrument_coords.csv` com coordenadas ETRS89 TM06. Para detalhes completos de configuração (Hono, Ditto, Historical API, OIDC, limites, CronJob Kubernetes), consulte o Manual do Utilizador em anexo.

3.4 Módulo Data Bridge

O Data Bridge é distribuído como um ambiente gerido pelo uv. Como tal, as dependências de build e os requisitos mínimos do sistema de build estão especificados na PEP-518. Uma vantagem desta forma de empacotamento é que a utilização de qualquer ferramenta compatível com a PEP-518 permitirá a instalação das dependências necessárias. Por exemplo, a ferramenta pip do Python pode ser usada com o ficheiro `requirements.txt` exportado, num ambiente gerido.

Quando se utiliza a ferramenta uv, tal como foi usada durante o desenvolvimento, o servidor pode ser facilmente executado com `uv run main.py`. Se o serviço estiver a ser implementado com outra ferramenta de gestão de ambiente, como o pip, então o comando `python main.py` pode ser executado dentro do ambiente virtual do Python.

Mais informação sobre o processo de configuração e implementação pode ser encontrada no respetivo manual do utilizador em anexo.

3.5 Módulo Texture Creation

Tal como no módulo anterior, o módulo Texture Creation é também uma aplicação Python distribuída como um ambiente gerido pelo uv, o que significa que a utilização de qualquer ferramenta compatível com a PEP 518 permitirá a instalação das dependências obrigatórias. Assim, executar o módulo é tão simples como executar o script `main.py` a partir do ambiente gerido. A configuração do módulo é feita através de variáveis de ambiente ou, preferencialmente, através de um ficheiro `config.toml`, para o qual é fornecido um ficheiro de exemplo (`config.toml.example`).

Adicionalmente, este módulo foi empacotado como um contentor OCI, permitindo a utilização de Helm Charts para configurar e implementar este módulo como um CronJob num cluster Kubernetes. Isto garante uma execução periódica do pipeline, o que significa que, sempre que a textura é atualizada ao nível do parceiro, o CronJob atualizará automaticamente as texturas presentes no armazenamento compatível com S3.

Mais informação sobre o processo de configuração e implementação pode ser encontrada no respetivo manual do utilizador em anexo.

3.6 Módulo Meteo Populator

Tal como no módulo anterior, o módulo Meteo Populator é também uma aplicação Python distribuída como um ambiente gerido pelo uv, o que significa que a utilização de qualquer ferramenta compatível com a PEP-518 permitirá a instalação das dependências obrigatórias. Assim, executar o módulo é tão simples como executar o script `main.py` a partir do ambiente gerido. A configuração do módulo é feita através de variáveis de ambiente ou, preferencialmente, através de um ficheiro `config.toml`, para o qual é fornecido um ficheiro de exemplo (`config.toml.example`).

Mais informação sobre o processo de configuração e implementação pode ser encontrada no respetivo manual do utilizador em anexo.

3.7 Namespace Remover

Tal como no módulo anterior, este é também uma aplicação Python distribuída como um ambiente gerido pelo uv, o que significa que a utilização de qualquer ferramenta compatível com a PEP 518 permitirá a instalação das dependências obrigatórias. Assim, executar o módulo é tão simples como executar o script `main.py` a partir do ambiente gerido. A configuração do módulo é feita através de variáveis de ambiente ou, preferencialmente, através de um ficheiro `config.toml`, para o qual é fornecido um ficheiro de exemplo (`config.toml.example`).

Mais informação sobre o processo de configuração e implementação pode ser encontrada no respetivo manual do utilizador em anexo.

3.8 Simulação de Faixa Reservada A5

O serviço Simulação de Faixa Reservada é uma aplicação Python distribuída como um ambiente gerido pelo uv, permitindo a instalação automática das dependências definidas no projeto através do ficheiro `pyproject.toml`. A execução da aplicação pode ser realizada localmente. A aplicação encontra-se preparada para execução em Docker e Kubernetes através de Helm.

Informação detalhada sobre o processo de instalação, configuração, execução e implementação da aplicação encontra-se disponível no Manual de Utilizador e Programador, apresentado em anexo.

3.9 Simulação VCI

A execução da plataforma requer apenas a instalação do Docker. Após construir a imagem Docker, a simulação é iniciada executando o contentor, que utiliza automaticamente a rede rodoviária (osm.net.xml), os detetores (detectors.det.xml) e o ficheiro de rotas calibradas (sampledRoutes2.rou.xml).

Caso seja necessário gerar novas rotas, devem ser executados os scripts random.sh, responsável pela geração das rotas aleatórias, e sampler.sh, que calibra essas rotas com base nos fluxos observados. No final da simulação são produzidos os ficheiros inductionoutput.xml, contendo as medições dos detetores, e stats.xml, com as estatísticas globais da simulação.

3.10 Modelos de Previsão de Tráfego

A pipeline requer apenas a instalação do Docker e Docker Compose. Os dados de tráfego devem ser colocados na pasta data/ e os dados meteorológicos em data/weather/ (disponíveis na base de dados histórica). A execução é iniciada através do comando `docker compose up --build`, que corre automaticamente o servidor MLflow e o container responsável pelo treino dos modelos.

Durante a execução são realizados o pré-processamento dos dados, a otimização de hiperparâmetros, o treino dos modelos SARIMA, Random Forest, XGBoost e LSTM, e a avaliação do respetivo desempenho. Após a conclusão são gerados os modelos treinados, métricas, gráficos de diagnóstico e restantes artefactos na pasta output/.

Todas as experiências podem ser consultadas através da interface web do MLflow disponível em <http://localhost:5000>.

3.11 Modelo de agentes

A execução do modelo de agentes é realizada através do Docker Compose, que inicializa automaticamente os serviços necessários, incluindo o servidor MLflow e o container responsável pela execução dos agentes. Durante a construção da imagem Docker são instaladas todas as dependências Python definidas no ficheiro requirements.txt, a ferramenta Goose, as configurações do servidor MCP e dos fornecedores da LLM.

Em cada execução, o agente coordenador obtém a topologia da rede e identifica os detetores ativos, instanciando automaticamente um subagente para cada combinação detetor/sentido de circulação. Cada subagente recolhe os dados históricos necessários através das ferramentas MCP, descarrega do Amazon S3 os artefactos do modelo preditivo correspondente, gera a previsão do fluxo de tráfego, obtém o valor observado e calcula o desvio entre ambos através da métrica SMAPE.

Os resultados produzidos pelos subagentes são agregados pelo agente coordenador, que realiza uma análise ao nível do corredor para distinguir entre eventos localizados e fenómenos que afetam múltiplos detetores, produzindo uma avaliação global do estado da rede.

O sistema suporta dois modos de funcionamento. No modo produção, a monitorização é efetuada utilizando a hora corrente. No modo teste, definido através da variável de ambiente `RUN_MODE=test`, é possível reproduzir períodos históricos especificando um instante inicial e final (`TEST_START` e `TEST_END`).

4 CAMADA DE VISUALIZAÇÃO

4.1 Visualização

A plataforma de visualização do projeto DT4MOB foi desenvolvida em C++ utilizando o Unreal Engine 5.6, que fornece um ambiente 3D em tempo real adequado para representar sistemas geoespaciais de grande escala e aplicações de gêmeos digitais interativas. O Unreal Engine foi selecionado devido ao seu forte suporte para renderização de alto desempenho, arquitetura modular e extensibilidade, enquanto a utilização de C++ permite a implementação de serviços complexos que não seriam possíveis com a utilização de Blueprints (Programação Visual).

4.1.1 Arquitetura do sistema

A plataforma de visualização segue uma arquitetura modular concebida para integrar fontes de dados heterogêneas e representá-las num ambiente 3D em tempo real. O sistema é implementado como uma aplicação C++ construída sobre o Unreal Engine 5.6 e está estruturado em vários subsistemas responsáveis pela comunicação, processamento de dados, gestão de entidades e interação com o utilizador.

A um nível elevado, a plataforma de visualização atua como a camada de apresentação da arquitetura do gêmeo digital. O seu papel é receber informação de serviços externos, transformar essa informação em estruturas de dados internas, e representar as entidades correspondentes num ambiente 3D geoespacialmente preciso. A aplicação é, portanto, responsável tanto pela visualização espacial como pela interação do utilizador com o gêmeo digital.

A arquitetura está organizada em torno de quatro subsistemas principais: serviços de aquisição de dados, componentes de processamento de dados, sistemas de representação de entidades, e componentes de interface do utilizador.

A camada de aquisição de dados é responsável por comunicar com serviços externos e receber informação sobre o estado do gêmeo digital. Estes serviços incluem ligações WebSocket, endpoints de API, e integração com a plataforma Eclipse Ditto. Através destes mecanismos, a aplicação recebe mensagens JSON que descrevem as entidades relevantes.

As classes nesta camada são, na sua maioria, implementadas como subsistemas do Unreal Engine, que são geridos automaticamente pelo motor durante o ciclo de vida da aplicação e fornecem acesso global aos objetos de serviço.

Uma vez recebidos os dados, estes são encaminhados para a camada de processamento de dados, que os analisa (parse) e valida. Nesta fase, o sistema interpreta as estruturas JSON recebidas dos serviços externos. Estas são convertidas em estruturas de dados internas que representam os vários tipos de entidades suportados pela plataforma de visualização. Esta camada também realiza as transformações necessárias, como a conversão de coordenadas e a extração de atributos.

Os dados processados são então passados para o sistema de gestão de entidades, que instancia atores (actors) no ambiente do Unreal Engine. Cada entidade no gêmeo digital é representada como um ator na cena 3D. Estes atores encapsulam tanto a representação visual do objeto como os seus dados associados. O sistema de gestão de entidades garante que novas entidades podem ser criadas, atualizadas ou removidas dinamicamente, à medida que nova informação se torna disponível.

Por fim, a camada de interface do utilizador fornece mecanismos para interagir com o ambiente do gêmeo digital. Através desta camada, os utilizadores podem navegar na cena virtual, selecionar entidades, e inspecionar os seus atributos associados. A interface é implementada usando a

framework de UI do Unreal Engine e está ligada ao sistema de entidades através de componentes gestores (manager) que coordenam a lógica de interação.

Esta arquitetura em camadas fornece uma clara separação de responsabilidades entre a comunicação, o processamento de dados, a representação do mundo, e a interação do utilizador. Esta separação simplifica a manutenção e permite que o sistema evolua de forma incremental à medida que novas fontes de dados, tipos de entidades, ou funcionalidades de visualização são introduzidas.

4.1.2 Aquisição de dados

A plataforma de visualização obtém informação sobre o estado do gémeo digital através de um conjunto de serviços de comunicação que adquirem dados de sistemas externos. Estes serviços proporcionam a ligação entre a aplicação de visualização e a infraestrutura mais ampla do DT4MOB, permitindo que a aplicação receba tanto o estado inicial do sistema como atualizações contínuas à medida que nova informação se torna disponível.

A aquisição de dados é realizada através de dois mecanismos principais: pedidos de API baseados em HTTPS e ligações WebSocket. Cada mecanismo desempenha um papel distinto no pipeline de dados, e, em conjunto, permitem que o sistema suporte tanto a obtenção de dados estáticos como as atualizações em tempo real.

Os pedidos HTTPS são usados principalmente durante a fase de inicialização da aplicação. Quando a plataforma de visualização é iniciada, esta faz pedidos a uma API externa (Eclipse Ditto) para obter as coleções de entidades que já existem no sistema. Estas respostas são normalmente fornecidas em formato JSON e descrevem elementos de infraestrutura, sensores, condições ambientais, e outras entidades do gémeo digital.

Como o backend do gémeo digital contém uma vasta coleção de entidades, as respostas da API são obtidas por paginação. Isto permite que a aplicação peça e processe iterativamente as páginas de dados subsequentes até que o conjunto completo de dados tenha sido obtido. Isto permite que o sistema lide com conjuntos de dados de grande dimensão, carregando as entidades progressivamente em segundo plano.

Os dados recebidos são analisados (parsed) e encaminhados para o sistema de gestão de entidades, que cria as representações correspondentes no ambiente virtual.

Enquanto os pedidos HTTPS fornecem o estado inicial do sistema, as ligações WebSocket são usadas para receber atualizações em tempo real. Através de ligações WebSocket persistentes, a aplicação subscreve fluxos de dados que publicam eventos sempre que o estado de uma entidade muda ou são criadas novas entidades. Este modelo de comunicação permite que a plataforma de visualização mantenha uma representação continuamente atualizada do gémeo digital, sem depender de sondagem (polling) periódica.

Um componente central desta comunicação em tempo real é a integração com o Eclipse Ditto, que atua como o backend do gémeo digital para a gestão do estado de dispositivos e entidades. O Ditto fornece uma interface unificada para aceder e monitorizar o estado das entidades do gémeo digital. A plataforma de visualização estabelece uma ligação WebSocket ao serviço Ditto e subscreve os fluxos de mensagens relevantes. Estas mensagens contêm atualizações sobre as propriedades das entidades do gémeo digital, como alterações de atributos ou leituras de sensores.

As mensagens recebidas chegam em formato JSON e são processadas pelos serviços de comunicação da aplicação. O sistema interpreta a estrutura de cada mensagem, extrai os identificadores e atributos relevantes das entidades, e encaminha os dados resultantes para o pipeline de processamento interno. Esta informação é depois usada para criar novas entidades ou atualizar as já existentes no ambiente virtual.

Ao combinar a obtenção baseada em HTTPS com as atualizações em tempo real baseadas em WebSocket, o sistema garante que a plataforma de visualização pode tanto reconstruir o estado atual do gêmeo digital como permanecer sincronizada com as alterações que ocorrem nos sistemas externos. Esta estratégia híbrida de comunicação permite que a plataforma forneça uma representação precisa e continuamente atualizada da infraestrutura do DT4MOB.

4.1.3 Conversão de coordenadas geográficas

Um requisito fundamental da plataforma de visualização é o posicionamento correto das entidades do gêmeo digital no ambiente 3D. Como a maioria das fontes de dados na plataforma DT4MOB descreve localizações usando coordenadas geográficas, a aplicação de visualização tem de converter estas coordenadas em posições compatíveis com o sistema de coordenadas do Unreal Engine.

Os serviços externos normalmente fornecem informação espacial como valores de latitude e longitude expressos no sistema de coordenadas WGS84, o sistema de referência geográfico padrão usado pela maioria das aplicações de mapeamento e geoespaciais. No entanto, o Unreal Engine usa um sistema de coordenadas cartesiano local, no qual os objetos são posicionados relativamente à origem do mundo virtual. Para representar objetos do mundo real na cena, estas coordenadas geográficas têm, portanto, de ser transformadas no espaço de coordenadas do motor.

Para realizar esta transformação, a plataforma de visualização integra o plugin Cesium for Unreal, que fornece capacidades de referência geoespacial e ferramentas para converter entre coordenadas geográficas e coordenadas do motor. O Cesium permite que a aplicação trabalhe diretamente com valores de longitude, latitude e altitude, convertendo-os automaticamente em posições do mundo do Unreal Engine através do seu sistema de georreferenciação.

Quando uma entidade é recebida de um serviço externo, o sistema extrai primeiro a informação de localização geográfica dos atributos da entidade. Estas coordenadas são então passadas para um Serviço de Conversão de Coordenadas dedicado, dentro da aplicação. Este serviço usa o sistema de georreferenciação do Cesium para calcular a posição correspondente no espaço do mundo do Unreal Engine.

Uma vez realizada a conversão, a posição resultante é usada para colocar o ator da entidade no ambiente 3D. Como o mesmo sistema de referência geoespacial é usado para todas as entidades, os objetos provenientes de diferentes fontes de dados permanecem espacialmente consistentes entre si. Isto garante que os elementos de infraestrutura, sensores, objetos de tráfego, e dados ambientais aparecem corretamente alinhados na representação do gêmeo digital.

A utilização de um componente dedicado de conversão de coordenadas também simplifica a integração de novas fontes de dados. Desde que os dados recebidos incluam coordenadas geográficas, o sistema pode converter e posicionar as entidades no ambiente sem exigir alterações à lógica de visualização. Esta abordagem garante que a plataforma permanece flexível e pode integrar fluxos de dados geoespaciais heterogêneos, mantendo a precisão espacial no ambiente virtual.

4.1.4 Representação de entidades

Após os dados serem recebidos e processados, a plataforma de visualização tem de transformar a informação em objetos que possam ser representados no ambiente 3D. Na aplicação de visualização do DT4MOB, isto é alcançado através de um sistema de representação de entidades que converte os dados recebidos em atores (actors) no mundo do Unreal Engine.

Cada entidade no gêmeo digital é representada por um Actor do Unreal Engine, que encapsula tanto a representação visual do objeto como os seus dados associados. Esta abordagem permite

que a plataforma trate todos os elementos do gémeo digital de forma consistente, independentemente de representarem infraestrutura estática, sensores, informação ambiental, ou dados de tráfego dinâmicos.

Quando novos dados são recebidos dos serviços de aquisição, estes são primeiro analisados para determinar o tipo de entidade a criar. Com base nessa classificação, o sistema converte os dados JSON recebidos dos serviços externos em estruturas de dados internas que representam os atributos da entidade. Para cada tipo de entidade suportado, o sistema define modelos de dados estruturados em C++ que mapeiam o esquema JSON dos serviços externos para objetos fortemente tipados dentro da aplicação Unreal Engine. Estas estruturas simplificam o processamento de dados e garantem que os atributos das entidades podem ser acedidos de forma consistente e segura em termos de tipos, em toda a plataforma de visualização.

Para simplificar a criação e gestão destes objetos, o sistema usa uma fábrica de entidades (entity factory) que instancia dinamicamente os atores na cena. A fábrica recebe os dados processados, determina o tipo de ator apropriado, e cria a entidade correspondente no ambiente virtual. Este design permite que o sistema crie dinamicamente entidades à medida que nova informação se torna disponível a partir dos serviços externos.

Cada ator de entidade armazena os seus dados, incluindo os atributos estruturados extraídos das mensagens JSON recebidas e, quando necessário, a mensagem original em si.

Durante a inicialização, a entidade também determina a sua posição espacial na cena. As coordenadas geográficas associadas à entidade são extraídas dos seus atributos e passadas para o Serviço de Conversão de Coordenadas descrito anteriormente. A posição resultante é então aplicada ao ator, garantindo que a entidade aparece na localização correta dentro do ambiente do gémeo digital.

Esta representação baseada em entidades oferece uma abordagem flexível e extensível para visualizar diversas fontes de dados num ambiente 3D unificado. Ao encapsular tanto os dados como a visualização dentro de objetos-ator, o sistema pode criar, atualizar e remover entidades dinamicamente, à medida que o estado do gémeo digital evolui.

As imagens seguintes apresentam exemplos de entidades e a interface do utilizador usada para exibir os seus dados associados.

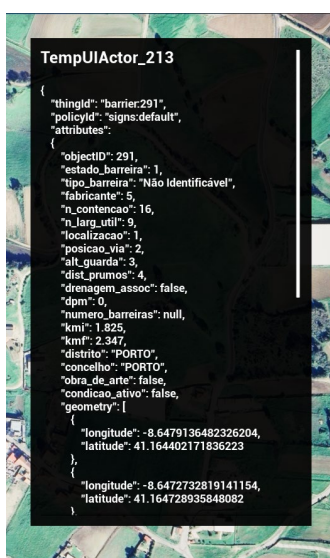


Figura 36 - Exemplo de Informação de Barreira

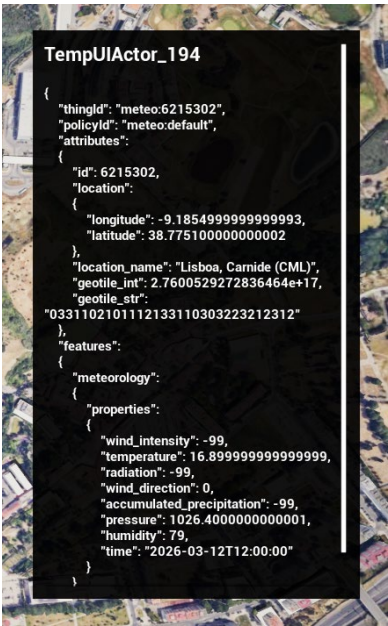


Figura 37 - Exemplo de Informação de Estação Meteorológica



Figura 38 - Exemplo de Informação de Talude

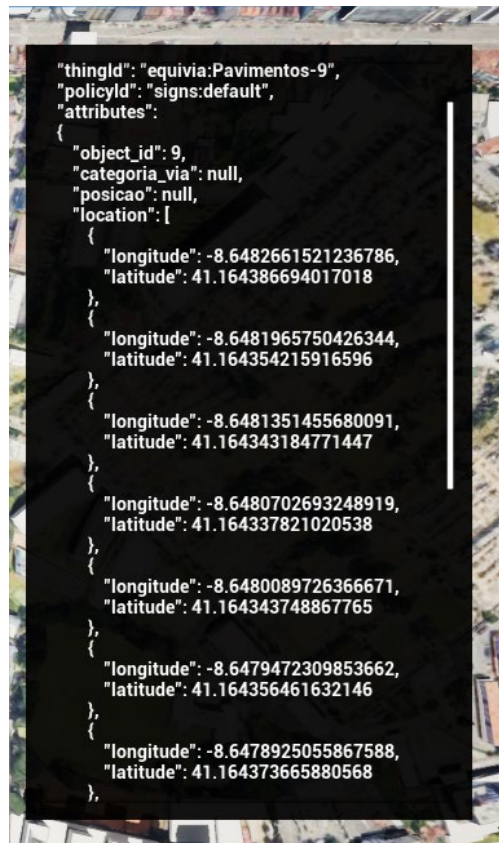


Figura 39 - Exemplo de Informação de Equivia Pavimentos

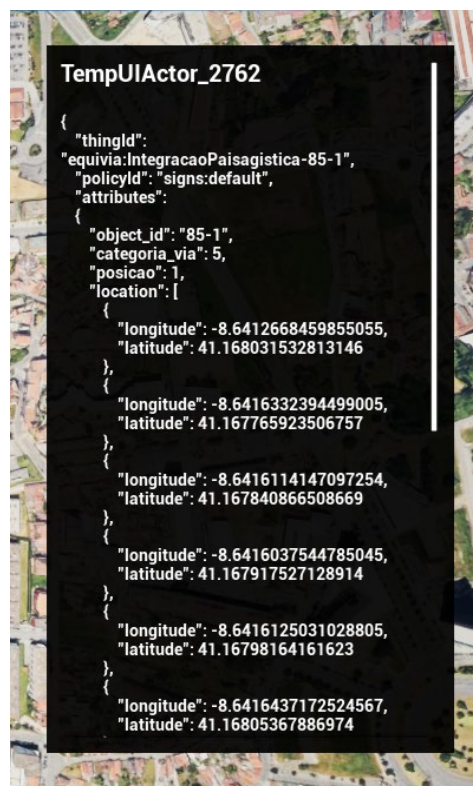


Figura 40 - Exemplo de Informação de Equivia Integração Paisagística

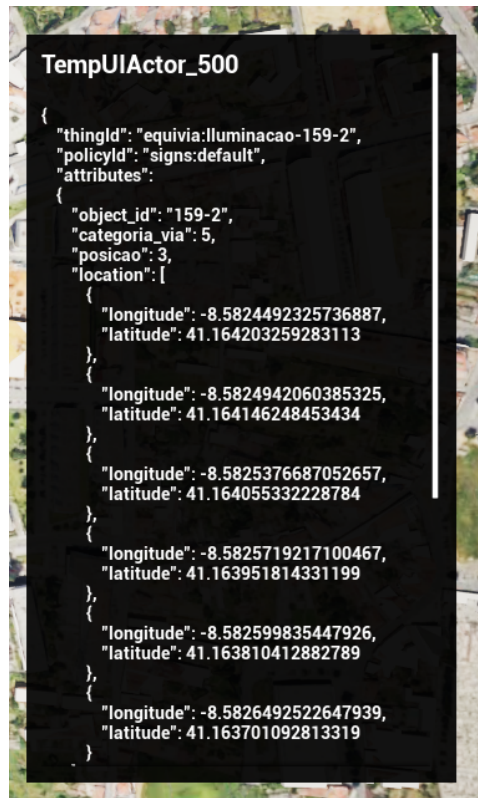


Figura 41 - Exemplo de Informação de Equivia Iluminação

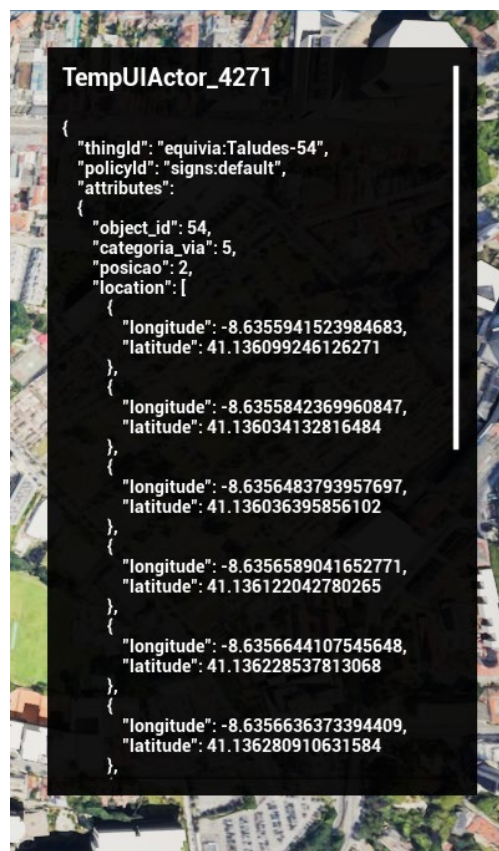


Figura 42 - Exemplo de Informação de Equivia Taludes

4.1.5 Sistema de navegação e câmara

A plataforma de visualização fornece um sistema de navegação interativo que permite aos utilizadores explorar livremente o ambiente do gémeo digital. Este sistema de navegação é implementado usando a framework de jogador (player framework) do Unreal Engine, que separa as responsabilidades entre vários componentes que tratam da entrada (input), do comportamento da câmara, e da lógica de movimento.

O ponto de vista do utilizador na cena é representado por um Pawn, que serve como a entidade controlável que o utilizador usa para navegar no ambiente. O Pawn representa a presença física da câmara no mundo e serve como o componente central responsável pelo movimento e pela interação espacial. Ao contrário dos sistemas de navegação tradicionais baseados em personagens, a plataforma de visualização usa uma abordagem de navegação livre, permitindo que o utilizador se mova pelo ambiente do gémeo digital sem restrições físicas.

A entrada do utilizador é processada através de um Player Controller, que atua como a interface entre as ações do utilizador e o comportamento do Pawn. O Player Controller recebe eventos de entrada do teclado e do rato, e traduz-los em comandos de movimento ou ações da câmara. Estas entradas são depois encaminhadas para o Pawn ou para os seus componentes de movimento associados, permitindo que o utilizador controle a posição e a orientação da câmara na cena.

A lógica de movimento é implementada através de componentes de movimento dedicados, que definem como o Pawn se move através do ambiente. Estes componentes interpretam os comandos de entrada recebidos do Player Controller e aplicam as translações e rotações apropriadas ao Pawn. Este design modular permite que o sistema de movimento seja estendido ou ajustado sem modificar a framework central do jogador.

O sistema de navegação foi concebido para facilitar a exploração de ambientes geoespaciais de grande escala. Como o gémeo digital representa infraestrutura e localizações geográficas do mundo real, os utilizadores precisam de navegar eficientemente por grandes distâncias, mantendo simultaneamente um controlo preciso ao examinar entidades específicas. Consequentemente, os utilizadores podem alternar entre uma câmara de visão geral rápida e uma câmara de voo livre precisa, para inspecionar entidades ao nível do solo mais de perto.

Através desta arquitetura, a plataforma de visualização permite que os utilizadores explorem intuitivamente o ambiente do gémeo digital, naveguem entre áreas de interesse, e inspecionem entidades a partir de diferentes perspetivas na cena 3D.

4.1.6 Gestores e lógica de aplicação

Para coordenar as interações entre os diferentes sistemas da plataforma de visualização, a aplicação inclui um conjunto de componentes gestores (managers) que tratam da lógica global da aplicação. Estes gestores atuam como camadas intermédias entre as entidades do mundo, a interface do utilizador, e os sistemas de interação do utilizador, garantindo que as diferentes partes da aplicação permanecem levemente acopladas (loosely coupled).

Um dos componentes-chave deste sistema é o gestor de seleção (selection manager), que gere o estado de seleção das entidades no ambiente virtual. Quando o utilizador interage com objetos na cena, normalmente através da entrada do rato, o sistema determina qual o ator visado. O gestor de seleção atualiza então o estado de seleção atual e notifica os outros subsistemas que dependem desta informação.

Este mecanismo permite que diferentes partes da aplicação respondam a alterações de seleção sem estarem diretamente ligadas à lógica de tratamento de entrada. Por exemplo, quando uma entidade é selecionada, o gestor de seleção difunde um evento indicando que o objeto selecionado

mudou. Outros sistemas, como os componentes da interface do utilizador, podem subscrever esses eventos e atualizar o seu estado de acordo.

Além do gestor de seleção, a aplicação inclui um componente de gestão de UI que coordena a visibilidade e o comportamento dos elementos da interface. Este gestor controla o ciclo de vida dos widgets da interface do utilizador, tais como painéis de informação e janelas de inspeção. Quando uma entidade é selecionada, o gestor de UI recebe a notificação correspondente e atualiza a interface para exibir a informação relevante.

Estes componentes gestores desempenham um papel importante na manutenção de uma separação clara entre subsistemas. Em vez de permitir que as entidades e os elementos da interface do utilizador comuniquem diretamente, os gestores fornecem pontos de coordenação centralizados. Esta abordagem reduz as dependências entre componentes e simplifica a extensão ou modificação da aplicação.

Através desta camada de gestão, a plataforma de visualização mantém um estado de aplicação consistente e garante que as interações no ambiente 3D são refletidas com precisão na interface do utilizador.

4.1.7 Interface do utilizador

A plataforma de visualização inclui uma interface gráfica de utilizador que permite aos utilizadores interagir com o ambiente do gémeo digital e inspecionar os dados associados às entidades na cena. A interface foi implementada usando o Unreal Motion Graphics (UMG), a framework nativa do Unreal Engine para criar elementos de interface de utilizador interativos.

A interface do utilizador é o principal meio através do qual os utilizadores obtêm informação detalhada sobre as entidades no ambiente de visualização. Enquanto a cena 3D fornece contexto espacial e representação visual, a interface permite ao utilizador aceder aos atributos e metadados associados a cada entidade.

A interface está estruturada em torno de um widget HUD raiz, que serve como o contentor central para os vários elementos de UI exibidos no ecrã. Este componente raiz inicializa e gere os painéis de interface que possam surgir durante a interação com a aplicação. Através deste sistema, a plataforma pode criar e exibir dinamicamente janelas de informação com base no estado atual da aplicação.

Um dos principais elementos da interface é a janela de inspeção de entidades, que se torna visível quando uma entidade é selecionada na cena. Esta janela exibe os dados associados ao objeto selecionado, incluindo o seu identificador e atributos provenientes de fontes de dados externas. A informação é apresentada num layout estruturado que permite ao utilizador compreender rapidamente as características e o estado da entidade selecionada.

A interface do utilizador está ligada aos sistemas de interação através dos componentes gestores descritos anteriormente. Quando o gestor de seleção deteta que uma nova entidade foi selecionada, notifica o gestor de UI, que por sua vez atualiza a interface para exibir a informação correspondente à entidade. Esta abordagem baseada em eventos garante que a interface permanece sincronizada com o estado atual do ambiente do gémeo digital.

Embora a implementação atual se foque em apresentar os dados das entidades de forma clara e funcional, a estrutura modular da interface permite que esta seja estendida no futuro com vistas mais especializadas. Estas podem incluir painéis personalizados para tipos específicos de entidades, indicadores visuais do estado do sistema, ou controlos interativos para manipular elementos dentro do gémeo digital.

Através desta camada de interface, a plataforma de visualização permite que os utilizadores explorem o ambiente do gémeo digital enquanto acedem a informação detalhada de cada entidade, fornecendo tanto consciência espacial como inspeção contextual de dados, num sistema unificado.

4.1.8 Fluxo de interação do utilizador

A interação do utilizador com a plataforma de visualização segue um fluxo de trabalho estruturado que liga o sistema de navegação, os componentes de gestão de entidades, e a camada de interface do utilizador. Este fluxo de trabalho garante que as ações do utilizador no ambiente 3D são corretamente interpretadas pela aplicação e refletidas na interface.

O processo de interação começa normalmente quando o utilizador navega pela cena usando o sistema de câmara e movimento descrito anteriormente. Enquanto explora o ambiente, o utilizador pode interagir com objetos no gémeo digital ao passar o cursor sobre eles e selecioná-los usando ações de entrada.

Quando ocorre um evento de entrada, o player controller processa a entrada e realiza uma consulta de raycasting a partir da câmara para a cena 3D. Este raycast determina se um ator de entidade foi visado. Se for detetada uma entidade válida, o evento é encaminhado para o gestor de seleção, que atualiza o objeto atualmente selecionado no estado da aplicação.

Quando o gestor de seleção regista uma nova entidade selecionada, difunde uma notificação indicando que o estado de seleção mudou. Outros sistemas que dependem desta informação, particularmente o gestor de UI, recebem esta notificação e reagem em conformidade.

O gestor de UI atualiza então a interface, abrindo ou atualizando a janela de inspeção de entidades associada ao objeto selecionado. Esta janela obtém os atributos armazenados no ator da entidade e exibe-os num formato estruturado, permitindo que o utilizador inspecione os dados associados a essa entidade.

Este pipeline de interação estabelece um fluxo claro de informação entre os componentes do sistema. As ações do utilizador são captadas pelos sistemas de entrada e navegação, interpretadas pelos gestores de interação, e finalmente refletidas na interface do utilizador.

Através deste fluxo de trabalho, a plataforma de visualização fornece uma forma intuitiva para os utilizadores explorarem o ambiente do gémeo digital e acedem à informação associada às entidades representadas na cena. A estrutura modular deste pipeline de interação também permite que novas funcionalidades de interação ou elementos de interface sejam integrados no sistema sem exigir alterações significativas à arquitetura existente.

5 FERRAMENTAS, LINGUAGENS E BIBLIOTECAS UTILIZADAS

5.1 Plataforma Digital Twins

A plataforma de gémeos digitais baseia-se em tecnologias de código aberto, nomeadamente nos projetos de IoT da Fundação Eclipse: o Hono, que funciona como camada de ingestão da plataforma, e o Ditto, que armazena e disponibiliza os gémeos digitais através de interfaces programáticas.

Ambos os componentes dependem de outros projetos e bibliotecas de código aberto, nomeadamente o Apache Kafka, utilizado como camada de mensagens entre eles, o MongoDB, que atua como camada de armazenamento dos gémeos digitais, e a configuração da camada de ingestão.

Toda a plataforma é orquestrada utilizando o Kubernetes e paradigmas nativos da nuvem. Foi desenvolvido um operador personalizado na linguagem de programação Go para gerir o ciclo de vida da plataforma, nomeadamente a criação de inquilinos, a distribuição de certificados e a configuração da camada de mensagens.

O aprovisionamento da plataforma é fornecido num modelo «como serviço» através do Sistema de Apoio Operacional de Código Aberto (OSS) Openslice. Isto permite que um cliente solicite a plataforma como serviço através de uma interface de utilizador simples baseada na Web. Após a aprovação do administrador, o sistema configura e implementa automaticamente todos os componentes relevantes. Para que isto seja possível, todos os componentes são empacotados como Helm Charts. Na maioria dos casos, estes já são fornecidos a montante. No entanto, alguns não os possuem e têm de ser empacotados durante o projeto, enquanto outros apresentavam lacunas que foram corrigidas com o objetivo de integrar este trabalho a montante no futuro. Todos os Helm Charts são carregados para o Harbor, um registo OCI, para que possam ser obtidos pelo cluster.

O OpenSlice criará manifestos de aplicação ArgoCD para cada componente durante o aprovisionamento da plataforma. O ArgoCD é utilizado para suportar estratégias de implementação mais complexas, atualizações de versão e monitorização agregada do estado de funcionamento. Estes manifestos são configurados com o Helm Chart relevante armazenado no registo OCI e, em alguns casos, são fornecidos manifestos adicionais através de um repositório Git para disponibilizar recursos específicos do DT4Mob aos componentes subjacentes

5.1.1 Segurança

Para garantir a confidencialidade e a integridade da informação processada pela plataforma, são implementados controlos de segurança rigorosos em toda a plataforma. O primeiro é a utilização de Transport Layer Security (TLS) para toda a comunicação na plataforma. Isto aplica-se não só ao tráfego proveniente do exterior (parceiros, dispositivos e outros), mas também ao tráfego entre os componentes internos da plataforma, incluindo o tráfego para a base de dados e para o message broker.

O TLS é usado não só para criar uma ligação segura entre dois componentes, mas também para os autenticar mutuamente. Para os componentes dentro do cluster Kubernetes, isto é feito através da gestão das Autoridades Certificadoras (CA) no próprio cluster. Aos dispositivos e componentes externos é atribuída uma CA que emite todos os certificados usados nos pontos de entrada (ingress), podendo também obter certificados de cliente através de um portal de self-service para parceiros. Isto cobre toda a autenticação máquina-a-máquina na plataforma.

Para os componentes destinados a operadores humanos, tais como o portal de self-service de certificados, o Visualizador 3D, o Ditto Dashboard, e outros, o Keycloak serve como Fornecedor de Identidade (Identity Provider) para gerir contas de utilizador e fornecer autenticação centralizada.

5.1.2 Monitorizarização

Para garantir o funcionamento contínuo da plataforma e a rápida resolução de problemas à medida que surgem, é implementada, em conjunto com a plataforma, uma stack de monitorização baseada em ferramentas de código aberto da Grafana. Os dados de monitorização são automaticamente agregados e etiquetados a partir de todos os componentes da plataforma, sendo depois processados e armazenados para análise e visualização.

Estão também disponíveis múltiplos dashboards, com métricas comuns representadas graficamente para uma visualização e referência cruzada rápidas, e com os logs relevantes de cada componente exibidos ao lado. Da mesma forma, podem ser configurados alertas para notificar os operadores sobre condições anómalas na plataforma antes que estas possam causar problemas.

Para permitir também o ajuste fino (fine-tuning) e a análise de desempenho da plataforma, são recolhidos e armazenados traces de execução. Isto permite a rápida identificação de estrangulamentos (bottlenecks) na plataforma e de casos-limite (edge cases) patológicos no processamento dos dados do gémeo digital. Nem todos os traces de execução são armazenados; em vez disso, são amostrados estatisticamente. Isto é feito para reduzir as necessidades de armazenamento dos traces de execução, uma vez que, de outra forma, seria proibitivamente dispendioso.

5.1.3 Serviços

A secção anterior descreveu o núcleo da plataforma, que fornece o conjunto mínimo de componentes necessários para tratar a ingestão de dados do mundo real e criar gémeos digitais a partir deles. No entanto, a plataforma foi concebida para ser extensível, de forma a suportar não apenas os casos de uso atualmente definidos, mas também outros que possam surgir.

As secções seguintes descrevem os serviços que já foram criados para estender a plataforma e suportar os casos de uso atuais.

5.1.3.1 Armazenamento e recuperação de dados históricos

O Eclipse Ditto, o núcleo da plataforma, foi concebido para armazenar apenas o estado atual de um gémeo digital. Sempre que uma nova atualização é recebida, o valor anteriormente armazenado é substituído. Para suportar casos de uso adicionais, tais como a análise de tendências históricas, o treino de IA, e a análise de dados, tornou-se necessário implementar um serviço de dados históricos.

Para este efeito, foi selecionada uma base de dados de séries temporais que cumprisse os requisitos do sistema. A solução escolhida foi o TimescaleDB, uma base de dados de séries temporais de código aberto, otimizada para SQL e distribuída como uma extensão do PostgreSQL. Para interagir com a base de dados, foram desenvolvidos módulos em Python: um para escrever as atualizações recebidas e outro para fornecer uma API de obtenção de dados históricos.

O módulo de escrita foi desenvolvido para consumir eventos exportados do Eclipse Ditto através do Apache Kafka, e para os persistir na base de dados. Para tal, o módulo usa o pacote `confluent_kafka` para interagir com o cluster Kafka, e o pacote `SQLModel` para interagir com a base de dados de forma “pythonica”.

A API responsável pela obtenção de dados usa também a extensão `SQLModel` para interagir com a base de dados. Para simplificar e acelerar o desenvolvimento, foi adotado o `FastAPI`, permitindo a criação de múltiplos endpoints e fornecendo funcionalidades como a injeção de dependências para acesso à base de dados e gestão de ligações. A autenticação é gerida através de verificação de token fornecido por um servidor de `keycloak`, o token contém a informação do utilizador e é verificado através do servidor fornecendo as roles do mesmo.

Ambos os módulos foram desenvolvidos usando o `uv`, o gestor de projetos e dependências desenvolvido pela Astral. Para gerir as definições da aplicação e criar modelos de dados "Pythonic" com validação incorporada, foi usada a biblioteca `Pydantic`, permitindo a definição de modelos `strongly typed` e a verificação automática de tipos durante o desenvolvimento.

5.1.3.1.1 Modelo de dados históricos

O Eclipse Ditto armazena o estado atual de um gémeo digital. No entanto, para muitas aplicações e casos de uso, é também necessário aceder a dados históricos que descrevam como o estado do gémeo evoluiu ao longo do tempo. O acesso a esta informação histórica permite o desenvolvimento de modelos de IA e de aprendizagem automática, apoia a tomada de decisões informada através da visualização das alterações de estado, e facilita a análise de tendências de longo prazo e de padrões emergentes. Esta análise pode revelar anomalias, comportamentos recorrentes, e condições em evolução que poderiam não ser evidentes apenas a partir do estado atual, permitindo previsões mais precisas e uma tomada de decisão proativa.

Para criar informação histórica sobre todos os estados do gémeo, foi desenvolvido um módulo de dados históricos para armazenar o histórico dos estados de múltiplas Things. O modelo usado para estes dados históricos é baseado no modelo usado pelo Eclipse Ditto para eventos. Este segue uma estrutura de evento em que cada evento representa uma alteração numa Thing, baseada no caminho (path) do dado alterado; isto significa que cada evento está associado a um caminho onde foi editado, por exemplo `/Feature/posição`. Cada registo contém o `Thing_id`, a hora do evento, a ação realizada, o caminho do atributo modificado, e o valor correspondente que foi atualizado. O valor é armazenado usando `JSONb`, permitindo que o sistema suporte múltiplas estruturas de dados.

<i>HistoricDittoEvent</i>	<i>Action(Enum)</i>
time: datetime	modified
thingId: string	merged
action: Action	created
revision: int	deleted
path: string	
value: jsonb	

Figura 43 - Definição da tabela da base de dados

A imagem seguinte contém um exemplo de um evento existente na base de dados:

```
{
  "action": "merged",
  "time": "2026-06-25T15:28:46.057513",
  "path": "/",
  "thing_id": "traci:Car-118670828_0_35539503.qPKW.0.20",
  "revision": 471,
  "value": {
    "features": {
      "state": {
        "properties": {
          "accel": 2.463778059277683,
          "angle": 107.70790682924218,
          "speed": 8.774375825925071,
          "latitude": 41.16320437053306,
          "longitude": -8.581618035264883
        }
      }
    },
    "attributes": {
      "geotile": 3752843375,
      "expiry_ts": "2026-06-25T15:30:44.092741+00:00"
    }
  }
}
```

Figura 44 - Exemplo de evento presente na base de dados

5.1.3.1.2 Arquitetura de software

A implementação deste serviço é baseada em três componentes principais:

- **Base de dados:** É usada uma base de dados de séries temporais para armazenar eficientemente os eventos históricos e otimizar as consultas em intervalos de tempo (time buckets). Este design permite o processamento e a obtenção de grandes volumes de eventos com baixa latência.
- **API:** Uma interface programática que fornece aos utilizadores e a outros componentes da infraestrutura acesso aos dados históricos armazenados na base de dados. A API expõe endpoints para consultar e gerir os dados, permitindo uma integração transparente (seamless) com aplicações e serviços externos.
- **Writer:** Um microserviço responsável por receber os eventos gerados pelo Eclipse Ditto e persisti-los na base de dados de séries temporais. Este componente garante que todas as alterações no estado do gémeo digital são registadas, permitindo que o histórico completo do gémeo seja reconstruído e consultado.

No diagrama abaixo, podemos ver como os três componentes, Writer, API, e base de dados, são usados na infraestrutura, ilustrando o fluxo de dados desde o Eclipse Ditto, através do message broker, para o serviço Writer, e finalmente para a base de dados de séries temporais, onde fica disponível para consulta através da API.

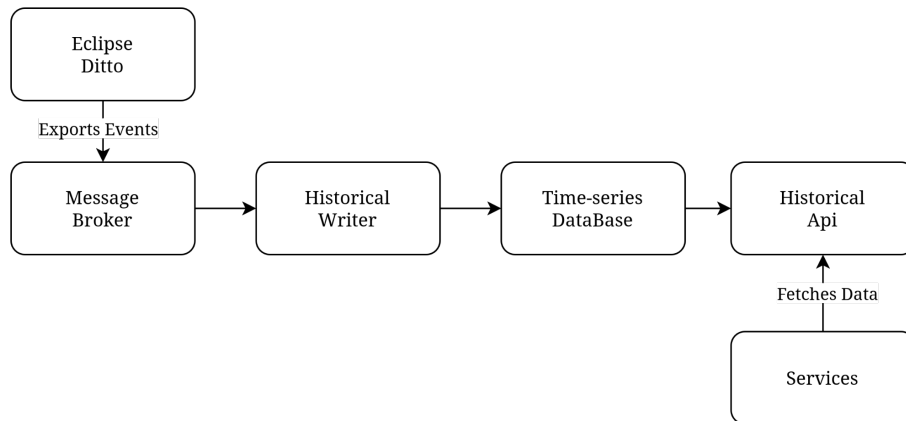


Figura 45 - Interação dos componentes com a infraestrutura

5.1.3.1.3 Historical Writer

O Historical Writer é um microserviço central responsável por captar e persistir a evolução das alterações de estado do gêmeo digital, geradas pelo Eclipse Ditto. Atua como uma camada de ingestão para o fluxo de eventos, recebendo mensagens e escrevendo-as na base de dados de séries temporais, garantindo que todas as transições de estado relevantes são armazenadas de forma fiável para análise posterior.

O serviço funciona subscrevendo um message broker através do qual o Eclipse Ditto publica os eventos de atualização do gêmeo. Estes eventos representam alterações nas Things do Ditto e incluem normalmente o caminho (path) que foi modificado, juntamente com o valor atualizado correspondente. Assim, cada evento descreve uma alteração precisa no estado de um gêmeo digital.

Uma vez recebido um evento, este é escrito diretamente na base de dados de séries temporais, onde é armazenado numa estrutura otimizada para consultas eficientes baseadas em tempo e para obtenção posterior. Isto permite análises, análise de tendências, e a reconstrução histórica do estado do gêmeo digital ao longo do tempo.

5.1.3.1.4 Historical Data API

Para facilitar o acesso aos dados históricos, foi desenvolvida uma API para a base de dados. A API fornece uma interface estruturada e controlada para consultar dados de séries temporais, desacoplando os consumidores externos da implementação subjacente da base de dados.

Permite aos utilizadores pesquisar eventos num intervalo de tempo específico e por um prefixo de Thing_id, obter todos os eventos associados a uma única entidade num intervalo de tempo definido, e listar todas as entidades atualmente armazenadas na base de dados. Suporta também agregações baseadas numa chave dentro do valor do evento, bem como projeções de campos específicos, reduzindo a necessidade de obter os payloads completos dos eventos quando apenas dados parciais são necessários.

Adicionalmente, a API suporta a inserção e eliminação manual de eventos, permitindo a inclusão de dados históricos que possam existir independentemente do Eclipse Ditto, mas que nunca foram processados através dele.

A API foi concebida principalmente para suportar aplicações downstream, tais como pipelines de análise, ferramentas de visualização, e modelos de aprendizagem automática que dependem de dados históricos do gêmeo digital. Ao fornecer capacidades eficientes de consulta baseadas em

tempo, permite a obtenção escalável de grandes volumes de eventos, mantendo uma separação clara entre as camadas de armazenamento e de consumo de dados.

Para melhor documentação e facilidade de utilização ao implementar serviços baseados nesta API, está disponível uma interface Swagger UI como parte da infraestrutura.

A tabela abaixo resume todos os endpoints disponíveis da API.

Método	Endpoint(Path)	Descrição	Notas
GET	/historic/things	Obtém todas as Things na base de dados	Aceita um prefixo do Thing_id para filtrar
GET	/historic/events	Obtém todos os eventos num intervalo de tempo, podendo filtrar por prefixo de Thing_id	Usa since e until para o intervalo de tempo, e aceita um prefixo de Thing_id para filtrar
POST	/historic/events	Insere eventos na base de dados, para preencher eventos que não foram processados através do Ditto	—
GET	/historic/events/{thing_id}	Obtém todos os eventos num intervalo de tempo para uma Thing	Usa since e until para o intervalo de tempo
DELETE	/historic/events/{thing_id}	Elimina eventos num intervalo de tempo de uma Thing	Usa since e until para o intervalo de tempo
GET	/events/projection/{thing_id}	Extraí campos do valor das Things filtradas, com base no JSONPath fornecido	Usa since e until para o intervalo de tempo, e thing_id e path_prefix para filtrar
GET	/events/time-buckets/{thing_id}	Realiza agregações por intervalos de tempo (time-buckets), com base no intervalo, operação e JSONPath fornecidos	Usa since e until para o intervalo de tempo, e thing_id e path_prefix para filtrar

Tabela 29 - Endpoints da Historical Data API

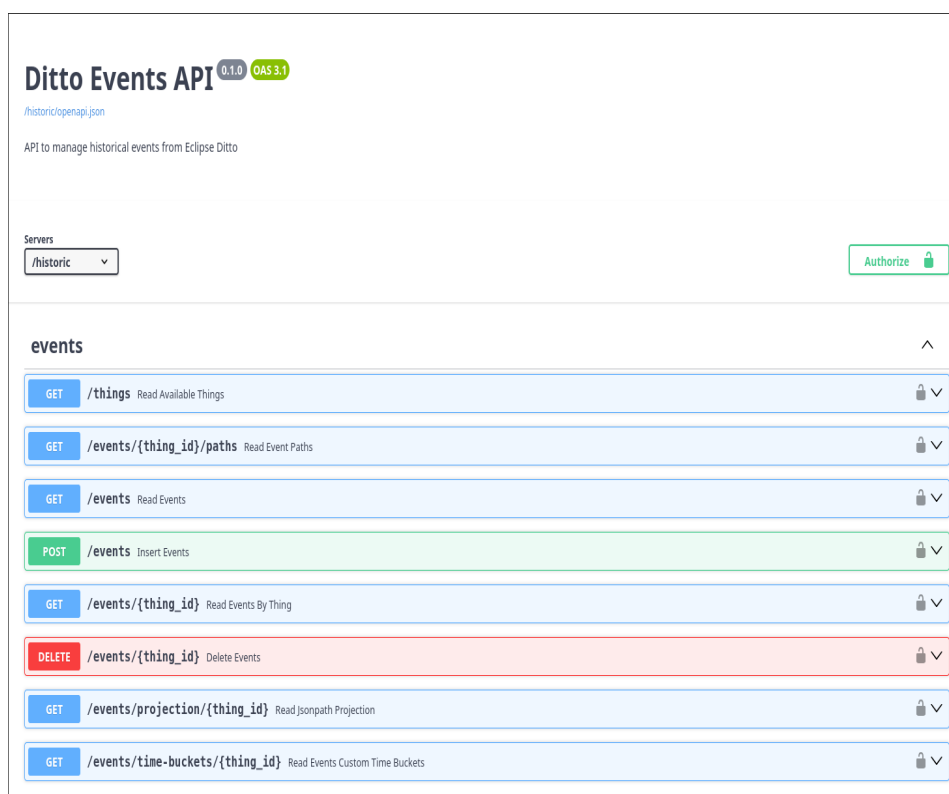


Figura 46 - Especificação OpenAPI da Historical Data API

5.1.3.2 Garbage Collector

Em sistemas onde múltiplos serviços criam entidades de gêmeos digitais, é comum que entidades obsoletas ou não utilizadas permaneçam na plataforma, uma vez que os serviços individuais frequentemente não têm visibilidade total nem a responsabilidade de rastrear o ciclo de vida de todas as entidades criadas. Para resolver este problema, foi desenvolvido em Python um serviço Garbage Collector, que identifica e remove periodicamente as entidades de gêmeos digitais expiradas, evitando a acumulação desnecessária de dados.

O serviço introduz um atributo padrão opcional, `expirar_ts`, que armazena o timestamp de expiração de uma entidade. Durante cada execução, o serviço consulta a API de Pesquisa (Search API) do Eclipse Ditto por entidades que contenham este atributo com um valor anterior ao momento atual, usando a consulta RSQL `It(attributes/expirar_ts,'{timestamp}')`. Os resultados são processados em grupos de até 500 entidades, para evitar esperar pelo conjunto completo de resultados.

Para cada entidade expirada, o serviço envia um evento de eliminação (delete) através do protocolo Ditto, usando uma ligação WebSocket, acionando a remoção do gêmeo digital correspondente. O serviço é executado de cinco em cinco minutos, como um Job Kubernetes dentro do cluster.

Em termos de implementação, o serviço usa a biblioteca `httpx` para os pedidos HTTP à Search API, `gmqtt` para a comunicação MQTT, `Pydantic` para a modelação e validação de dados, e `uv` para a gestão do projeto e das dependências.

5.2 Data Bridge

O Data Bridge é uma aplicação Python responsável por integrar diferentes fontes de dados, sejam serviços externos ou ficheiros locais, nos gêmeos digitais, enviando os dados normalizados para o Eclipse Hono, que por sua vez os reencaminha para o Eclipse Ditto.

Para tratar a modelação de dados, foi usada a biblioteca *Pydantic*. Toda a comunicação HTTP é gerida pela biblioteca Python *aiohttp*, devido às suas capacidades assíncronas. Para gerir as projeções e cálculos geográficos, foram usadas as bibliotecas *Geopy* e *pyproj*. O registo de logs é feito através da biblioteca Python *loguru*.

Quanto às ferramentas de desenvolvimento, foi usado o sistema de gestão *uv*, juntamente com o linter e formatador *ruff*, e a ferramenta de verificação de tipos e LSP *ty*.

5.3 Meteo Populator

O Meteo Populator é uma aplicação Python responsável por atualizar diferentes gémeos digitais com as estações meteorológicas mais próximas, que são representadas como o ThingId do gémeo digital que representa essas estações meteorológicas.

A gestão de dependências e do projeto foi feita usando o sistema de gestão de pacotes *uv*, juntamente com o linter e formatador *ruff* e a ferramenta de verificação de tipos e LSP *ty*. A criação de logs é feita com o auxílio da biblioteca Python *loguru*. A modelação de dados é feita usando o *Pydantic*, sendo a comunicação HTTP assíncrona gerida pelas bibliotecas *aiohttp* e *AsyncIO*.

5.4 Texture Creation

Texture Creation é uma pipeline de processamento de nuvens de pontos desenvolvido em Python que, dado uma nuvem de pontos com dados de deslocamento de um muro de suporte, cria diferentes texturas para ajudar a visualizar as respetivas deformações. Este pipeline consiste em carregar não só a nuvem de pontos fornecida, mas também um ficheiro de polígono ao qual a textura será aplicada, terminando com o carregamento dos resultados para uma API compatível com S3.

Tal como nos componentes anteriores, a gestão de dependências e do projeto foi feita usando o sistema de gestão de pacotes *uv*, juntamente com o linter e formatador *ruff*, e a ferramenta de verificação de tipos e LSP *ty*. A interação com a API compatível com S3 é feita com a biblioteca *boto3*, e para a comunicação HTTP foi usada a biblioteca *httpx*. Para a modelação de dados, foi usada a biblioteca *Pydantic*, e para gerir operações com nuvens de pontos e malhas (meshes), foram usadas as bibliotecas *pyvista*, *trimesh* e *xatlas*; estas bibliotecas dependem de *numpy*, *pandas*, *pillow* e *scipy*.

5.5 Namespace Remover

O Namespace Remover é um utilitário de eliminação em massa desenvolvido em Python que, dado um conjunto de namespaces e/ou uma consulta RQL personalizada, se autentica numa instância do Eclipse Ditto e elimina todas as Things correspondentes, de forma a limpar entidades órfãs ou obsoletas. Este utilitário consiste em autenticar-se via OIDC password grant contra a instância Keycloak da infraestrutura, pesquisar o endpoint Things-Search do Eclipse Ditto com paginação baseada em cursor, e emitir pedidos DELETE para cada Thing devolvida.

Tal como nos componentes anteriores, a gestão de dependências e do projeto foi feita usando o sistema de gestão de pacotes `uv`, juntamente com o linter e formatador `ruff`, e a ferramenta de verificação de tipos e LSP `ty`. Para a comunicação HTTP e pedidos assíncronos, foi usada a biblioteca `aiohttp`. Para tratar o carregamento de configurações a partir de ficheiros TOML e variáveis de ambiente, foi usada a biblioteca `pydantic-settings`, e para o processamento de argumentos de linha de comandos, foi usado o módulo `argparse` da biblioteca padrão.

5.6 Gantry Service

A aplicação Gantry Service é escrita em Python e usa estas três bibliotecas principais de comunicação:

- **aMQTT** — Para enviar e receber mensagens de brokers MQTT; esta biblioteca foi escolhida pelas suas capacidades assíncronas, tipagem mais forte, e facilidade de utilização.
- **FastAPI** — Para atuar como webhook, escolhida devido ao seu vasto conjunto de funcionalidades e facilidade de configuração e desenvolvimento, juntamente com uma boa integração com o `asyncio`.
- **HTTPX** — Esta é a principal biblioteca usada para todos os pedidos HTTP, uma vez que possui uma excelente tipagem e ótimas capacidades assíncronas.

Para tratar todas as necessidades de modelação de dados, foi usado o `Pydantic`, juntamente com o seu módulo de gestão de definições (settings), o que permite uma aplicação altamente flexível e configurável.

A aplicação foi desenvolvida usando o sistema de gestão `uv`, formatada com o `ruff`, e verificada quanto a tipos usando o `mypy`.

Todas as bibliotecas acima mencionadas usam a licença MIT, com exceção do `httpx`, que tem uma licença BSD de 3 cláusulas.

Ferramenta/Biblioteca	Propósito
Ferramenta/Biblioteca	Propósito
Python 3.13	Ambiente de execução
uv	Gestão do ambiente virtual e das dependências
amqtt	Cliente MQTT assíncrono para interagir com o Hono
httpx	Cliente HTTP assíncrono para o HTTP Sender e o Consumer Webhook
Pydantic	Validação e serialização do esquema de mensagens
pydantic-settings	Configuração através de ficheiro TOML e variáveis de ambiente
loguru	Registo de logs estruturado

Tabela 30 - Ferramentas utilizadas no desenvolvimento do Gantry Service

5.7 Serviço de conversão

O Conversion Service foi também construído em Python e reutiliza a maioria das bibliotecas incluídas no Gantry Service. A exceção é o `httpx`, que não é usado nesta aplicação.

Além destas bibliotecas, a aplicação usa também duas novas: `Numpy` e `pymap3d`, ambas escolhidas pelas suas funcionalidades: o `Numpy` permite cálculos matemáticos complexos e rápidos usando matrizes, e o `pymap3d` permite conversões simples de coordenadas planas para coordenadas WGS84.

O `Numpy` está sob uma licença BSD de 3 cláusulas, e o `pymap3d` está sob uma licença BSD de 2 cláusulas.

Ferramenta/Biblioteca	Propósito
Python 3.13	Ambiente de execução
uv	Gestão do ambiente virtual e das dependências
FastAPI	Interface HTTP para o consumer webhook
httpx	Cliente HTTP assíncrono para o HTTP Sender e o Consumer Webhook
Numpy	Operações matriciais para transformações de coordenadas
pymap3d	Conversão de coordenadas geodésicas ↔ ENU (geodetic2enu, enu2geodetic)
Pydantic	Validação e serialização do esquema de mensagens
pydantic-settings	Configuração através de ficheiro TOML e variáveis de ambiente
amqtt	Cliente MQTT assíncrono para interagir com o Hono

Tabela 31 - Ferramentas utilizadas no desenvolvimento do serviço de conversão de dados de sensores

5.8 Script de carregamento da API LevelAMS

O script LevelAMS Loader foi construído em Python e usa muitas das bibliotecas já mencionadas, como `amqtt`, `httpx` e `Pydantic`. As principais diferenças são a adição do módulo `pydantic-argparse` para as suas capacidades de análise de argumentos de linha de comandos (CLI), integrado com o sistema de validação mais robusto do `Pydantic`.

Além disso, foi adicionado o `pyproj`, usado para converter do sistema de coordenadas ETRS89/Portugal TM06 para WGS84.

Todas as bibliotecas mencionadas usam a licença MIT, com exceção do `httpx`, que tem uma licença BSD de 3 cláusulas.

A tabela abaixo resume as ferramentas usadas no desenvolvimento do LevelAMS Loader:

Ferramenta/Biblioteca	Propósito
Python 3.13	Ambiente de execução
uv	Gestão do ambiente virtual e das dependências
httpx	Cliente HTTP assíncrono para o HTTP Sender e o Consumer Webhook
amqtt	Cliente MQTT assíncrono para interagir com o Hono
Pydantic	Validação e serialização do esquema de mensagens
pydantic-settings	Configuração através de ficheiro TOML e variáveis de ambiente
pydantic-argparse	Análise de argumentos de linha de comandos usando Pydantic
pyproj	Transformação de projeção de coordenadas ETRS89 TM06 → WGS84
loguru	Registo de logs estruturado

Tabela 32 - Ferramentas utilizadas no desenvolvimento do LevelAMS Loader

5.9 Serviço de simulação de incêndios

O Serviço de Simulação de Incêndios baseia-se numa arquitetura de wrapper, na qual o Python atua como uma camada de coordenação que prepara e gere todos os dados geoespaciais e meteorológicos obrigatórios, delegando a modelação computacionalmente intensiva da propagação do fogo ao motor ForeFire, escrito em C++. Neste design, o Python trata a ingestão de eventos do Eclipse Ditto, a construção de rasters e ficheiros NetCDF da paisagem, o processamento de dados de vento, e a orquestração das execuções de simulação, enquanto o ForeFire realiza a propagação do fogo propriamente dita, baseada no Modelo de Rothermel, e devolve os perímetros, que são depois pós-processados e armazenados.

O Serviço de Simulação de Incêndios (DT4MOB) foi construído em Python 3.11, com um motor ForeFire em C++ responsável pela propagação de incêndios florestais usando o Modelo de Rothermel. Depende de bibliotecas geoespaciais como GDAL, rasterio, pyproj, shapely e NetCDF4 para o processamento de rasters, transformações de coordenadas, e modelação da paisagem. Usa também WebSockets e httpx para a integração com o Eclipse Ditto, Keycloak para autenticação (OAuth2/OIDC), e boto3 para a interação com armazenamento compatível com S3 (SeaweedFS/MinIO). Ferramentas adicionais, como trimesh, scipy e Pydantic, são usadas para exportação 3D, interpolação, e validação de dados.

5.10 Historical Data API

A API de Dados Históricos do DT4MOB está implementada em Python 3.13 e usa o FastAPI como a sua principal framework web, para expor uma interface REST e gerar automaticamente a documentação OpenAPI/Swagger. A persistência de dados é gerida através do TimescaleDB (uma extensão do PostgreSQL otimizada para dados de séries temporais), com o SQLAlchemy e o SQLAlchemychemy a fornecerem uma camada ORM para as interações com a base de dados. A estrutura do código, a gestão de configurações e a validação de dados são geridas com o pydantic-settings e o Pydantic.

A autenticação e autorização são implementadas usando o `FastAPI-OIDC`, integrado com o Keycloak como fornecedor de identidade. O controlo de acesso baseia-se em permissões por função (role-based), incluindo funções separadas para leitura e escrita de dados históricos.

Do ponto de vista da base de dados, o sistema recorre a hypertables do TimescaleDB para armazenar e consultar eficientemente os fluxos de eventos do Eclipse Ditto, sendo o `psycopg[binary]` usado como driver do PostgreSQL. O serviço é empacotado e gerido usando o `uv`, para a gestão de dependências e do ambiente Python.

5.11 Historical Data Writer

O serviço Data Writer está implementado em Python e é construído em torno de uma arquitetura de streaming baseada em Kafka, para ingerir eventos do Eclipse Ditto e persisti-los numa base de dados de séries temporais TimescaleDB. Usa o `confluent-kafka` como cliente consumidor principal, para fazer polling contínuo de mensagens dos tópicos Kafka, decodificá-las, e encaminhá-las através de um pipeline de processamento.

O `Pydantic` é usado para tipar o código e validar as mensagens provenientes do broker, e o `pydantic-settings` é usado para carregar as configurações utilizadas pelo serviço. O serviço transforma as mensagens brutas do protocolo Ditto em objetos `DittoEvent` estruturados, através de uma camada dedicada de processamento de mensagens.

Para a persistência, o sistema recorre ao TimescaleDB, sendo o acesso à base de dados gerido pelo `SQLAlchemy`, combinado com o `SQLModel`, de forma a fornecer uma camada ORM tipada para escrever de forma segura na base de dados. A conectividade com o PostgreSQL é implementada usando o driver `psycopg`.

O projeto usa o `uv` como gestor de pacotes e de ambiente Python.

5.12 Base de dados de dados históricos

A base de dados histórica é construída sobre o PostgreSQL, um sistema de base de dados relacional de código aberto, altamente extensível e versátil. Sobre este, é usada a extensão TimescaleDB para transformar o PostgreSQL numa base de dados de séries temporais, permitindo um armazenamento e uma consulta eficientes de grandes volumes de dados temporais. Esta combinação melhora significativamente tanto o desempenho das consultas como o débito de inserção (insert throughput) para conjuntos de dados indexados por tempo, tais como os históricos de eventos do Eclipse Ditto.

5.13 Garbage Collector

O projeto é construído usando Python 3.13 e gerido com `uv`. Integra-se com o Eclipse Ditto usando tanto APIs REST como WebSocket, e usa OIDC para autenticação com tokens JWT. As principais bibliotecas Python usadas são o `AsyncIO` para execução assíncrona, o `httpx` para pedidos HTTP, o `WebSockets` para comunicação via WebSocket, e o `Pydantic` com o `pydantic-settings` para gestão de configurações e validação de dados.

5.14 Implementação e orquestração

Para simplificar o desenvolvimento, a implementação e a orquestração dos diferentes serviços, é usado um pipeline DevOps baseado em Docker, Kubernetes e Helm.

O Docker é usado para empacotar cada serviço num contentor leve, contendo todas as dependências obrigatórias. Isto permite também o versionamento, mantendo diferentes imagens para as versões de desenvolvimento e de produção. O Kubernetes atua como a plataforma de orquestração, gerindo a implementação, scaling, e o ciclo de vida destes serviços containerizados. Para melhorar a reprodutibilidade e a gestão da infraestrutura, o Helm é usado para empacotar e implementar recursos Kubernetes através de Helm Charts, permitindo que a configuração seja facilmente personalizada usando o ficheiro values.yaml.

5.15 Simulação da Faixa Reservada A5

O serviço foi desenvolvido em Python 3.13, utilizando o FastAPI para a implementação da API REST e da interface web, permitindo o processamento assíncrono de pedidos de simulação. A validação dos dados de entrada, das respostas e da configuração da aplicação é efetuada com recurso às bibliotecas Pydantic e pydantic-settings.

A simulação microscópica de tráfego utiliza Eclipse SUMO (Simulation of Urban Mobility), e o TraCI (Traffic Control Interface) permite controlar a execução da simulação em tempo real, monitorizar os veículos e implementar a lógica de acesso dinâmico à faixa reservada.

A rede viária é obtida a partir de dados do OpenStreetMap (OSM), acedidos através da Overpass API e convertidos para um formato compatível com o SUMO utilizando a ferramenta netconvert. O tráfego é reconstruído a partir dos registos históricos de veículos e processada com os scripts randomTrips.py e routeSampler.py, responsáveis pela geração e amostragem das rotas utilizadas na simulação.

A execução paralela dos diferentes cenários é suportada pelo módulo multiprocessing de Python, permitindo que cada cenário seja executado numa instância independente do SUMO. A gestão das tarefas assíncronas é efetuada através da biblioteca asyncio, possibilitando a submissão de múltiplos pedidos.

A gestão das dependências do projeto é realizada com o uv, e a contentorização da aplicação é suportada pelo Docker. A implementação em Kubernetes é através de Helm, permitindo configurar e instalar o serviço consistentemente em diferentes ambientes de execução.

5.16 Simulação VCI

A plataforma de simulação foi desenvolvida recorrendo à linguagem Python, responsável pela automatização da geração de rotas, execução das simulações e processamento dos resultados. A simulação microscópica do tráfego é realizada através do SUMO (Simulation of Urban MObility), utilizando uma rede rodoviária importada do OpenStreetMap, detetores de indução e ficheiros de rotas calibradas. Para a geração das rotas foram utilizadas as ferramentas randomTrips.py, responsável pela criação de viagens aleatórias, e routeSampler.py, que calibra essas rotas de acordo com os fluxos observados nos detetores. A biblioteca sumolib foi utilizada para interação com o SUMO, enquanto as bibliotecas subprocess e multiprocessing permitiram, respetivamente, executar automaticamente as ferramentas do simulador e paralelizar múltiplas simulações. Todo o ambiente de execução encontra-se containerized em Docker.

5.17 Modelos de Previsão de Tráfego

A pipeline de treino foi desenvolvida em Python, utilizando um conjunto de bibliotecas especializadas para processamento de dados e aprendizagem automática. O Pandas e o NumPy são utilizados para manipulação, transformação e análise dos dados de tráfego e meteorologia.

Os modelos Random Forest recorrem ao Scikit-learn, enquanto o modelo XGBoost utiliza a biblioteca homónima. A implementação da rede neuronal LSTM baseia-se no TensorFlow/Keras, e o modelo SARIMA é disponibilizado pela biblioteca Statsmodels.

A otimização automática dos hiperparâmetros é efetuada com Optuna, recorrendo a mecanismos de pruning e early stopping, e todas as experiências são registadas através do MLflow, permitindo armazenar parâmetros, métricas, modelos e artefactos produzidos. A pipeline é executada em Docker, com o Docker Compose utilizado para orquestrar simultaneamente o servidor MLflow e o treino.

5.18 Modelo de agentes

O modelo de agentes foi desenvolvido em Python 3.12. A orquestração dos agentes é realizada através do Goose, um framework para sistemas multiagente, responsável por coordenar o agente principal e os subagentes, gerir o contexto da execução e efetuar chamadas às ferramentas disponibilizadas através do protocolo Model Context Protocol (MCP).

O servidor MCP foi implementado com a biblioteca FastMCP, que permite expor funções Python como ferramentas acessíveis aos agentes. Estas ferramentas incluem a obtenção dos dados históricos de tráfego e meteorologia, execução dos modelos preditivos, comparação entre previsões e valores observado.

Para a recolha de informação são utilizadas chamadas assíncronas à plataforma Eclipse Ditto, implementadas com a biblioteca HTTPX, permitindo obter os fluxos históricos de tráfego, dados meteorológicos e informação sobre eventos relevantes. O pré-processamento das variáveis de entrada é realizado com Pandas e NumPy, enquanto as previsões são produzidas por modelos XGBoost previamente treinados e armazenados no repositório Amazon S3. Durante a execução, os artefactos necessários de cada modelo (modelo, scalers e restantes ficheiros auxiliares) são obtidos a partir do S3.

Para monitorização das execuções foi utilizado o MLflow, responsável pelo registo de métricas, parâmetros, artefatos e resultados produzidos pelos agentes ao longo de cada execução, bem como pela recolha de *traces*. A orquestração dos diferentes serviços é realizada através do Docker Compose.

O modelo de linguagem utilizado pelos agentes é disponibilizado através de uma API compatível com o padrão OpenAI, numa instância OpenWebUI, permitindo a utilização de diferentes LLMs, como Gemma 4.

6 MÉTRICAS DE QUALIDADE

Para garantir a fiabilidade e a precisão dos modelos analíticos que sustentam o gémeo digital, é estabelecido um conjunto rigoroso de métricas de qualidade. A avaliação está dividida em dois domínios principais: o desempenho preditivo dos modelos de regressão de Aprendizagem Automática (Machine Learning, ML) e a precisão estrutural das distribuições da simulação de tráfego.

Para os modelos que preveem variáveis de tráfego contínuas — como taxas de fluxo e velocidades médias em pórticos (gantries) específicos —, recorreremos a métricas de regressão padrão. Estas quantificam a magnitude do erro entre as previsões do nosso modelo e os valores realmente observados:

Erro Médio Absoluto (MAE): fornece uma medida linear da magnitude média dos erros.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Raiz do Erro Quadrático Médio (RMSE): Penaliza mais fortemente os erros maiores, o que é crucial para identificar falhas graves de previsão durante eventos de tráfego anómalos (por exemplo, acidentes).

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Coefficiente de Determinação (R²): Mede a proporção da variância na variável dependente que é previsível a partir das variáveis independentes.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Erro Percentual Absoluto Médio Simétrico (SMAPE): Dado que os volumes de tráfego flutuam significativamente entre as horas de menor movimento e as horas de ponta, os erros absolutos tradicionais podem ser enganadores. O SMAPE fornece uma medida de precisão relativa e baseada em percentagem, que lida bem com a variação de escala. Ao contrário do MAPE padrão, o SMAPE é simétrico; penaliza igualmente as subestimações e as sobrestimações, dividindo o erro absoluto pela média entre o valor real e o valor previsto. Esta limitação probabilística torna-o numa excelente métrica para avaliar de forma robusta os nossos modelos de fluxo de tráfego em diferentes localizações de pórticos.

$$SMAPE = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2}$$

Como a simulação de tráfego funciona de forma estocástica, com base em distribuições históricas, é necessário verificar se o resultado da simulação corresponde à forma estatística dos dados do mundo real. Para medir esta semelhança de distribuição, usamos:

Divergência de Kullback-Leibler (KL): Mede o quanto uma distribuição de probabilidade P se desvia de uma segunda distribuição de probabilidade esperada, Q .

$$D_{KL}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

Distância de Wasserstein (Earth Mover's Distance): Avalia o "custo" mínimo necessário para transformar a distribuição de tráfego simulada na distribuição de tráfego do mundo real, fornecendo uma métrica estável mesmo quando as distribuições não se sobrepõem perfeitamente.

$$W_1(P, Q) = \inf_{\gamma \in \Gamma(P, Q)} \int_{\mathcal{X} \times \mathcal{X}} d(x, y) \, d\gamma(x, y)$$

6.1 Simulação de tráfego VCI

A precisão da simulação foi avaliada comparando as contagens horárias de tráfego registradas pelos detetores virtuais com as medições históricas dos detetores.

O desempenho foi avaliado usando o Erro Médio Absoluto (MAE), a Raiz do Erro Quadrático Médio (RMSE), o Erro Percentual Absoluto Médio Simétrico (SMAPE), e o coeficiente de determinação (R^2). A eficiência computacional foi também considerada através do fator de execução da simulação reportado pelo SUMO, permitindo um equilíbrio entre precisão e tempo de execução.

A simulação selecionada, tendo em consideração simultaneamente as métricas de desempenho e a eficiência computacional, apresentou os resultados resumidos na Tabela 33. As métricas MAE, RMSE, SMAPE e R^2 correspondem à média dos valores obtidos para todos os detetores considerados na calibração. A gama de valores dos fluxos médios horários nos detetores está compreendida entre 221 e 5 036 veículos por hora, com uma média global de 2 893 veículos por hora.

Fator	MAE	RMSE	SMAPE	R^2
6.34	482.81	683.23	0.1538	0.8134

Tabela 33 - Resultados da simulação selecionada

6.2 Modelos de ML de tráfego

A validação do modelo segue uma abordagem de *cross validation* para séries temporais, seguida de uma validação final contra um conjunto de teste independente (held-out test set).

O desempenho dos modelos foi avaliado através do Erro Quadrático Médio (MSE), da Raiz do Erro Quadrático Médio (RMSE), do Erro Médio Absoluto (MAE), do Erro Percentual Absoluto Médio Simétrico (SMAPE) e do coeficiente de determinação (R^2). As métricas MSE, RMSE, MAE e SMAPE devem ser minimizadas, enquanto valores de R^2 mais próximos de 1 indicam um melhor ajuste do modelo aos dados.

A Tabela 34 apresenta os resultados médios obtidos durante a cross validation, enquanto a Tabela 35 apresenta o desempenho médio no conjunto de teste. Os valores apresentados correspondem à média das métricas obtidas para todos os detetores utilizados na avaliação.

Na *cross validation*, os modelos XGBoost, Random Forest e LSTM foram avaliados sobre os dados normalizados (scaled), uma vez que o treino foi realizado nessa escala. O modelo SARIMA, por não recorrer à normalização dos dados, foi avaliado diretamente sobre os valores originais (unscaled). No conjunto de teste, todas as métricas são apresentadas na escala original dos dados (unscaled), permitindo uma comparação direta entre os diferentes modelos e uma interpretação dos erros em termos das contagens reais de tráfego. Considerando os valores originais dos fluxos, os dados utilizados na *cross-validation* apresentam valores compreendidos entre 0 e 7 332 veículos por hora, com uma média de 2 726 veículos por hora. Na escala normalizada utilizada pelos modelos XGBoost, Random Forest e LSTM, os valores dos fluxos considerados na *cross-validation* variam entre -1,6916 e 2,3599, com uma média de 0,0108. No conjunto de teste, os fluxos variam entre 2 e 7 334 veículos por hora, com uma média de 2 787 veículos por hora.

Modelo	MSE ↓	RMSE ↓	MAE ↓	SMAPE ↓	R ² ↑
XGBoost	0.0357	0.1847	0.1155	0.2195	0.9645
Random Forest	0.0368	0.1866	0.1172	0.2160	0.9636
LSTM	0.0532	0.2219	0.1470	0.2670	0.9471
SARIMA	1 038 131	821.54	597.08	0.3057	0.6431

Tabela 34 - Resultados da simulação selecionada. Desempenho no *cross validation*.

Modelo	MSE ↓	RMSE ↓	MAE ↓	SMAPE ↓	R ² ↑
Random Forest	65 935.74	254.83	149.15	0.0655	0.9736
XGBoost	69 132.11	260.34	156.93	0.0700	0.9726
LSTM	75 122.19	270.04	168.83	0.0830	0.9704
SARIMA	1 872 019.34	981.99	765.51	0.4051	0.4353

Tabela 35 - Desempenho no conjunto de teste.

Observa-se que os modelos baseados em árvores de decisão (Random Forest e XGBoost) apresentaram o melhor desempenho, seguidos da LSTM, com um desempenho ligeiramente inferior. O modelo SARIMA apresentou um desempenho significativamente inferior aos restantes modelos. Este resultado poderá dever-se às limitações impostas na sua configuração. Devido ao elevado custo computacional do treino, o modelo foi limitado a uma sazonalidade de 24 períodos, enquanto os restantes modelos puderam explorar um histórico temporal mais alargado, o que poderá ter condicionado a sua capacidade de captar padrões de tráfego mais complexos.

6.3 Simulação da Faixa Reservada A5

A avaliação da solução baseia-se nos indicadores produzidos pela própria simulação, o número total de veículos simulados, a taxa de utilização da faixa reservada, a velocidade média por faixa e

a receita estimada da portagem. Estas métricas permitem analisar simultaneamente o desempenho operacional e a viabilidade económica da estratégia proposta, possibilitando a comparação objetiva entre diferentes níveis de procura, preços de portagem e percentagens de veículos aderentes ao sistema.

6.4 Simulação de propagação de incêndios

A simulação de propagação de incêndios baseia-se no simulador ForeFire, pelo que todas as métricas de qualidade são as do ForeFire.

Para avaliar a precisão do modelo na previsão do movimento do fogo, é tipicamente utilizada a superfície queimada simulada em comparação com a superfície queimada observada. A descrição das métricas utilizadas é apresentada abaixo.

- **Índice de Similaridade de Sørensen** - Mede a sobreposição espacial entre as áreas queimadas simuladas e observadas finais, penalizando fortemente falsos positivos e falsos negativos.
- **Coeficiente de Similaridade de Jaccard** - Semelhante ao Sørensen, avalia a interseção sobre a união das áreas queimadas. Uma pontuação de 1.0 representa sobreposição perfeita.
- **Estatística Kappa** - Mede a frequência com que a simulação concorda com a observação (P_a), ajustada para a probabilidade de a concordância ter ocorrido ao acaso (P_e).
- **Concordância do Tempo de Chegada (Arrival Time Agreement - ATA)** - Avalia a dinâmica temporal, calculando o erro entre quando o modelo previu que um local iria queimar e quando efetivamente queimou.
- **Concordância de Forma (Shape Agreement - SA)** - Avalia o quão bem o contorno geral e a dinâmica da forma da simulação corresponderam à realidade ao longo do tempo.

A tabela abaixo demonstra a robustez do ForeFire através dos estudos de Filippi et al. (2014)¹ e Hackett et al. (2025)².

Referência do Estudo	Contexto da Avaliação	Métrica / Parâmetro	Desempenho do ForeFire	Interpretação da Robustez
Filippi et al. (2014)	Precisão no mundo real: Testado contra 80 incêndios no Mediterrâneo utilizando dados automatizados de “primeira estimativa”.	Capacidade de Detecção (<i>Frequência com que um local queimado foi previsto corretamente</i>)	0.41 (Balbi não-estacionário)	Maior precisão preditiva. Superou o modelo de Rothermel (0.36) e o modelo empírico de 3% (0.11).

¹ Filippi, J.-B., Mallet, V., and Nader, B. (2014). “Evaluation of forest fire models on a large observation database.” *Natural Hazards and Earth System Sciences*, 14, 3077–3091. DOI: 10.5194/nhess-14-3077-2014

² Hackett, C., de Andrade Moral, R., Misra, G., McCarthy, T., and Markham, C. (2025). “An efficient method to simulate wildfire propagation using irregular grids.” *Natural Hazards and Earth System Sciences*, 25, 2909–2928. DOI: 10.5194/nhess-25-2909-2025

Filippi et al. (2014)	Idem ao anterior.	Probabilidade de Queima (<i>Fiabilidade de uma previsão corresponder à realidade</i>)	0.29 (Balbi não-estacionário)	Altamente fiável em comparação com modelos empíricos de base (0.11), próximo de Rothermel (0.31).
Filippi et al. (2014)	Idem ao anterior.	Classificação de Erro Relativo	1º lugar	Menores erros espaciais e de forma ao longo de 80 casos, provando elevada robustez do solucionador físico.
Hackett et al. (2025)	Referência Computacional: 20 incêndios simulados com terrenos e condições de vento variadas (Irlanda).	Resolução Espacial	Plano contínuo (Alta precisão)	O ForeFire é a “ verdade absoluta ” (<i>ground truth</i>) pois o seu solucionador de espaço contínuo evita erros geométricos de grelhas.
Hackett et al. (2025)	Idem ao anterior.	Pontuação Média de Ameaça (<i>Semelhança de novos modelos face ao ForeFire</i>)	0.80 (Novo modelo IGS vs ForeFire)	O output do ForeFire é o padrão-ouro de robustez (1.0/perfeito); novos softwares validam-se tentando alcançá-lo.
Hackett et al. (2025)	Idem ao anterior.	Estabilidade Determinística	Zero desvio (Output exato por execução)	O ForeFire provou ser 100% determinístico , produzindo linhas de fogo matemáticas exatas em todas as execuções.

Tabela 36 - Robustez do ForeFire

7 LICENÇAS UTILIZADAS

Esta seção descreve os detalhes relativos ao licenciamento dos componentes de software integrados no projeto DT4MOB, conforme ilustrado em:

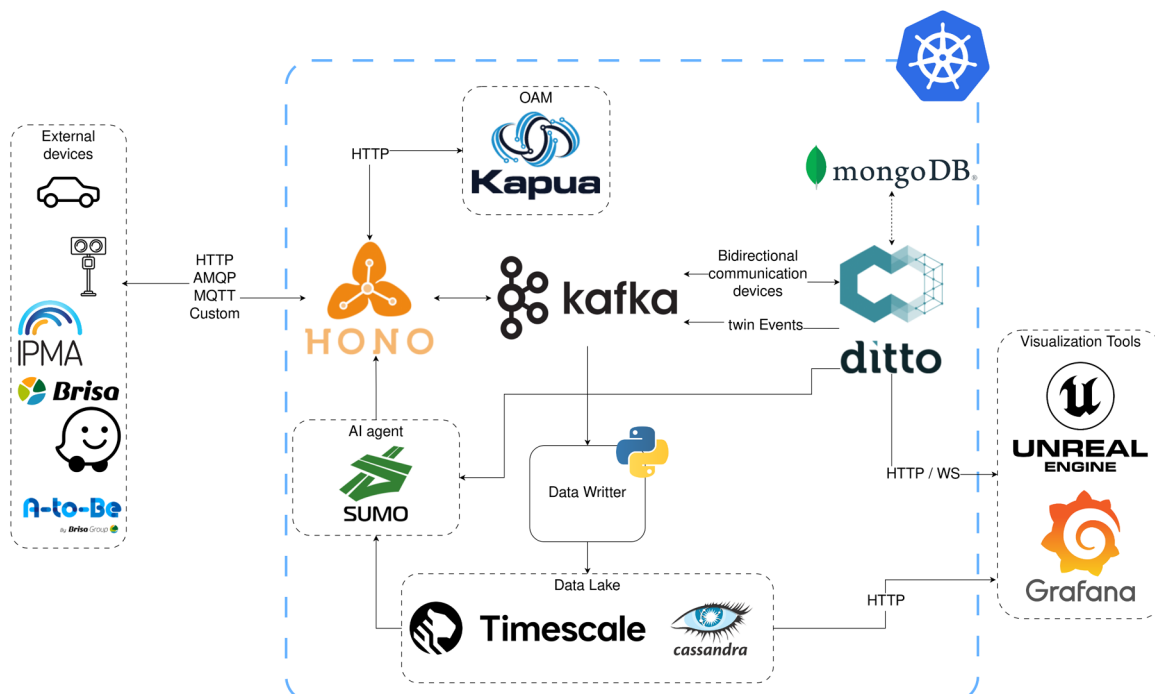


Figura 47 - Diagrama de implementação e componentes de software adotados

Kubernetes - Apache License 2.0.

Limites comerciais: Completamente permissiva. Não impõe quaisquer obrigações de copyleft, permitindo que as organizações implementem redes de contêineres proprietárias ou empacotem distribuições empresariais personalizadas de Kubernetes para uso comercial.

HTTP / AMQP / MQTT - Sem licença de software (Padrões Abertos via IETF e OASIS).

Limites comerciais: Padrões públicos isentos de royalties. As bibliotecas de código que os implementam têm as suas próprias licenças de software, mas a utilização dos protocolos de comunicação não implica custos ou obrigações legais.

Eclipse Hono / Eclipse Kapua / Eclipse Ditto / Eclipse SUMO - Eclipse Public License 2.0 (EPL-2.0).

Limites comerciais: Copyleft fraco. É possível executar estes binários comercialmente e interagir com as suas APIs em sistemas empresariais de código fechado, sem expor o código-fonte da aplicação. Contudo, se modificar diretamente o código C++ central do SUMO ou os ficheiros-fonte Java do Hono, Kapua ou Ditto, e distribuir os binários resultantes, essas modificações específicas do código-fonte devem ser publicadas sob a EPL-2.0.

Apache Kafka / Apache Cassandra - Apache License 2.0.

Limites comerciais: Open-source de alta permissividade. As organizações podem livremente empacotar, modificar ou estender estas plataformas para implementações comerciais. Não existem requisitos de copyleft para partilhar modificações.

Grafana - GNU Affero General Public License v3 (AGPLv3).

Limites comerciais: Copyleft de rede forte. Modificar o código-fonte principal do Grafana e alojá-lo numa rede para acesso de clientes terceiros exige tornar o código-fonte modificado disponível a esses utilizadores da rede. As comunicações padrão via API ou configurações de dashboards não desencadeiam este requisito.

MongoDB Server Side Public License (SSPL v1).

Limites comerciais: Disponível em código-fonte, mas estritamente não open-source. Pode ser usado como backend de aplicação padrão gratuitamente. Contudo, o MongoDB não pode ser oferecido como serviço na cloud gerido a terceiros, a menos que toda a camada de orquestração de gestão da cloud seja também disponibilizada em código aberto sob a SSPL.

TimescaleDB - Dupla licença (Apache 2.0 e Timescale License / TSL).

Limites comerciais: Os elementos principais de séries temporais estão sob a licença permissiva Apache 2.0. As extensões avançadas de desempenho (compressão, clustering multi-nó) operam sob a TSL, que proíbe expressamente usar o software para operar um produto comercial de "TimescaleDB-as-a-Service".

Unreal Engine - Contrato de Licença de Utilizador Final (EULA) do Unreal Engine.

Limites comerciais: A Epic Games utiliza um modelo baseado em licenças (seats) para implementações industriais não relacionadas com jogos. O software permanece gratuito para empresas que geram menos de um milhão de dólares (USD) em receita bruta anual. Para entidades empresariais que superam este limiar e desenvolvem gémeos digitais internos, configuradores personalizados ou projetos de cinema/TV, um nível de subscrição por licença custa 2.119,29€ por licença por ano. Se uma empresa desenvolver uma aplicação de runtime personalizada que dependa de código do Unreal Engine licenciado a utilizadores finais terceiros, reverte para um modelo de royalties padrão de 5% assim que o produto ultrapassar um milhão de dólares em receita bruta acumulada. Organizações governamentais podem obter o Unreal Engine gratuitamente para uso não comercial, caso em que os limites de receita não se aplicam.

Em resumo, as licenças de todos os componentes de software utilizados no projeto DT4MOB são permissivas e apoiam o paradigma open-source. No entanto, existem algumas limitações com o Unreal Engine para empresas com receita superior a um milhão de dólares. Estas limitações não se aplicam a organizações públicas.

httpx, trimesh, xatlas, pyvista, loguru, geopy, pyproj, Pydantic, ruff e ty - Licença MIT.

Limites comerciais: Esta é uma licença completamente permissiva que impõe quase nenhuma restrição à utilização comercial. Não existem obrigações de copyleft, o que significa que a organização pode integrar sem problemas estas bibliotecas em produtos de cibersegurança ou arquiteturas de software comerciais de código fechado. O único requisito legal é manter o aviso de direitos de autor original e uma cópia do texto da licença no software distribuído ou na sua documentação. Oferece máxima liberdade para implementação comercial, sem qualquer garantia ou aceitação de responsabilidade por parte dos autores originais.

uv - Licença MIT e Apache License 2.0.

Limites comerciais: Este modelo de dupla licença oferece uma flexibilidade comercial extraordinária, permitindo que a organização adotante escolha a licença que melhor se alinha com as suas políticas de conformidade corporativa. Optar pela Licença MIT proporciona um acordo minimalista e altamente permissivo, com apenas requisitos básicos de atribuição. Optar pela Licença Apache 2.0 concede proteções explícitas contra infração de patentes e uma cláusula de retaliação de patente defensiva. Como nenhuma das opções contém obrigações de copyleft, é possível incorporar livremente esta ferramenta em ambientes empresariais proprietários sem expor código-fonte proprietário.

AsyncIO, argparse - Python Software Foundation License Version 2.

Limites comerciais: Esta licença é extremamente permissiva e concebida especificamente para regular o Python e os seus componentes oficiais da biblioteca padrão. Não contém disposições de copyleft, garantindo que entidades comerciais possam construir, implementar e monetizar aplicações proprietárias que dependam fortemente de rotinas assíncronas. Os desenvolvedores nunca são obrigados a tornar open-source qualquer lógica proprietária que ligue, importe ou se baseie no AsyncIO. Garante uma base segura, isenta de royalties e legalmente desimpedida para engenharia de software comercial.

Aiohttp, boto3 - Apache License 2.0.

Limites comerciais: Open-source de alta permissividade. As organizações podem livremente empacotar, modificar ou estender estas plataformas para implementações comerciais. Não existem requisitos de copyleft para partilhar modificações.

pillow - Licença MIT-CMU.

Limites comerciais: Esta licença opera de forma quase idêntica à Licença MIT padrão em termos de liberdades comerciais, apresentando apenas pequenas diferenças históricas na sua formulação textual, originárias da Carnegie Mellon University. É totalmente permissiva e livre de restrições de copyleft, permitindo uma utilização extensa em software empresarial de código fechado. As empresas não enfrentam barreiras legais à monetização, desde que mantenham os avisos de atribuição e de exclusão de garantia exigidos nas suas distribuições.

pandas e scipy - Licença BSD-3-Clause.

Limites comerciais: Esta licença altamente permissiva apoia uma implementação comercial direta, sem exigir que os desenvolvedores divulguem as suas modificações proprietárias. Além das obrigações padrão de manter os avisos de direitos de autor tanto em forma de código-fonte como binária, introduz uma cláusula crucial sobre limites promocionais. As entidades comerciais estão explicitamente proibidas de usar os nomes dos autores ou contribuidores originais para endossar ou promover os seus produtos proprietários sem consentimento prévio por escrito. Isto protege a reputação dos desenvolvedores originais, ao mesmo tempo que concede às empresas máxima flexibilidade técnica.

NumPy - Licença personalizada.

Limites comerciais: Esta estrutura legal personalizada opera exatamente como uma licença BSD de 3 cláusulas clássica, concedendo robustos direitos de utilização comercial e proprietária sem qualquer exposição a copyleft. As organizações podem redistribuir livremente tanto a forma de código-fonte como binária do software juntamente com as suas aplicações proprietárias. A restrição fundamental é uma política estrita de não-endosso, que impede legalmente entidades comerciais de usar o nome dos "NumPy Developers" para comercializar ou validar os seus produtos derivados. Manter a conformidade exige simplesmente transportar as cláusulas de licença originais juntamente com a distribuição e respeitar este limite promocional explícito.

TigerData (TimescaleDB) - Apache License 2.0 (núcleo) + Timescale License (funcionalidades Enterprise/Community).

Limites comerciais: A TigerData utiliza um modelo de dupla licença para separar as funcionalidades fundamentais da base de dados das suas capacidades de análise avançada. O software principal é open source sob a Apache License 2.0, o que significa que pode ser usado, modificado e distribuído em produtos comerciais sem royalties ou restrições de copyleft rígidas. Contudo, as funcionalidades avançadas de séries temporais são regidas pela Timescale License (TSL). Sob a TSL, as organizações podem usar e modificar o software para aplicações empresariais internas gratuitamente. O limite comercial crítico aplica-se se a plataforma pretender oferecer a base de dados como um serviço gerido: está legalmente restrito o uso de funcionalidades licenciadas sob a TSL para fornecer um "database-as-a-service" comercial autónomo que concorra diretamente com as ofertas de cloud da própria TigerData.

PostgreSQL - Licença PostgreSQL.

Limites comerciais: Permissiva, sem praticamente nenhuma restrição comercial. O PostgreSQL é distribuído sob a sua própria licença open-source liberal, estruturalmente semelhante às licenças MIT e BSD. O software da base de dados e a sua documentação podem ser livremente integrados, copiados, modificados e distribuídos dentro da plataforma para qualquer fim comercial ou não comercial, sem taxas de licenciamento, subscrições por licença ou royalties baseados em receita. A única obrigação legal é administrativa: o aviso de direitos de autor original e a cláusula padrão de exclusão de responsabilidade de três parágrafos devem ser preservados e incluídos em todas as cópias ou distribuições substanciais do software, o que constitui prática padrão na documentação de licenças de terceiros da plataforma.

Flask - BSD-3-Clause.

Limites comerciais: Permissiva. Altamente flexível; as alterações ao código podem permanecer proprietárias.

OpenStreetMap - ODbL 1.0.

Limites comerciais: Share-Alike (a nível da base de dados). As alterações aos dados de mapa devem ser partilhadas de volta; o código da aplicação não é afetado.

Cesium for Unreal / Cesium ion - Apache License 2.0 (plugin) + Termos de Serviço do Cesium ion (serviço na cloud).

Limites comerciais: O plugin Cesium for Unreal em si é open source sob a Apache License 2.0 e pode ser livremente usado, modificado e distribuído em qualquer produto sem obrigações de copyleft ou royalties. Contudo, a plataforma depende do Cesium ion, um serviço de cloud comercial que transmite terreno 3D, imagens e conjuntos de tiles em tempo de execução (runtime). O Cesium ion tem os seus próprios Termos de Serviço e preços de subscrição. O nível gratuito está restrito a uso não comercial e de avaliação; implementações de produção ou comerciais requerem uma subscrição paga do Cesium ion, com preços baseados no volume de transferência de dados e utilização.

GLTFRuntime - Licença MIT.

Limites comerciais: Permissiva, com requisitos mínimos. O plugin pode ser usado, modificado e distribuído em aplicações comerciais sem restrições. A única obrigação é incluir o aviso de direitos de autor original e o texto da licença em qualquer distribuição do software ou de partes substanciais do mesmo, o que normalmente é satisfeito através da inclusão no diretório de licenças do projeto.

Forefire - Licença GNU General Public License v3.0.

Limites comerciais: Copyleft, com requisitos de reciprocidade. O engine e as suas bibliotecas podem ser usados, modificados e distribuídos em aplicações comerciais, desde que os requisitos de open code sejam respeitados. A principal obrigação é disponibilizar o código-fonte de qualquer modificação ou trabalho derivado sob a mesma licença GPL-3.0, garantindo a inclusão do aviso original de direitos de autor, o texto da licença e a transparência do código em qualquer distribuição do software.